Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Kevin Kudakwashe Murera

Sunday 23rd August, 2026

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Kevin Kudakwashe Murera

Day of issue	:	TODO: date of issue
Day of release	:	TODO: date of submission

Reviewer	:	Prof. Dr. Karsten Berns
Supervisor	:	TODO: supervisor

Hereby I declare that I have self-dependently composed the Master Thesis at hand. The sources and additives used have been marked in the text and are exhaustively given in the bibliography.

Sunday 23rd August, 2026 – Kaiserslautern

(Kevin Kudakwashe Murera)

Acknowledgements

TODO: acknowledgements.

Abstract

Hydraulic manipulators are difficult to control precisely, because the actuator response is nonlinear, depends on load and supply pressure, and varies between individual machines. Simulating that response faithfully is possible, but the models that do so are far too slow to be evaluated inside reinforcement learning. Ignoring the hydraulics and commanding joint positions directly avoids that cost, but discards the behaviour that makes the machine difficult to control in the first place.

This thesis investigates how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation, and used for reinforcement-learning control of a verge mowing arm. Two actuator representations are identified from randomised simulation campaigns. The first is a modular chain of an inverse model, which maps a desired cylinder motion to a valve command, and a forward model, which predicts the resulting response. The second is a physics-structured reduced-order model, in which pressure, force and extension propagation remain explicit while learning covers valve realisation and unmodelled loading. Both representations are realised in the deployment simulator and used as the actuation interface for policy learning. A damped least-squares controller provides privileged demonstrations during training and is removed afterwards, so that the distilled policy becomes the sole deployed controller.

The inverse and forward representation is shown to transfer to the deployment simulator without deployment-specific retraining, while the reduced-order model is transferred with a small bounded calibration against the deployment plant. Supervised accuracy is shown to be an unreliable guide to control, because a forward model achieved a high predictive score while learning to ignore the valve command. Excitation design and input ablation are established as the tests that expose this failure. Both representations then support closed-loop terrain following, and the choice of representation is shown to change the control strategy that the policy learns. Online adaptation is examined last, against the robustness already obtained. Plant identity proves recoverable only from deliberate excitation and not from the data the task controller produces, and the trained stacks absorb the plant variation studied here without assistance, which limits the benefit that conditioning on plant identity can provide.

Contents

1 Introduction **1**

1.1 The Role of Mechanised Verge Maintenance 1

1.2 Advances in Learning-Based Control for Hydraulic Manipulators 1

1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms 2

1.4 Contribution of This Thesis 3

1.5 Thesis Structure 5

2 Fundamentals and Related Work **9**

2.1 Manipulator Actuation and Control Fundamentals 9

2.1.1 Application Context and Implement Carriers 9

2.1.2 The Electro-Hydraulic Actuation Chain 10

2.1.3 High-Fidelity Simulation as a Data Source 10

2.2 Kinematics and Task-Space Tracking 11

2.2.1 Differential Kinematics 11

2.2.2 From Joint Space to Cylinder Space 11

2.3 Model-Based and Conventional Control Approaches 11

2.4 Data-Driven System Identification and Learned Dynamics 12

2.4.1 Learned Forward and Inverse Dynamics 12

2.4.2 Closed-Loop Identifiability 12

2.5 Reduced-Order Modeling 12

2.6 Domain Randomization and Latent Adaptation 13

2.6.1 Robustness Through Randomized Training 13

2.6.2 From Robustness to Adaptation 14

2.7 Reinforcement Learning and Structured Priors 14

2.7.1 Policy Optimization 14

2.7.2 Structured Priors for Policy Learning 15

2.7.3 Reinforcement Learning on Hydraulic Machines 15

2.8 Research Gap and Thesis Positioning 15

3 System Fundamentals and Problem Formulation 18
3.1 Manipulator Platform . . . 18
3.2 Kinematic Structure and Coordinate Mappings . . . 20
3.2.1 Joint Space and Cylinder Space . . . 20
3.2.2 Task Space and the Calibrated Cutter Jacobian . . . 21
3.3 Actuation and Measured State . . . 21
3.3.1 Sensing and Observability . . . 21
3.4 High-Fidelity Simscape Model . . . 23
3.4.1 The Command ROM as Deployment Actuator . . . 23
3.4.2 Mechanical Subsystem . . . 24
3.4.3 Hydraulic Subsystem . . . 24
3.4.4 Validated Scope of the Model . . . 25
3.5 IsaacLab Environment . . . 26
3.5.1 Simulation and Control Rates . . . 26
3.5.2 Environment Configuration Details . . . 26
3.6 The Verge-Following Task . . . 26
3.6.1 Task Definition . . . 26
3.6.2 Task Feasibility and Plant Capability . . . 27
3.7 Control Architecture . . . 27
3.8 Problem Formulation . . . 28
3.9 System-Level Evaluation Criteria . . . 28
3.9.1 Eight-Gate Acceptance Contract . . . 29
3.9.2 Cylinder and Model Checks . . . 29

4 Data Generation and Preparation 31
4.1 Dataset Objectives and Design Requirements . . . 31
4.2 Randomized Parameter Space of the Simscape Campaign . . . 33
4.2.1 Sampling Design . . . 33
4.2.2 Hydraulic Parameters . . . 33
4.2.3 The Compressibility Axis . . . 34
4.2.4 Mechanical Parameters . . . 34
4.3 Closed-Loop Data Collection . . . 35
4.3.1 The Controller as a Data-Collection Instrument . . . 35
4.4 Open-Loop Excitation Experiments . . . 35
4.4.1 Limitations of the Closed-Loop Batch . . . 35
4.4.2 Design of the Excitation Signal . . . 36
4.4.3 Plant Draws and Acceptance . . . 37
4.5 Combining Closed-Loop and Open-Loop Datasets . . . 37
4.6 Closed-Loop Calibration Data for the Command ROM . . . 38
4.7 The IsaacLab Adaptation Collection . . . 39
4.7.1 Provenance of the Adaptation Data . . . 39
4.7.2 The Physical Factor Vector . . . 39
4.7.3 Domains and Factor Design . . . 39
4.7.4 Collection Procedure . . . 40
4.7.5 The Frozen Dataset . . . 42
4.7.6 Learning Timebase and Hydraulic State . . . 42
4.8 Feature and Target Construction . . . 42
4.8.1 The Actuator Models . . . 42
4.8.2 Observable History of the Adaptation Models . . . 43
4.9 Preprocessing and Data Splits . . . 44
4.10 Dataset Diagnostics . . . 44

5 Learned Models and Online Adaptation 47
5.1 Modelling Overview and Common Notation . . . 47
5.2 Inverse Dynamics Model . . . 49
5.2.1 Paired Control Fine-Tuning . . . 49
5.2.2 Transfer of the Frozen Inverse Model . . . 50
5.3 Forward Dynamics Model . . . 51
5.3.1 Assessment of the Supervised Score . . . 51
5.4 Reduced-Order Model . . . 52
5.4.1 Closed-Loop Calibration Against the Deployment Plant . . . 55
5.5 The Privileged Latent and Its Limits . . . 56
5.6 Arm Adaptation Module . . . 57
5.6.1 What the Latent Actually Encodes . . . 58
5.7 Latent-Conditioned Inverse-Model Adapter . . . 59
5.7.1 Open-Loop Evidence . . . 59
5.7.2 Closed-Loop Evidence, and Why It Is Null . . . 59
5.7.3 The Chain Verdict . . . 60
5.8 Training and Model-Selection Protocol . . . 60

6 Control Framework and Simulation Integration 63
6.1 The DLS Teacher and Its Two Roles . . . 63
6.1.1 Scope and Role of the Controller . . . 63
6.1.2 Privileged Information . . . 63
6.1.3 From Task Error to Cylinder Command . . . 64
6.1.4 Execution Through the Student Interface . . . 65
6.1.5 Teacher Performance . . . 66
6.2 Policy Formulation . . . 66
6.2.1 Policy Definition . . . 66
6.2.2 The Observation Vector . . . 67
6.2.3 The Action and Its Bounds . . . 68
6.2.4 The Reward Function . . . 69
6.2.5 Optimization . . . 71
6.3 Training Protocol and the Distillation Finding . . . 72
6.3.1 Distillation by DAgger . . . 72
6.3.2 Objective Mismatch in Unconstrained Fine-Tuning . . . 72
6.4 Reference-Anchored PPO . . . 73
6.4.1 Critic Warm-Up . . . 74
6.4.2 The Reference Anchor . . . 74
6.4.3 Trust Regions and Rollback . . . 74
6.4.4 The Block Ratchet . . . 75
6.4.5 Theoretical Status . . . 75
6.5 R: Reference Actuator Baseline . . . 77
6.6 IF: Non-Adaptive Learned Baseline . . . 78
6.7 AIF: The Adaptive Stack . . . 78
6.8 IsaacLab Integration and Timing . . . 80
6.9 Constraints and Fair-Comparison Rules . . . 80
6.9.1 Limits and Safeguards . . . 80
6.9.2 Matching Rules for the Three-Stack Comparison . . . 81

7 Experimental Evaluation and Discussion 84
7.1 Hypotheses and Fairness Controls . . . 84
7.2 Excitation Design and the Identifiability Envelope . . . 85
7.3 Sufficiency of the Frozen Inverse Model . . . 87
7.4 Anatomy of the Adaptive Latent . . . 88
7.5 Latency, from the Valve to the Tool . . . 90
7.5.1 Dead Time in the Plant . . . 90
7.5.2 The Chain Compared Across the Three Stacks . . . 91
7.5.3 Step Response to a Commanded Change . . . 93
7.5.4 The Latency Budget of the Closed Loop . . . 94
7.6 Checkpoint Selection and the Scoring Horizon . . . 95
7.7 End-Effector and Mowing Performance . . . 96
7.7.1 Control Effort . . . 98
7.8 Response to a Mid-Episode Plant Change . . . 99
7.9 An Oracle Reference for Plant-Identity Conditioning . . . 101
7.10 Latent Bandwidth on the Policy Path . . . 102
7.11 Robustness to Plant Variation . . . 103
7.12 Statistics and Reporting . . . 104
7.13 Integrated Discussion . . . 104
7.14 Threats to Validity . . . 106

8 Conclusion and Outlook 114
8.1 What Was Established . . . 114
8.2 What This Means for Design . . . 116
8.3 Contributions . . . 117
8.4 Limitations . . . 118
8.5 Outlook . . . 118

A System Parameters and Coordinate Mappings 121
A.1 Rates . . . 121
A.2 Command-Path Limits . . . 122
A.3 Course and Episode Geometry . . . 122

B Dataset Manifests and Factor Ranges 123
B.1 Trajectories . . . 123
B.2 Execution and Logging . . . 123
B.3 Parameter Ranges and Dataset Audit . . . 124
B.4 The Frozen Adaptation Collection . . . 124

C Model Architectures, Normalizers and Checkpoint Hashes 126
C.1 The Inverse and Forward Models . . . 126
C.2 The Adaptation Module . . . 127

D PPO and IsaacLab Configuration **128**
D.1 The Observation Vector 128
D.2 The Action 128
D.3 The Reward 128
D.4 PPO Hyperparameters 130

E Supplementary Results **131**
E.1 Per-Family Results on the Reference Mesh 131
E.2 The Latent Substitution Arms 132
E.3 Reproduction 132

Bibliography **133**

1. Introduction

1.1 The Role of Mechanised Verge Maintenance

Road verges are cut because they are infrastructure. Vegetation left to grow closes sight lines at junctions and bends, obscures signs and barriers, blocks the drainage channels that keep water off the carriageway, and encroaches on the running surface itself. A national road network carries tens of thousands of kilometres of verge, and each kilometre is cut several times in a growing season, so the work is continuous, repetitive and performed almost entirely by machine.

It is also performed from a moving vehicle. Stopping to cut each section would require closing the lane, so a carrier drives the road at a few kilometres per hour while a boom-mounted mowing head works the verge alongside it. The head must hold a cut height of a few centimetres over ground that rises and falls continuously, follow a grass line that curves with the road, and do so without striking the surface hard enough to damage the tool or lift the machine.

The demands on the manipulator follow from that. The machine studied here, shown in Figure 1.1, carries a heavy rotating tool at the end of a long boom, works in intermittent contact with the ground, and must place that tool accurately while its own base is in motion. Hydraulic actuation is the standard answer at this scale: it supplies the force density and the tolerance to shock loading that electric drives do not, and it does so with components rugged enough for continuous outdoor operation. Precision, however, is not what hydraulics are good at, and that tension is the subject of this thesis.

1.2 Advances in Learning-Based Control for Hydraulic Manipulators

Two developments have changed what is achievable on machines of this kind.

The first is that policy optimisation in simulation has become practical for contact-rich manipulation. GPU-parallel physics engines run thousands of environments simultaneously, which makes reinforcement learning feasible on tasks that would take years of real-machine

experience, and randomising the simulator across a family of plants produces policies that transfer without being tuned to one machine. The second is that the actuator itself can be learned. Rather than deriving valve and cylinder behaviour from first principles and fitting its parameters, a network trained on recorded motion can represent the command-to-motion relationship directly, including the nonlinearities that analytical models approximate away.

Combining the two is attractive and not straightforward. A learned actuator model must be cheap enough to evaluate inside the training loop, which rules out the high-fidelity models used for offline analysis, and it must remain valid across the family of machines the policy will meet. This has motivated a line of work in which a compact code describing the current plant is inferred from recent interaction history and supplied to the controller, so that a single policy can serve a family rather than a single machine. Such methods report substantial gains in legged locomotion and in manipulation, and they are the direct antecedent of the architecture examined here.

Figure 1.1: MULAG FME 500 front mower on a Unimog during roadside verge maintenance. Source: [MULAG Fahrzeugwerk 26].

1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms

A valve command does not produce a proportional cylinder motion. The relationship is nonlinear, depends on load and supply pressure, and is subject to dead time between the command and any measurable response. It also differs between machines: seal condition, fluid temperature, hose compliance and the attachment fitted all change it, so two machines of the same model respond differently and one machine responds differently across a season. A controller that inverts only the geometry therefore leaves the largest source of tracking error unaddressed.

That difficulty admits two responses, and they are routinely conflated. The first is robustness: train across a randomised family so that a single fixed model is adequate everywhere. It costs nothing at run time and gives up the difference between the family and the individual. The second is adaptation: infer which member of the family is present from the interaction history and condition the model or the policy on that estimate. It promises to recover that difference, at the cost of an estimator that must work from observable signals alone, since a deployed machine has no access to its own plant parameters.

The second response is only worth pursuing under conditions that are rarely checked before the work is done. Plant identity must be recoverable from the data the machine actually produces, which depends on how the machine is being driven while that data is collected. And the difference between knowing the plant and not knowing it must be large enough to matter once the estimate reaches a feedback controller. Neither condition is guaranteed, both are specific to the task, and both can be measured before an estimator is built. Establishing those measurements, and applying them to a hydraulic mowing arm, is the problem this thesis addresses.

1.4 Contribution of This Thesis

The difficulty described in Section 1.3 is a modelling problem before it is a control problem. A hydraulic actuator can be simulated accurately, but the models that do so are too slow to sit inside a reinforcement-learning loop that requires millions of environment steps. The alternative usually taken is to discard the hydraulics and command joint positions directly, which removes the load and pressure dependence that makes the machine difficult to control. What is needed is a representation that is cheap enough to train against and faithful enough that the resulting policy is controlling something close to the real actuator.

Learned actuator models and latent-conditioned adaptation are both established techniques, and Section 1.2 described the progress that makes them practical on machines of this class. What remains unclear is how such models should be identified so that they retain the command dependence a controller needs, whether they transfer between simulators without being retrained, and whether online adaptation adds anything once a policy has been trained against them. This thesis therefore treats representation and transfer as the primary questions, and treats online adaptation as the last of them rather than the first. The research statement of the thesis is as follows:

Comparison notation.

For readability, the three policy–actuator stacks are denoted by short labels throughout this thesis: *R* is PPO–ROM, *IF* is PPO–ID–FD, and *AIF* is PPO–(AAM)–ID–FD. R uses the command-conditioned reduced-order model; IF uses the frozen inverse/forward pair; and AIF augments IF with online adaptation. The labels name the complete stacks, not just their actuator models.

To determine how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation improves on the robustness those learned models already provide.

The statement decomposes into six research questions, which are answered in the order given. The first three concern how the actuator is represented, the fourth concerns control through that representation, and the last two concern what remains for adaptation once the first four are settled. The six questions are:

1. How can data from a high-fidelity Simscape model be used to learn computationally tractable representations of hydraulic actuation while preserving the command dependence a controller requires?
2. To what extent do actuator models identified from Simscape transfer to an IsaacLab manipulator without deployment-specific retraining?
3. Can a physics-structured reduced-order hydraulic model supply dynamics that are accurate and stable enough for GPU-parallel policy training?
4. Can a policy trained from a privileged classical teacher become the sole deployed controller while acting through learned actuator dynamics?
5. How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?
6. Where plant variation survives that robustness, is it identifiable from recent observations, and does conditioning the controller on it improve closed-loop performance?

The first is answered in Chapters 4 and 5, the second and third in Chapter 5, the fourth in Chapter 6, and the last two in Chapter 7.

RQ6 is the question named in the title, and it is deliberately placed last. It is only worth asking once the preceding questions have established that the learned models transfer, that a policy can control through them, and how much plant variation the resulting closed loop already absorbs unaided. A small answer to RQ6 is then a result about the system rather than a failure of the method, because accurate models and a robust policy leave less for adaptation to recover. The work rests on the following contributions:

Transferable Hydraulic Actuator Learning: A pipeline that identifies hydraulic actuation from high-fidelity Simscape data and transfers it into IsaacLab through two representations, a modular inverse and forward chain and a physics-structured reduced-order model. The inverse model is trained on Simscape data and evaluated on deployment runs drawn entirely outside that training distribution, where it reproduces most of the valve-command variance with an accuracy that is nearly flat across plant domains. It requires no deployment-specific retraining and remains frozen in every downstream stack. The reduced-order model is transferred differently and the difference is stated wherever it matters: it receives a small bounded calibration against the deployment plant, which is why it serves as a reference condition rather than as an independent competitor to the inverse and forward pair.

Control-Relevant Identification and Validation: A demonstration that predictive accuracy can conceal the loss of exactly the property a controller needs, together with the procedure that detects it. The first forward model reached a near-perfect supervised score while learning to ignore the valve command and reconstruct motion from the state history

instead. The validation loss did not show this; ablating the command did. Redesigning the excitation and retraining recovered genuine command sensitivity, and the pairing of excitation design with input ablation is established here as a required test for learned hydraulic models intended for control rather than for prediction.

Reduced-Order Actuator Model: A command-conditioned grey-box actuator in which pressure, force, velocity and extension propagation remain explicit while learning covers valve realisation, unmodelled loading and bounded residual effects. It is accurate enough to serve as the deployment plant, stable in recursive rollout, and cheap enough to evaluate inside GPU-parallel policy training. Zeroing or shuffling its command collapses its extension and velocity accuracy, which establishes that the command authority is real rather than inferred from context.

Policy-Only Control Through Learned Hydraulics: A damped least-squares controller supplies privileged demonstrations during training and is then removed, so that the distilled policy emits desired cylinder motion directly into a learned actuator representation and constitutes the entire deployed controller. Both representations support closed-loop terrain following in this configuration, although two learned models placed in series could have compounded their errors.

Robustness and the Limits of Adaptation: A matched evaluation protocol that scores the resulting stacks on tracking, control effort, latency, disturbance rejection and sensitivity to the magnitude of plant variation, together with a procedure that decides in advance whether online plant conditioning will repay the effort. The first of its two measurements establishes whether plant identity is present in the data the machine produces. The second supplies the true plant parameters through a privileged encoder and measures what the closed loop does with them. Both precede estimator training, and either can end the effort before it is incurred.

The components on which these contributions rest, namely the causal plant estimator, the bounded adaptation path and the eight-gate evaluation harness, are described where they are used. They are technical enablers rather than separate claims.

1.5 Thesis Structure

This thesis investigates how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, grounding its investigation in the identifiability of the actuator models and in the robustness of the controllers built on them. The document is organized as follows:

Chapter 2 establishes the background against which the remainder is read. It introduces the machine class and its electro-hydraulic actuation, recalls the kinematic tools underlying the classical controller, and reviews how conventional model-based control handles hydraulic uncertainty. It then treats the learning methods this work builds on, namely data-driven dynamics models and the pitfalls of identifying them from closed-loop data, reduced-order and grey-box modelling, domain randomisation and latent adaptation, and policy optimisation with structured priors. The chapter closes by stating the gap that remains and the position of the three control stacks examined here.

Chapter 3 analyses the platform, using the MULAG FME 500 mounted on a Unimog carrier as the case study. It describes the rigid bodies and the serial chain they form, the coordinate mappings between task space and cylinder space, and the closed kinematic loops introduced by the cylinder mounts. It specifies what the controller can observe, separating measured quantities from quantities produced by a model, and presents the high-fidelity Simscape model from which the actuator data derives. The chapter also states the task, the plant capability limits and the acceptance criteria against which every later result is scored.

Chapter 4 presents the data generation strategy and answers the first research question. It describes the excitation design, the randomised factor set defining the family of plants, and the procedures that keep training, validation and reserved test configurations separate. It then reports the identifiability audit applied to the resulting collection, including the measured finding that excitation gathered under a task controller does not separate plants, and the open-loop collection that repairs this.

Chapter 5 develops the learned models and answers the first three research questions. It presents the inverse model that maps a desired cylinder motion to a valve command, the forward model that predicts the response, and the evidence that supervised accuracy alone does not establish that either model retains command authority. It then presents the command-conditioned reduced-order model that serves as the deployment plant inside GPU-parallel training. The chapter closes with the material required for the final research question, namely the privileged latent and its measured limits, the causal estimator that infers a deployable substitute from observable history, and the bounded adapter through which that estimate reaches the actuator path.

Chapter 6 describes the control framework and its integration with the simulator, and answers the fourth research question. It presents the damped least-squares teacher used for demonstration and data collection, the observation and action interfaces shared by all three stacks, the distillation of the teacher into a student policy, and the reference-anchored fine-tuning that makes further policy optimisation admissible without discarding the distilled behaviour. It then assembles the three deployed control stacks and states the rules under which they may be compared.

Chapter 7 reports the experimental evaluation and answers the last two research questions. It establishes the excitation envelope within which plant identity is recoverable, the sufficiency of the frozen inverse model, and what the inferred latent encodes. It measures latency through the actuation chain and the response of each stack to a commanded change, then compares the three deployed controllers on closed-loop tracking and control effort. It closes by measuring the disturbance envelope of each controller, an oracle reference for what plant-identity conditioning contributes within this architecture, and the sensitivity of each stack to the magnitude of plant variation.

Chapter 8 concludes the thesis. It summarises what the work establishes and what it does not, states the design implications that follow from the measured results, records the limitations of the evidence, and identifies the measurements that should be taken next.

Appendix A provides the system parameters and the coordinate mappings between task space, joint space and cylinder space referred to in Chapter 3. **Appendix B** lists the dataset manifests and the factor ranges that define the family of plants used in Chapters 4 and 7. **Appendix C** records the model architectures, the normalisers and the checkpoint

hashes for every learned model reported in Chapter 5. **Appendix D** documents the PPO and IsaacLab configuration used for the training described in Chapter 6. **Appendix E** collects supplementary results supporting Chapter 7.

2. Fundamentals and Related Work

Chapter 1 argued that precise task-space tracking with a hydraulic mowing arm is limited less by kinematics than by the actuator. The path from a valve command to actual cylinder motion is nonlinear and load dependent, and it differs from one machine, season, or attachment to the next. This chapter collects the background needed to judge that argument and to understand the design decisions made later in the thesis. Section 2.1 introduces the machine class and its electro-hydraulic actuation. Section 2.2 recalls the kinematic tools behind the classical controller, and Section 2.3 reviews how conventional model-based control deals with hydraulic uncertainty. The remaining sections cover the learning methods this work builds on. Section 2.4 treats data-driven dynamics models and the pitfalls of identifying them from closed-loop data. Section 2.5 covers reduced-order and grey-box modeling, Section 2.6 domain randomization and latent adaptation, and Section 2.7 policy optimization with structured priors. Section 2.8 then states the gap that remains and how the three control stacks of this thesis (R, IF, and AIF) address it. As defined in Chapter 1, these abbreviate PPO–ROM, PPO–ID–FD, and PPO–(AAM)–ID–FD, respectively. Only material that is needed for the later chapters is included. The concrete system definition and problem formulation follow in Chapter 3.

2.1 Manipulator Actuation and Control Fundamentals

2.1.1 Application Context and Implement Carriers

The MULAG FME 500 mounts a multi-link hydraulic boom and a 1.2 m flail head on a Unimog carrier [MULAG Fahrzeugwerk 26]. It reaches about 6.5 m laterally, below road level and over obstacles, while an operator adjusts cutter height, tilt and lateral position as the vehicle moves. Automating this continuous tracking task is the practical motivation of the thesis.

Hydraulic cylinders suit the required force, shock tolerance and existing vehicle power supply, but their command gain changes strongly with load and configuration. This trade-off between force density and precision is central to hydraulic manipulator control [Mattila 17].

2.1.2 The Electro-Hydraulic Actuation Chain

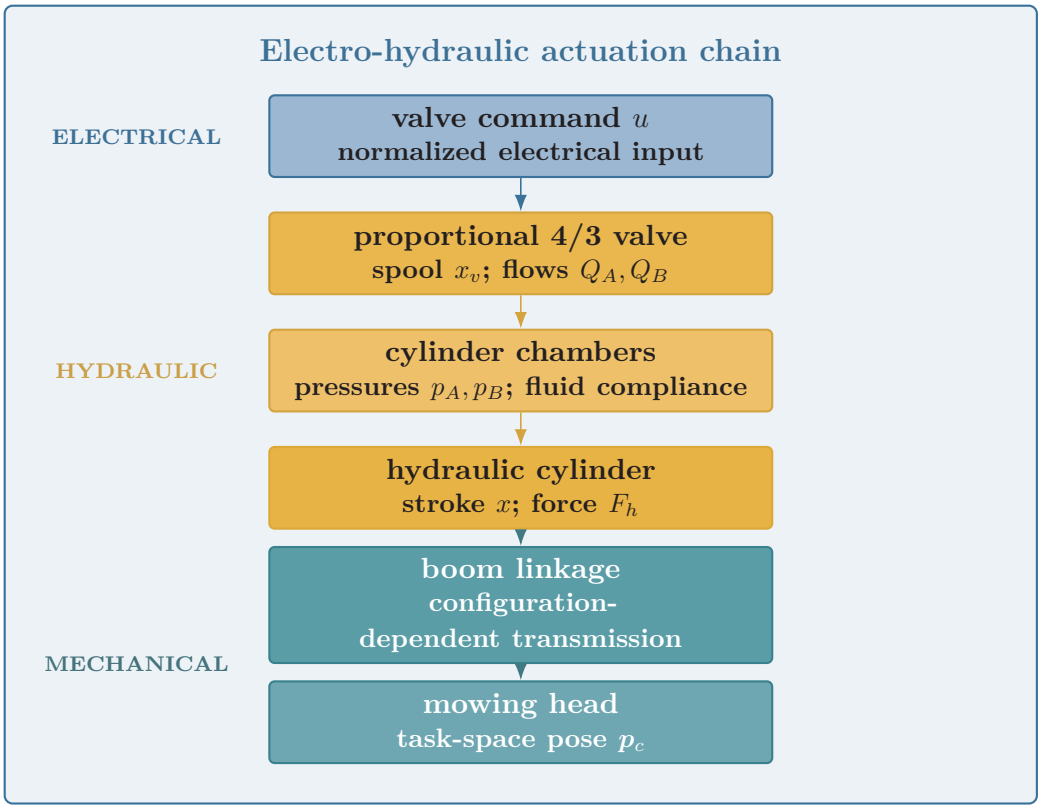

Figure 2.1: The electro-hydraulic actuation chain from valve command to mowing-head motion. The block colors distinguish the electrical, hydraulic and mechanical stages; each subtitle identifies the principal quantity passed to the next stage.

In Figure 2.1, an electrical command moves a proportional valve spool and meters flow to the cylinder chambers. The orifice law [Merritt 67] makes flow depend on spool position and the square root of the pressure margin. Valve authority is therefore load-dependent and directional; overlap creates a dead zone near null and the spool saturates at large commands [Jelali 03].

Compressibility, changing chamber volume, relief limits, leakage and seal friction further separate flow from motion. Several of these effects are non-smooth, while bulk modulus, leakage and friction drift with temperature and wear. The mechanical linkage then converts cylinder translation through configuration-dependent lever arms. Chapter 5 consequently models the hydraulic realization in cylinder space, before kinematic effects are mixed into cutter error.

2.1.3 High-Fidelity Simulation as a Data Source

The component-level Simscape plant [The MathWorks, Inc. 24a] resolves the stiff hydraulic dynamics but runs one machine at small integration steps. Policy optimization instead benefits from thousands of GPU-parallel plants [Makoviychuk 21]. The detailed model is therefore used offline to generate identification data; compact actuator models carry its behavior into parallel training, as summarized in Figure 2.2.

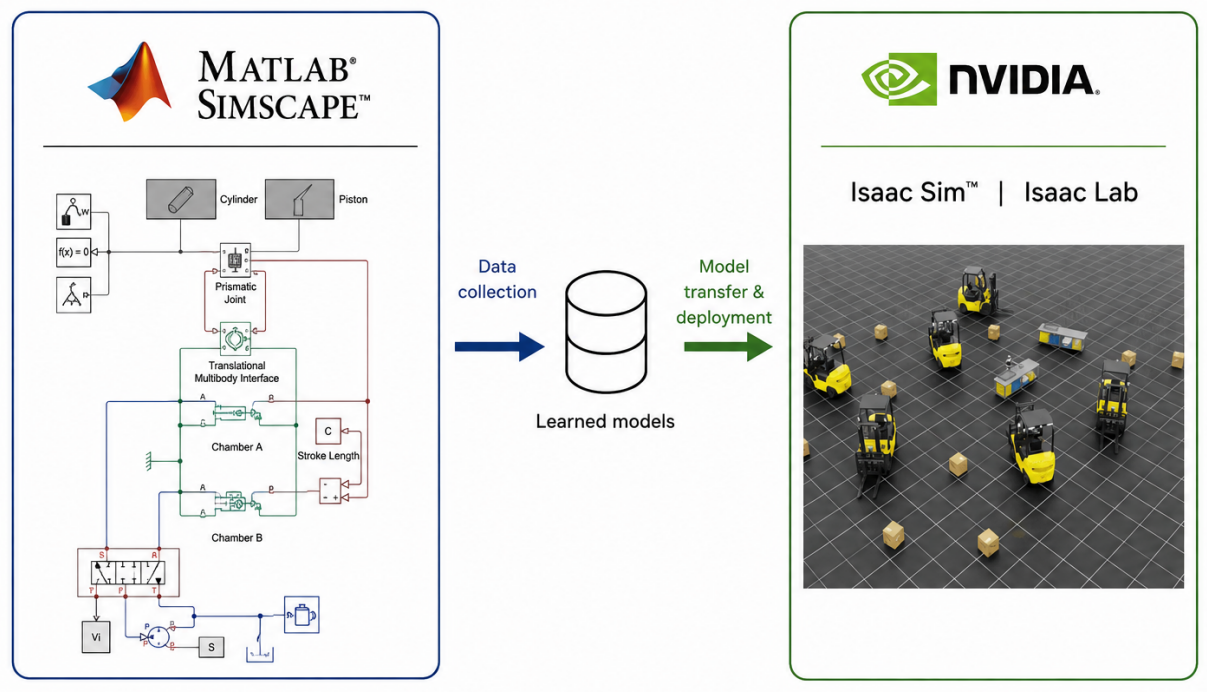

Figure 2.2: Simscape supplies high-fidelity hydraulic data; compact actuator models transfer that behavior to GPU-parallel training in IsaacLab. Product images: MathWorks and NVIDIA.

2.2 Kinematics and Task-Space Tracking

2.2.1 Differential Kinematics

The cutter path is specified in vehicle coordinates, so the manipulator Jacobian maps joint rates to the required tool velocity [Siciliano 09]. Near singular configurations the ordinary pseudo-inverse produces excessive commands. The damped least-squares inverse of Wampler [Wampler 86] and Nakamura and Hanafusa [Nakamura 86] trades some local accuracy for bounded commands and is retained as the classical controller.

2.2.2 From Joint Space to Cylinder Space

The actuated coordinates are cylinder lengths, and their nonlinear cylinder-to-joint map has a configuration-dependent lever arm. A passive head joint further limits independently controllable directions. The calibrated cutter Jacobian in Chapter 3 composes these mappings. It still assumes that commanded cylinder motion is realized; the learned actuator models address that remaining gap.

2.3 Model-Based and Conventional Control Approaches

Industrial systems commonly map joystick input directly to valve current. Fixed-gain position–velocity or position–pressure cascades improve on this practice [Jelali 03], but tuning across large changes in valve gain and linkage leverage produces either weak response or oscillation.

Nonlinear model-based control instead cancels known rigid-body dynamics [Slotine 91], estimates uncertain parameters online [Åström 95], or bounds their effect. Yang et al. [Yang 26], for example, combine state constraints with explicit valve dead-band inversion. Such methods can track accurately, but depend on valve, friction and inertia models that are costly to maintain on a commercial implement [Mattila 17].

This thesis therefore retains reliable structure and learns the uncertain part. The DLS kinematic controller acts as teacher and data-collection controller; learning covers the mapping from cylinder command to realized motion and the policy that issues that command (Section 2.7).

2.4 Data-Driven System Identification and Learned Dynamics

2.4.1 Learned Forward and Inverse Dynamics

Robot model learning distinguishes forward models, which predict the next state from state and command, from inverse models, which predict the command for a desired transition [Nguyen-Tuong 11]. Section 5.1 fixes both interfaces. One-step fit is insufficient when a forward model is recursively rolled out, because small errors compound [Nagabandi 18]. The models here are therefore trained with multi-step objectives. Probabilistic ensembles represent growing rollout uncertainty [Chua 18]; recurrent architectures instead retain unobserved pressure and valve history [Hochreiter 97, Cho 14], as used by the reduced-order model.

2.4.2 Closed-Loop Identifiability

Informative identification requires inputs that excite the plant independently of feedback [Ljung 99, Isermann 11]. In closed-loop data, command and state history are correlated; a flexible predictor can score well by ignoring the command, and validation from the same distribution will not reveal the shortcut [Schoukens 19].

That failure occurred here: a closed-loop forward model reached one-step $R^2 \approx 0.999$ yet showed almost no command dependence under ablation. Chapter 4 therefore uses open-loop valve excitation, and every learned model is checked with zeroed and shuffled commands. Prediction score alone is not treated as evidence of control authority.

2.5 Reduced-Order Modeling

Reduced-order models retain dominant behavior at lower cost [Benner 15]. Here the full Simscape plant is reduced to per-cylinder extension, velocity and pressure difference. The construction is grey-box: known structure is embedded so that data fits what remains unknown [Karniadakis 21]. As in the structured dynamics network of Lutter et al. [Lutter 19], explicit physical layers improve interpretability. The hydraulic ROM retains the orifice, pressure and integration structure; learned terms represent valve realization, configuration-dependent load and bounded residuals.

Hwangbo et al. [Hwangbo 19] combined analytic rigid-body mechanics with a learned actuator response, a pattern later scaled in GPU-parallel simulation [Rudin 22]. Yamamori

et al. [Yamamori 26] instead shaped the physical actuator to match the simulation. A commercial valve and pump cannot be redesigned in that way, so this thesis learns the hydraulic realization while PhysX retains the rigid-body mechanics (Figure 2.3).

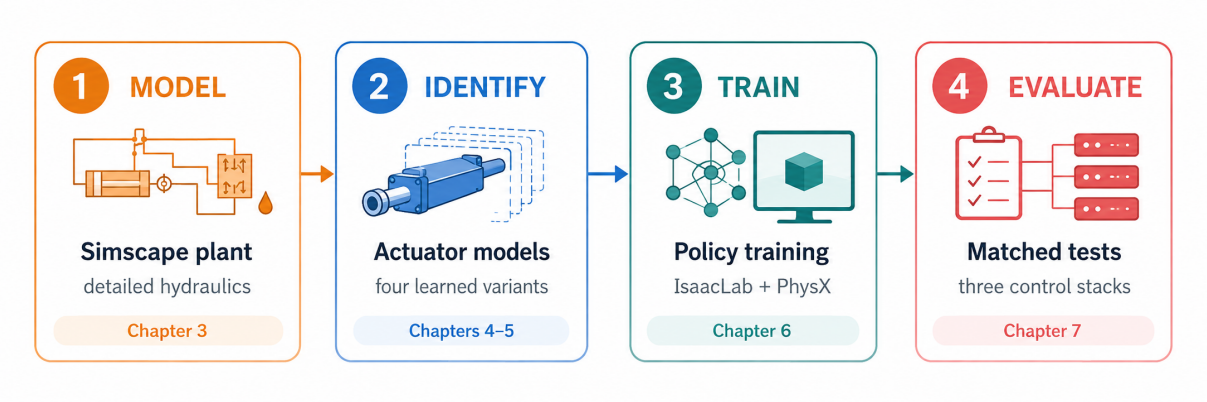

Figure 2.3: Workflow from the hydraulic plant model to matched evaluation. The separation between rigid-body physics and learned actuation follows the actuator-network approach in [Hwangbo 19].

2.6 Domain Randomization and Latent Adaptation

2.6.1 Robustness Through Randomized Training

A model trained on one simulated plant inherits its parameters as assumptions. Domain randomization instead trains across a family, first for visual appearance [Tobin 17] and then for dynamics such as mass, friction, gain and latency [Peng 18]. The hydraulic family in Chapter 4 varies bulk modulus, valve gain, leakage, friction and payload, producing robustness to physically plausible plant variation.

2.6.2 From Robustness to Adaptation

Figure 2.4 contrasts a policy trained for one robust response across the plant family with an adaptive stack that conditions its response on the current plant.

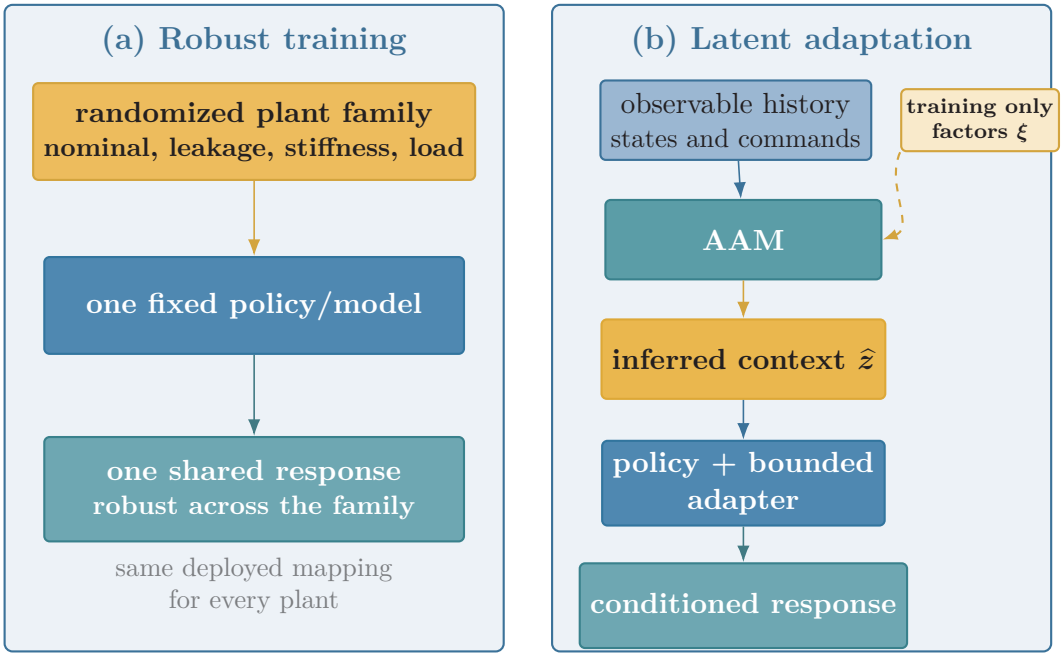

Figure 2.4: Comparison of robust training and latent adaptation. Robust training applies one fixed response across the randomized plant family. The adaptive stack uses privileged factors only to supervise training; during deployment, the AAM infers context from observable history and conditions the downstream response.

Teacher–student methods encode known simulation factors into a privileged latent and train an estimator to recover it from observable history. Lee et al. [Lee 20] used this construction for hidden terrain properties; Kumar et al. [Kumar 21] developed rapid motor adaptation. Unlike classical online parameter estimation [Åström 95], the learned latent need not have a direct physical interpretation. It can also condition a dynamics model rather than a policy, as in the contextual neural ODE of Yu et al. [Yu 26].

The AAM in Chapter 5 estimates a two-dimensional latent from one second of causal observable history. In AIF, that estimate enters the actor observation and a bounded adapter around frozen inverse dynamics; it does not condition the ROM, forward model or PhysX plant. The experiments keep two distinctions explicit: randomized training is robust but not automatically adaptive, and an informative privileged latent does not prove that an inferred latent improves closed-loop control.

2.7 Reinforcement Learning and Structured Priors

2.7.1 Policy Optimization

Reinforcement learning optimizes a policy from observations to actions for expected cumulative reward [Sutton 18]. This thesis uses PPO [Schulman 17] with generalized

advantage estimation [Schulman 16]; Section 6.2.5 gives the exact objective. GPU-parallel simulation supplies the required throughput [Makoviychuk 21, Rudin 22], which is why compact actuator surrogates replace the full hydraulic plant during training.

2.7.2 Structured Priors for Policy Learning

Learning from scratch would discard reliable kinematics. Residual RL instead adds a bounded learned correction to a controller at runtime [Johannink 19, Silver 18]. Policy distillation moves the prior to training: a privileged DLS teacher supplies demonstrations, then the student acts alone [Ross 11]. This thesis distils the teacher and then anchors PPO updates to the frozen student (Section 6.4). The deployed actor remains the sole controller.

The idea parallels real-world fine-tuning that retains a simulation-trained value function [Yin 25], but here the retained object is the reference policy. Anchoring was introduced because unconstrained PPO improved reward while degrading the physical tracking criterion (Section 6.3).

2.7.3 Reinforcement Learning on Hydraulic Machines

Deep RL has reached full-size hydraulic excavators in simulation-to-machine tracking [Egli 20] and general automation [Egli 22]. Spinelli et al. [Spinelli 24] extended it to commercial material handlers with underactuated tools. Maitan et al. [Maitan 26] learned force control on a physical cylinder, using a contraction-metric filter to bound exploration. These studies establish scale and feasibility, but do not combine a randomized plant family, online actuator adaptation, a privileged kinematic teacher and a matched comparison of actuator representations.

A policy can also compensate for a poor actuator model. Chapter 7 therefore reports rollout error, command magnitude and smoothness, and cylinder-level residuals alongside task tracking. Equal task error alone is not evidence of equal model quality.

2.8 Research Gap and Thesis Positioning

The literature leaves a specific combination unresolved. Hydraulic control provides structure but requires maintained plant models; learned dynamics replace derivation with data but depend on identifiable excitation; actuator surrogates and latent adaptation are established mainly for electric legged robots; and hydraulic RL studies do not isolate actuator representation under a matched protocol. Table 2.1 summarizes this position.

The five criteria are: (I) learned hydraulic actuation, (II) data audited for causal input identifiability, (III) online latent adaptation, (IV) a structured controller retained as a bounded prior, and (V) a matched actuator comparison. Parenthesized marks denote partial or implicit coverage.

Table 2.1: Positioning against the reviewed lines of work.

Line of work	I	II	III	IV	V
Model-based hydraulic control [Mattila 17, Jelali 03]	—	(✓)	(✓)	✓	—
Actuator networks for legged robots [Hwangbo 19, Rudin 22, Yamamori 26]	—	✓	—	—	—
RL for hydraulic machines [Egli 20, Egli 22, Spinelli 24, Maitan 26]	✓	(✓)	—	—	—
Latent online adaptation [Lee 20, Kumar 21, Yu 26, Liu 25, Hu 26]	—	—	✓	—	—
Residual RL and policy distillation [Johannink 19, Silver 18, Ross 11, Yin 25]	—	—	—	✓	—
This thesis (R / IF / AIF)	✓	✓	(✓)	✓	✓

The partial mark in column III reflects the results: the latent is recovered causally, but its closed-loop benefit is less conclusive. The contribution is therefore the complete experimental chain—identifiable cylinder data, physics-structured models fast enough for parallel training, online latent estimation, and a common evaluation protocol—rather than an unqualified claim that adaptation always improves control. The three tested stacks are:

R. A command-conditioned ROM realizes the policy's desired cylinder motion. It is the reference and the only actuator calibrated to the deployment plant (Section 6.9).

IF. The same policy interface drives the frozen inverse model followed by the strict forward model, forming the non-adaptive learned baseline.

AIF. The AAM infers a dynamics latent from causal observable history. It conditions the policy observation and a bounded adapter around the frozen inverse model, ahead of the same forward model and PhysX.

All stacks share the task, action contract, terrain, trajectories, deployment factors and acceptance gates. Their training runs are not fully matched, and the ROM is calibrated to the deployment plant; causal architectural claims therefore rely on within-checkpoint ablations rather than the across-stack contrast (Section 6.9). Chapter 3 now grounds this comparison in the concrete machine.

3. System Fundamentals and Problem Formulation

Chapter 2 established the control and modeling concepts used in this thesis. This chapter converts that background into a fixed system definition for the remaining work. It specifies the physical mechanism, coordinate mappings, available measurements, actuator models, task geometry, and evaluation interfaces before the controllers are compared. Keeping these definitions together prevents later chapters from changing the plant, observation set, or success criteria when interpreting results.

The physical machine and its two simulation representations are distinguished explicitly. Simscape is the high-fidelity hydraulic source used for actuator identification. IsaacLab/PhysX is the parallel environment used for policy training, adaptation-data collection, and closed-loop evaluation. Both use the cylinder ordering, kinematics, action contract, and cutter-tip task variables defined here, but they are not assumed to be identical plants.

Sections 3.1–3.3 define the mechanism, coordinate spaces, and signals. Sections 3.4 and 3.5 describe the two simulation models. The remaining sections define the verge-following task, the R, IF, and AIF control stacks, the control objective, and the acceptance gates used in the experiments.

3.1 Manipulator Platform

The MULAG FME 500 is modeled as a serial-link mechanism with five revolute joints. Four hydraulic cylinders actuate the slew, two boom, and head-pivot joints. The *Messerkopf* rotation has no independent actuator and is locked at the mowing configuration; it is therefore not a controlled degree of freedom. The cylinder order used throughout the thesis is slew (*Drehzapfen*), boom I (*Ausleger I*), boom II (*Ausleger II*), and head pivot (*Messerkopf-Schwenk*).

Table 3.1 gives the cylinder stroke ranges used in the training data and the fixed limit of the *Messerkopf* joint. Task-space errors are measured at the cutter tip, which lies 0.79 m below the *Messerkopf* body origin.

Table 3.1: Cylinder stroke ranges in the training data (Chapter 4) and the fixed *Messerkopf* joint limit used for the mowing task.

i	Joint	Function	Limit
1	*Drehzapfen*	slew	0.018–0.493 m
2	*Ausleger I*	boom I	0.275–0.791 m
3	*Ausleger II*	boom II	0.309–0.857 m
4	*Messerkopf-Schwenk*	head pivot	0.269–0.689 m
5	*Messerkopf*	head rotation (locked)	$q_{5,\min} = q_{5,\max} = q_{5,\text{lock}}$

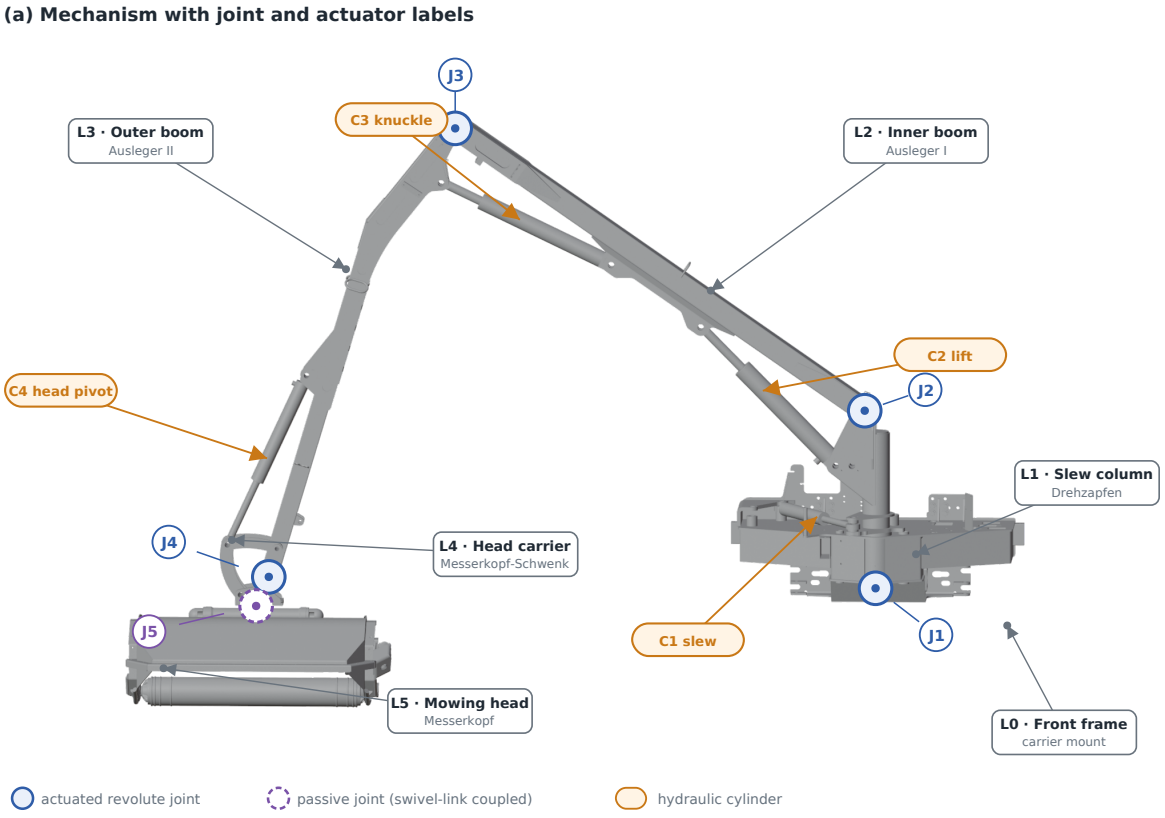

Figure 3.1: Annotated CAD view of the MULAG FME 500. Links L0–L5, joints J1–J5, and the four hydraulic cylinders C1–C4 are identified; J5 denotes the *Messerkopf* rotation, which is locked in the mowing configuration used in this thesis.

Figure 3.1 identifies the links, joints, and four actuator channels used throughout the model.

Figure 3.2 separates the rigid bodies that form the assembled mechanism. The catalog makes the L0–L5 body assignment explicit and shows the representative barrel–rod geometry used for cylinders C1–C4.

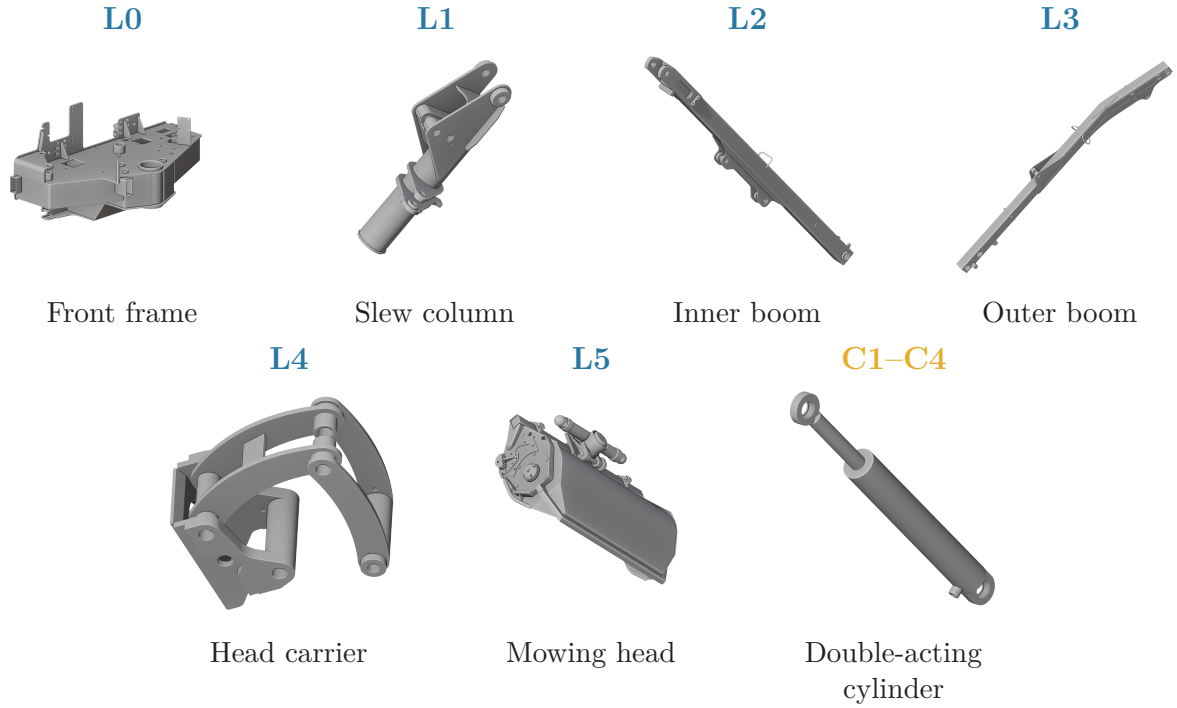

Figure 3.2: Individual CAD-derived bodies of the MULAG FME 500 shown at a common orthographic scale. L0–L5 are the front frame, slew column, inner boom, outer boom, head carrier, and mowing head. The C1–C4 panel shows the representative double-acting cylinder barrel–rod pair.

Manufacturer CAD supplies the link geometry, joint locations, masses, centers of mass, and inertia tensors through Phobos [von Szadkowski 20]. The same CAD-derived bodies and joint frames form the mechanical basis of the Simscape and IsaacLab models, keeping dimensions, axes, masses, and cylinder lever arms consistent while the hydraulic implementations remain model-specific.

3.2 Kinematic Structure and Coordinate Mappings

3.2.1 Joint Space and Cylinder Space

The configuration is $\boldsymbol{q} \in \mathbb{R}^5$; the actuated coordinates are the four cylinder extensions $\boldsymbol{x} \in \mathbb{R}^4$. Cylinder i joins CAD attachment points $\boldsymbol{p}_A$ and $\boldsymbol{p}_B$ on adjacent links. Its length and extension are

$$\ell_i(\boldsymbol{q}) = \left\| \boldsymbol{p}_B(\boldsymbol{q}) - \boldsymbol{p}_A(\boldsymbol{q}) \right\|, \qquad x_i = \ell_i(\boldsymbol{q}) - \ell_{0,i}, \tag{3.1}$$

with $\ell_{0,i}$ the retracted reference length. Differentiating (3.1) gives the scalar mapping between joint motion and extension rate,

$$\dot{x}_i = J_{\mathrm{cyl},i}(\boldsymbol{q})\, \dot{\boldsymbol{q}}, \qquad J_{\mathrm{cyl},i} = \boldsymbol{e}_i^{\top} \left(\boldsymbol{J}_B - \boldsymbol{J}_A \right), \tag{3.2}$$

where $\boldsymbol{e}_i$ is the cylinder-axis unit vector and $\boldsymbol{J}_A$ and $\boldsymbol{J}_B$ are attachment-point Jacobians. Virtual work gives $\boldsymbol{\tau}_{\text{act}} = \boldsymbol{J}_{\text{cyl}}^{\top}\boldsymbol{F}_h$. The mapping varies with configuration and stroke.

3.2.2 Task Space and the Calibrated Cutter Jacobian

The cutter tip is expressed in the truck chassis frame (`base_link`); using the moving slew-base frame would corrupt the tracking error. The controlled point includes the $-0.79\,\text{m}$ tool-tip offset. Vehicle motion supplies the along-track direction.

The calibrated Jacobian $\boldsymbol{J}_c \in \mathbb{R}^{3\times 4}$ maps cylinder extension rates to cutter-tip velocity. It was measured at the head-down mowing pose by stepping each cylinder target and recording the settled cutter displacement. Its singular values are 8.24, 5.69, and 2.79, giving rank 3. The damped least-squares controller in Chapter 6 uses this same Jacobian.

3.3 Actuation and Measured State

Table 3.2 lists the signals and their availability.

Table 3.2: Signal inventory of the modeled system. "S" marks signals present in the Simscape data exports, "I" signals available live in the IsaacLab environment, "M" quantities that exist there only through the learned models of Chapter 5, and "P" *privileged* quantities that exist in simulation but must never enter a deployable policy or estimator input.

Signal	Dimension	Availability	Meaning
$\boldsymbol{u}$	$[-1,1]^4$	S, M	normalized proportional-valve commands
$\boldsymbol{x}$, $\dot{\boldsymbol{x}}$	$\mathbb{R}^4$	S, I	cylinder extensions and velocities
$\Delta\boldsymbol{p}$	$\mathbb{R}^4$	S, M	chamber pressure differences
$\boldsymbol{F}$	$\mathbb{R}^4$	S	logged rod forces
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	$\mathbb{R}^5$	S, I	joint positions and velocities
$\boldsymbol{p}_c$	$\mathbb{R}^3$	S, I	cutter-tip position, truck frame
$\boldsymbol{h}$	$\mathbb{R}^9$	I	forward terrain-height profile ahead of the cutter
$\boldsymbol{c} = [\Delta\boldsymbol{x}^*, \Delta\dot{\boldsymbol{x}}^*]$	$\mathbb{R}^8$	I	desired cylinder command
$\mathbf{e}$	$\mathbb{R}^6$	P	physical randomization factors of the active plant
$\boldsymbol{z} = \mu(\mathbf{e})$	$\mathbb{R}^{d_z}$	P	privileged dynamics latent, $d_z = 2$
$\hat{\mathbf{z}}$	$\mathbb{R}^{d_z}$	M	AAM estimate of $\boldsymbol{z}$ from observable history

In IsaacLab, valve commands and differential pressures are learned-actuator states, not direct measurements. R produces them internally; IF and AIF obtain them from the inverse/forward pair. $\mathbf{e}$ and $\boldsymbol{z} = \mu(\mathbf{e})$ are privileged; only $\hat{\mathbf{z}}$ may enter AIF.

3.3.1 Sensing and Observability

The deployable observation contains articulation state, terrain rays, contact signals, the desired cylinder command, and actuator-model pressure state. Plant parameters are not observed; AIF infers their effect from causal history.

Figure 3.3 separates deployable measurements from privileged plant information. Articulation state is read from the physics engine without encoder noise or quantisation.

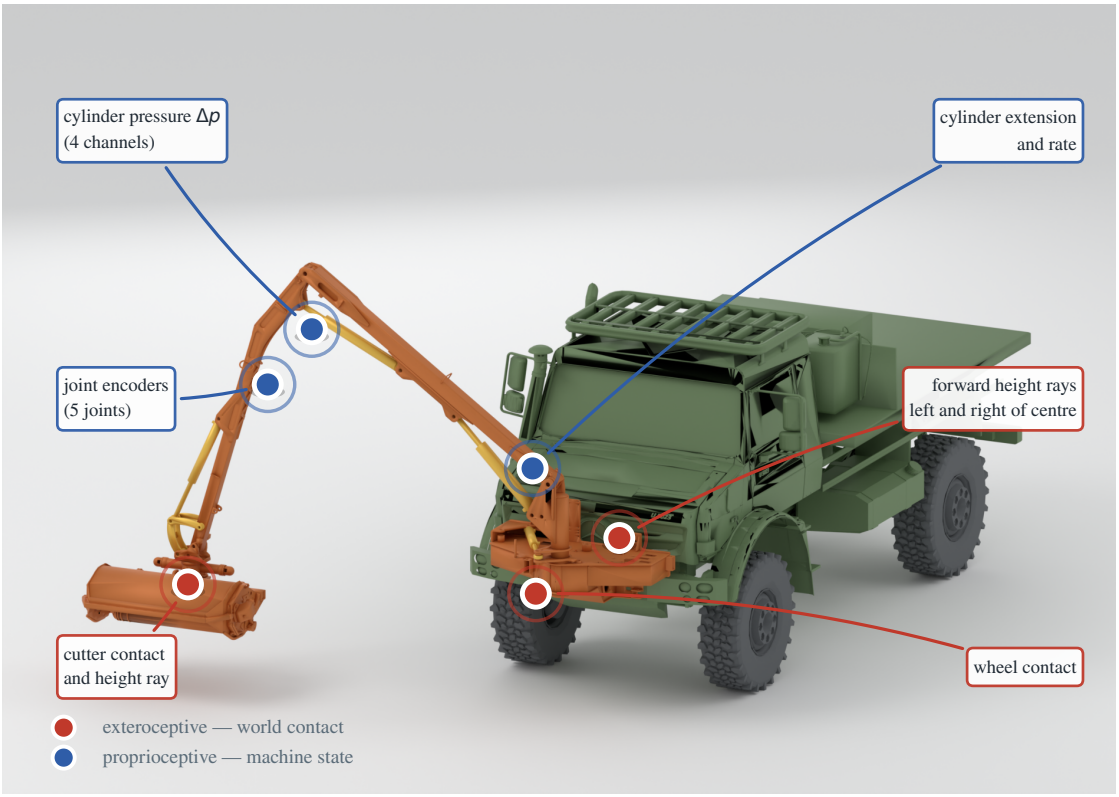

Figure 3.3: Sensor and observability map for the deployed controller. Blue callouts identify proprioceptive inputs: joint encoders, cylinder pressure differences, and cylinder extension and rate. Red callouts identify exteroceptive inputs: forward terrain-height rays and cutter and wheel contact.

3.4 High-Fidelity Simscape Model

The actuator models use a multi-domain Simscape model of the FME 500 developed in the preceding master project. Table 3.3 distinguishes the data source of each frozen component.

Table 3.3: Provenance of each frozen actuator component. The command ROM is the only one that has been calibrated against the deployment plant; the inverse/forward pair is Simscape-identified throughout. This asymmetry is a property of the comparison and is carried into the discussion of Chapter 7, not glossed over.

Component	Identified on	Calibrated on
Command ROM	Simscape closed-loop export (grey-box, full model)	IsaacLab/PhysX closed-loop rollouts, via a bounded command-gated correction head only
Control ID	Simscape closed-loop export	—
Strict FD	Simscape closed-loop export, retrained on the strict open-loop excitation export	—
SD_0, $\mu(\mathbf{e})$, SD_z, AAM	IsaacLab/PhysX adaptation collection (from scratch)	—

Simscape supplies the frozen ID and strict FD checkpoints. The ROM receives an additional bounded IsaacLab calibration (Section 5.4.1). The adaptation models are trained from IsaacLab data collected through the ROM and PhysX.

3.4.1 The Command ROM as Deployment Actuator

The command ROM plus PhysX is the deployment plant. Simscape is too slow for policy training and bare PhysX has no hydraulics. The ROM is therefore both the plant model and the actuator of R:

- R uses the deployment plant itself and is the reference condition.
- IF replaces it with an independently identified inverse/forward pair.
- AIF adds online adaptation to IF.

The R–IF gap measures the transfer cost of the independently identified pair; it is not a general ranking of the two model classes.

3.4.2 Mechanical Subsystem

The mechanism uses the CAD-derived links, joints, and cylinder attachment frames of Section 3.1. It is simulated in Simscape Multibody [The MathWorks, Inc. 24b]. The joint-space dynamics take the standard manipulator form

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) + \boldsymbol{\tau}_{\text{fric}}(\dot{\boldsymbol{q}}) = \boldsymbol{\tau}_{\text{act}} - \boldsymbol{J}^{\top}(\boldsymbol{q})\,\boldsymbol{F}_{\text{ext}}, \tag{3.3}$$

with inertia $\boldsymbol{M}$ from the imported CAD. Friction combines viscous, Coulomb, and Stribeck terms. The cutting load scales with feed velocity and vegetation density. Each cylinder uses a spherical–prismatic–cylindrical loop and hard stops enforce the limits in Table 3.1.

3.4.3 Hydraulic Subsystem

Figure 3.4 shows four 4/3 valve-driven asymmetric cylinders fed by a variable-speed, fixed-displacement pump ($Q_p = D_p\omega$), accumulator, and pressure-relief valve.

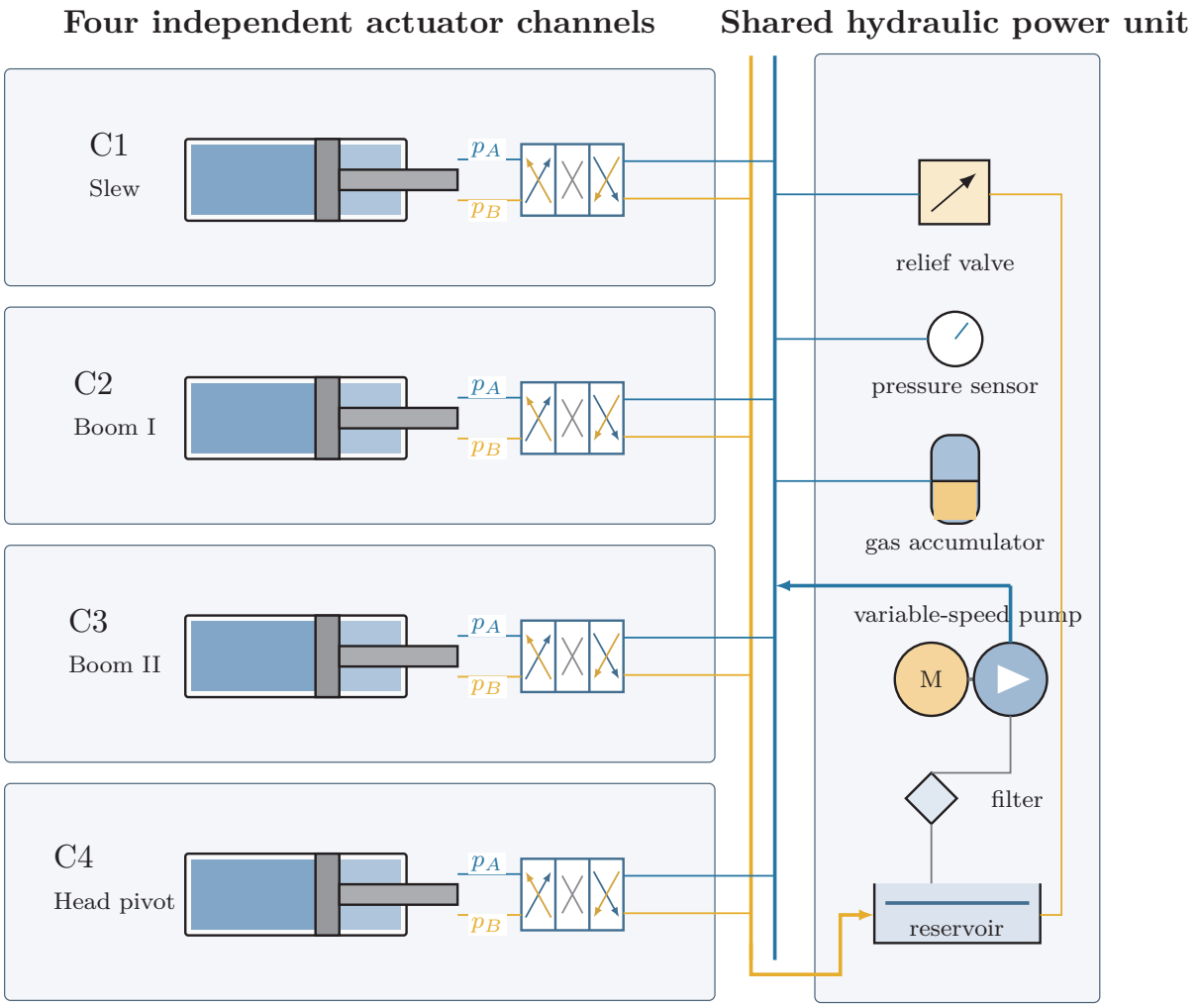

Figure 3.4: Hydraulic system schematic. Four proportional-valve cylinder channels share a variable-speed pump, reservoir, accumulator, sensor, and pressure-relief valve. Blue and orange rails denote supply and return.

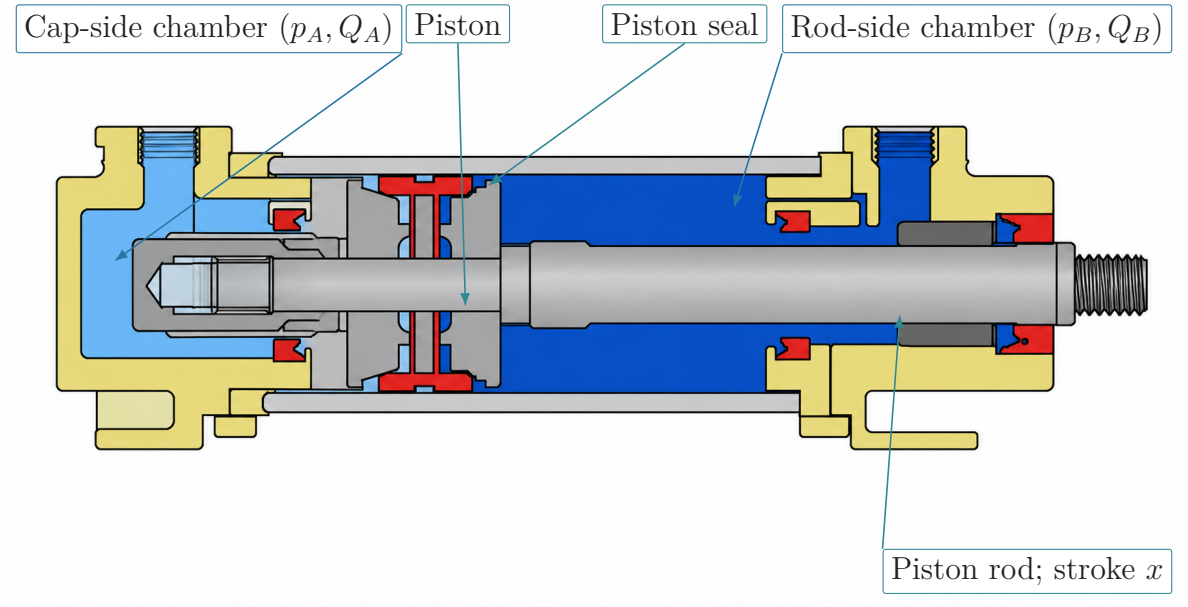

Figure 3.5: Annotated cutaway of an asymmetric double-acting hydraulic cylinder. The cap-side and rod-side chamber pressures generate the piston force and rod stroke.

Figure 3.5 identifies the cylinder chambers, piston, seal, and rod used in the actuator model.

The 4/3 valve in Figure 3.4 meters flow to the cylinder chambers. The chamber pressures follow

$$\dot{p}_1 = \frac{E_{\text{eff}}}{V_1(x)}\big(Q_1 - A_1\dot{x} - C_{\text{leak}}(p_1 - p_2)\big), \qquad \dot{p}_2 = \frac{E_{\text{eff}}}{V_2(x)}\big(-Q_2 + A_2\dot{x} + C_{\text{leak}}(p_1 - p_2)\big), \tag{3.4}$$

where $V_{1,2}(x)$ include dead volumes, $A_1 \neq A_2$, and C_{leak} is internal leakage. Entrained air is represented by

$$E_{\text{eff}}(p) = \left(\frac{1}{E_f} + \frac{\alpha}{p}\right)^{-1}, \tag{3.5}$$

Valve flow follows the orifice law of Section 2.1.2; spool dynamics are $\dot{x}_v = (u - x_v)/\tau_v$. The piston force is $F_h = p_1 A_1 - p_2 A_2 - F_{\text{fric}}(\dot{x})$. The coupled model is stiff and uses a variable-step implicit solver, which prevents RL-time simulation.

3.4.4 Validated Scope of the Model

Validation uses a fixed integral-backstepping controller at 0.01 s while sweeping $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ GPa. Slew-axis delay falls from about 0.85 s to 0.65 s, the system delay index from 0.45 s to 0.35 s, and task-space tracking error by about 11 %. The improvement saturates beyond 1.5 GPa.

The model represents one machine, omits thermal dynamics, and treats the logged rod-force channel as unreliable after the piston-area consistency audit (Section 5.4). Chapter 4 introduces variation through parameter randomization.

3.5 IsaacLab Environment

Policy training, adaptation-data collection, and evaluation run in IsaacLab [Mittal 23] on PhysX. The USD articulation contains the Unimog carrier, five arm links, and four prismatic cylinder joints. Learned actuator models supply the hydraulic behavior between the controller and PhysX cylinder targets.

3.5.1 Simulation and Control Rates

Table 3.4 lists the four simulation clocks. PhysX runs at 120 Hz and the policy at 60 Hz. The learned actuator models and AAM update every three policy steps (0.05 s); their prediction is held across the intervening PhysX updates. Data collection uses this same 50 ms grid.

Table 3.4: The four rates of the simulated system. They are reported separately throughout the thesis; "the control frequency" is not a well-defined quantity here.

Layer	Step	Rate
PhysX physics	1/120 s	120 Hz
Policy / control (decimation 2)	1/60 s	60 Hz
Learned actuator model (3 control steps)	0.05 s	20 Hz
AAM latent update	0.05 s	20 Hz

3.5.2 Environment Configuration Details

The arm root is spawned at $z = 1.26$ m; ground-height spawning buries the vehicle. Cylinder targets are integrated in reference space so gravity does not erase commanded increments. Vehicle motion is scripted because the authored tyres and suspension do not provide reliable PhysX traction; a planar fixture holds lateral position and heading.

3.6 The Verge-Following Task

3.6.1 Task Definition

The task is moving verge following. The truck drives a 26 m strip at a speed sampled from 2 to 7 km/h; development uses 4 km/h. Episodes allow 2808 steps (46.8 s), while scoring stops when the strip is completed. The arm must keep the cutter at the target height, on the verge centre line, in contact with the ground, and within its travel limits. Terrain height is randomized per environment (Figure 3.6).

Eleven reference-path families span straight, periodic, spline, sweep, and lane-change motion. Results use the worst path family and terrain mesh rather than a pooled score. A regression test verifies that all eleven families are distinct.

Figure 3.6: IsaacLab realization of the moving verge-following task. The scripted truck travels on the level road while the four-cylinder arm places the cutter above the procedurally generated verge and follows the white reference path. Randomized grass, bushes, soil patches, twigs, and logs form a non-colliding visual layer; the underlying verge mesh supplies the terrain disturbance and height-sensing geometry used by the controller.

3.6.2 Task Feasibility and Plant Capability

The action contract limits cylinders 1–3 to 4 mm per policy step, which bounds lateral cutter speed at

$$v_{\text{lat}}^{\max} = \frac{4\,\text{mm}}{1/60\,\text{s}} = 0.240\,\text{m/s}. \tag{3.6}$$

This uses the 1/60 s policy interval. Early path settings exceeded this limit; the worst case (`boom_sweep`) required 3.7× the available rate. Path and terrain parameters are now derived from (3.6), and a preflight check rejects infeasible tasks.

3.7 Control Architecture

The vehicle advances at the scripted speed and the reference generator supplies the verge centre line and cut height. The policy maps normalized observations to four desired cylinder increments; it has no runtime baseline controller. The action contract bounds those increments. R uses the command ROM, IF the frozen inverse/forward pair, and AIF adds the bounded adapter conditioned on $\hat{\mathbf{z}}$. All paths set the same PhysX cylinder target. The DLS controller is used only for teaching and identification.

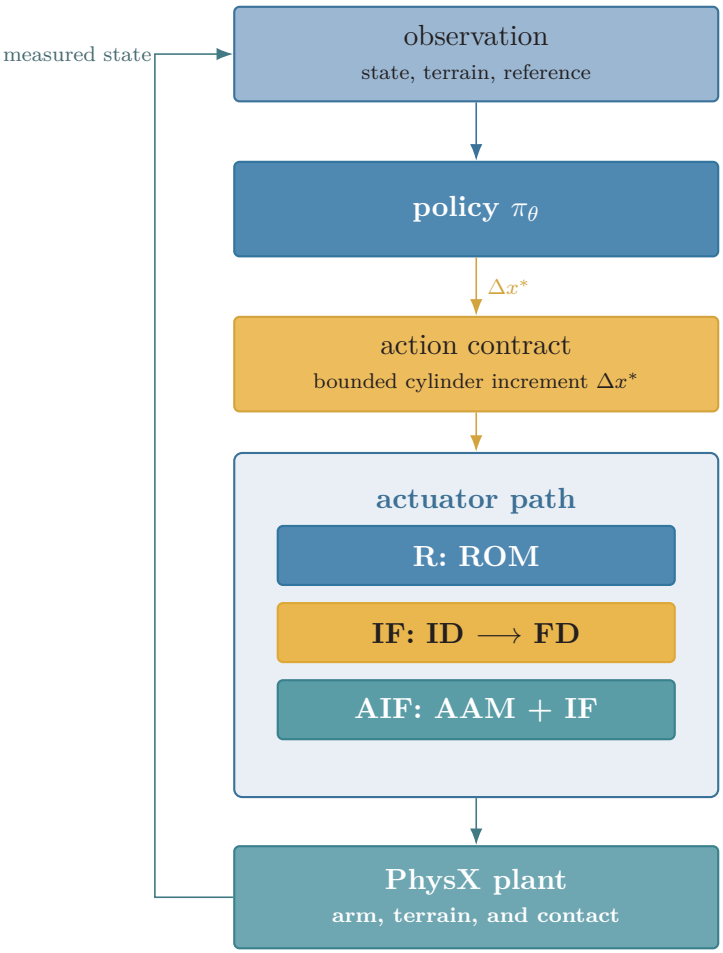

Figure 3.7: Deployed control stack. The policy produces a bounded cylinder increment; R, IF, or AIF realizes it in the shared PhysX plant. Measured state closes the loop.

3.8 Problem Formulation

The reference is the verge centre line $y_{\text{ref}}(t)$ and cut height h_{ref}, expressed in the truck frame. With cutter-tip position $\boldsymbol{p}_c(t)$ and terrain height $g(\cdot)$, the scored errors are

$$e_{\text{lat}}(t) = p_{c,y}(t) - y_{\text{ref}}(t), \qquad e_{\text{cut}}(t) = \Big(p_{c,z}(t) - g(\boldsymbol{p}_c(t))\Big) - h_{\text{ref}}. \tag{3.7}$$

The objective is to minimize the mean and tail of both errors across path families and terrain while maintaining contact, head alignment, and smooth motion. Learned models are evaluated on one-step and recursive cylinder predictions. The stacks are tested across the randomized plant family; AIF is compared with IF on held-out factors.

Constraints are cylinder limits and headroom (Table 3.1), the action contract, checkpoint output limits, and the head-down condition.

3.9 System-Level Evaluation Criteria

Controllers and models are evaluated separately. Controller scores can hide actuator-model errors through policy compensation.

The system-level criteria cover three aspects of operation. Cut-height and lateral errors measure task accuracy. Contact, head alignment, impact rate, and cutter acceleration describe tool–terrain interaction and motion quality. Cylinder headroom and safety terminations check whether the task is completed without driving the mechanism into its limits. The metrics are evaluated over the scored part of each episode and are kept in physical units where possible. This makes a failed gate traceable to a specific limitation rather than to a single combined reward value.

3.9.1 Eight-Gate Acceptance Contract

Table 3.5 defines the system-level acceptance contract.

All controller stacks are evaluated with the same path, terrain, and randomization conditions. A stack passes only when it satisfies every gate; strong mean tracking cannot compensate for loss of contact, inadequate cylinder headroom, excessive impacts, or safety terminations. Worst-case aggregation prevents easy terrain or path cases from masking a local failure. Mean absolute errors quantify sustained tracking performance, while the lower and upper percentiles capture short alignment, clearance, or acceleration events that a mean can hide. Contact and termination measures are reported as fractions so that episodes of different completed length remain comparable.

Table 3.5: Eight system-level acceptance gates. Each gate uses at least three terrain meshes and the worst mesh; tracking-critical gates also use the worst path family. †Hard impacts is a wear constraint.

Gate	Metric	Threshold	Aggregation
Cut height	mean $\lvert e_{\mathrm{cut}} \rvert$	$\leq 0.040\,\mathrm{m}$	worst mesh
Lateral tracking	mean $\lvert e_{\mathrm{lat}} \rvert$	$\leq 0.300\,\mathrm{m}$	worst mesh, worst family
Ground contact	contact fraction	≥ 0.700	worst mesh, worst family
Head alignment	p01 head-down alignment	≥ 0.900	worst mesh
Hard impacts†	hard-impact step fraction	≤ 0.350	worst mesh
Tool acceleration	p95 cutter acceleration	$\leq 120\,\mathrm{m/s^2}$	worst mesh
Cylinder headroom	p01 travel margin	$\geq 0.005\,\mathrm{m}$	worst mesh
Safety terminations	termination rate	≤ 0.080	worst mesh

3.9.2 Cylinder and Model Checks

Cylinder checks use fresh $\pm 10\,\mathrm{mm}$ commands, coordinated $80\,\mathrm{mm}$ motions, return error, and saturation. Model checks use one-step and rollout accuracy, input ablation, and Jacobian conditioning (Chapter 5). Chapter 7 reports these with the acceptance contract. The small command checks local response and deadband; the coordinated motion checks coupling between actuator channels. Return error tests reversibility, and saturation checks whether the learned model respects the command and stroke limits. These diagnostics are interpreted alongside the system gates, not as substitutes for closed-loop task performance.

The system is now specified. The next chapter turns to the data the learned models are built from.

4. Data Generation and Preparation

Chapter 3 defined two plant representations: detailed hydraulics in Simscape and fast rigid-body dynamics in IsaacLab. This chapter documents the datasets that connect them, including collection, auditing, feature construction and splitting. Table 3.3 records their provenance.

Simscape campaigns. Closed-loop runs provide realistic operating data for the inverse and reduced-order models. Separate open-loop valve excitation provides the causal command information needed by the strict forward model (Section 4.4).

ROM calibration. After Simscape training, the command ROM was fine-tuned on five closed-loop IsaacLab reference shapes. Only its bounded correction head remained trainable, with Simscape replay retained (Section 4.6). The ID–FD pair was not calibrated in IsaacLab.

IsaacLab adaptation data. The system-dynamics surrogate, privileged encoder and AAM use a separate 118-run collection from the deployed command ROM–PhysX plant, including dedicated PRBS windows (Section 4.7). No earlier adaptation or system-dynamics model was reused. Figure 4.1 summarizes the common preparation workflow.

4.1 Dataset Objectives and Design Requirements

Four requirements governed collection and preparation.

R1: realistic operating distribution. The learned models are deployed inside a tracking controller, so they must be accurate in the states that controller visits: coordinated multi-axis motion, gravity-loaded poses, moderate speeds. Data collected far from that distribution would be accurate where it does not matter.

R2: causal identifiability of the valve. A forward model is only useful to a policy if it responds to the command. This requires that the valve signal in the training data carry information that the state history does not already carry, which is a property of the *experiment*, not of the network [Ljung 99, Isermann 11].

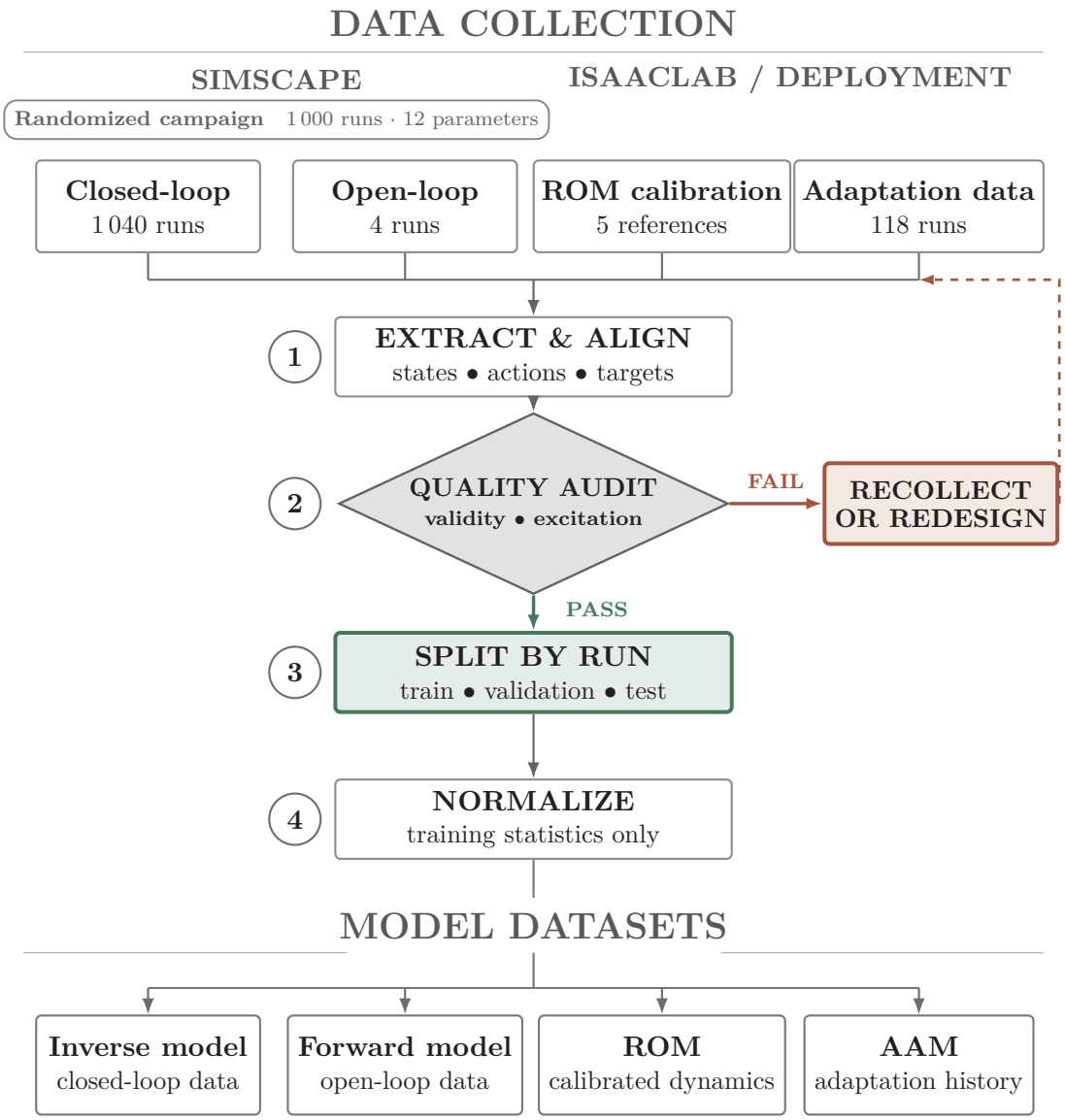

Figure 4.1: Data-preparation workflow. Two Simscape collections and two IsaacLab collections pass through common extraction, audit, run-level splitting and training-statistics normalization stages. The randomized Simscape campaign is provenance metadata rather than a pipeline stage. A failed audit returns the experiment to collection; accepted material forms the inverse, forward, ROM and AAM datasets.

R3: variation across a plant family. Robust training and, later, latent adaptation both need many plants rather than one. The variation has to be physically meaningful, it has to demonstrably change the measured response, and it has to be recorded per run, because the privileged latent of Section 5.5 is computed from those recorded factors.

R4: clean separation. Train, validation, and test material must be separable at the level of complete runs, so that no transition window straddles a split boundary and no run contributes to two splits.

R1 and R2 require different experiments. Closed-loop control gives realistic motion but correlates command with state; open-loop excitation breaks that correlation but is less representative of deployment. The two campaigns are therefore retained and combined as described in Section 4.5.

4.2 Randomized Parameter Space of the Simscape Campaign

This section records the Simscape randomization design. No reported model is trained on this curriculum; it is retained as collection provenance and because its compliance analysis (Section 4.2.3) informed the IsaacLab factor design. Readers concerned only with trained models may continue at Section 4.3.

4.2.1 Sampling Design

The closed-loop campaign draws its plants from a twelve-dimensional parameter space using Latin hypercube sampling [McKay 79], which spreads the draws more evenly over each marginal than independent uniform sampling would at this sample size. The twelve coordinates are fixed once for the whole batch, before any simulation runs, so the parameter table is reproducible and can be audited independently of the data it produced.

The batch is organised as a curriculum rather than a single homogeneous pool. Run 1 is the untouched nominal plant. Runs 2–251 randomize only the hydraulics, runs 252–501 only the mechanics, and runs 502–1000 both together, giving the composition shown in Figure 4.2(a). This structure is deliberate: it makes it possible to attribute a later modelling failure to the hydraulic or the mechanical axis by retraining on one block at a time, which a uniformly mixed batch would not allow.

4.2.2 Hydraulic Parameters

Table B.1 lists the hydraulic axes. They were chosen to correspond to the physical mechanisms that make one machine differ from another in service, in the sense discussed in Section 2.1.2: air works its way into the fluid, spools and seals wear, and relief settings drift.

The seal-wear coordinate deserves comment because it is the one place where a single sampled number is deliberately allowed to move several model parameters together. A worn cylinder does not become leakier without also becoming slower, so $w_{\mathrm{seal},i}$ scales the

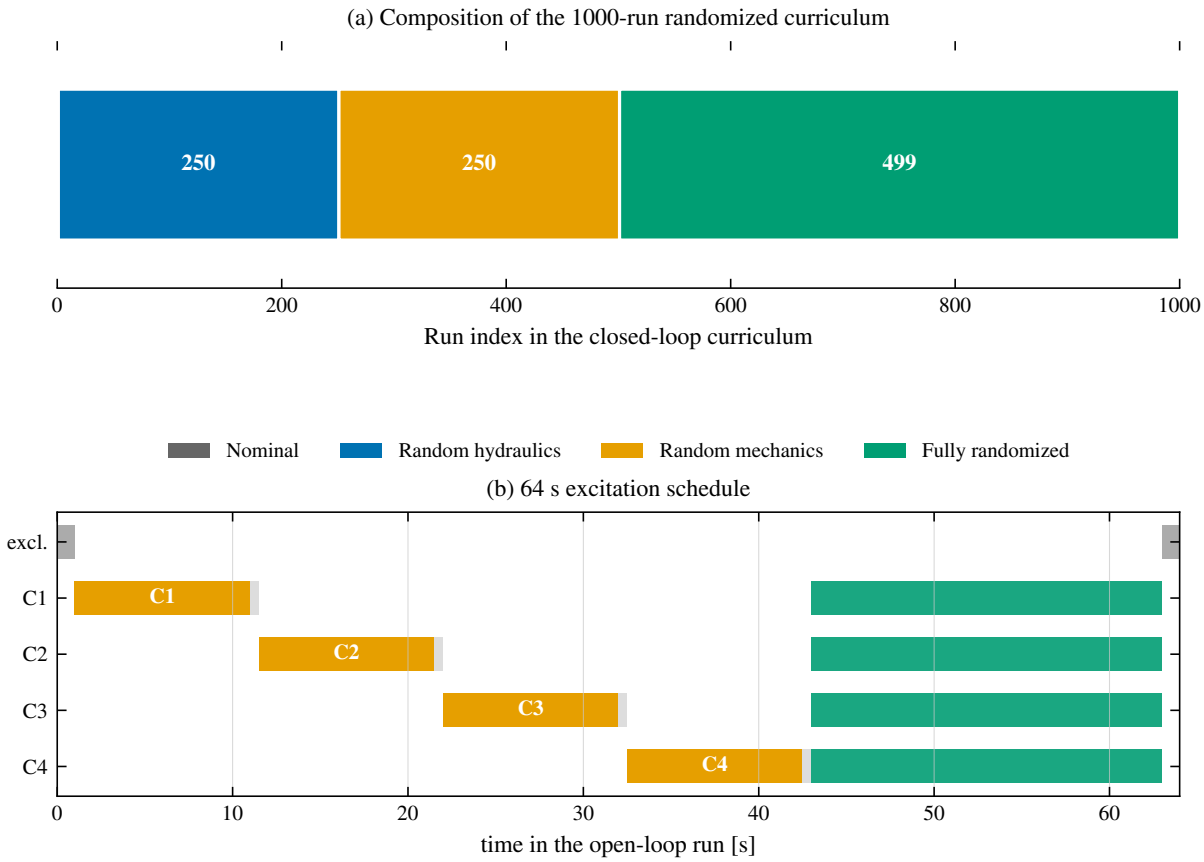

Figure 4.2: Design of the two collection campaigns. (a) Composition of the closed-loop curriculum: one nominal run, then 250 hydraulics-only, 250 mechanics-only and 499 fully randomized draws. (b) Timeline of one open-loop excitation run.

piston clearance linearly, the valve leakage area with the cube of the same factor, and the spool time constant from 8 ms to about 13 ms at the top of the range. Sampling those three independently would have produced physically incoherent machines, some of them very leaky and yet perfectly fast.

4.2.3 The Compressibility Axis

The entrained-air and dead-volume coordinates randomize the chamber dead volume, which spans 505 to 579 mL across the batch. This reaches the pressure dynamics through the continuity equation (3.4), which carries the chamber volume: a larger chamber is more compliant and slower, a smaller one stiffer and faster. The compressibility axis therefore enters the batch as capacitance variation.

4.2.4 Mechanical Parameters

The mechanical axes, listed in Table B.2, are applied to all fourteen rigid bodies of the mechanism: the six links, the four cylinder barrels, and the four pistons. For each body the mass is perturbed by up to ±20 %, the centre of mass by up to ±5 % of its nominal offset, and the inertia tensor together with its products of inertia is perturbed consistently

Algorithm 4.1: Closed-loop curriculum generation.

input : run budget $N = 1000$; joint limits $\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}$; nominal parameters $\boldsymbol{\theta}_0$
output : dataset $\mathcal{D} = \{(\text{trajectory}_i, \boldsymbol{\xi}_i)\}_{i=1}^{N}$

$\mathbf{L} \leftarrow \textsc{LatinHypercube}(N, 12)$ `// one design for the whole batch`
for $i \leftarrow 1$ **to** N **do**
  assign i to a curriculum block (nominal / hydraulic / mechanical / full)
  $\boldsymbol{\xi}_i \leftarrow \textsc{DrawParameters}(\mathbf{L}_i, \text{block})$ `//` $\varepsilon_{\text{air}}, \varepsilon_{\text{vol}}, c_d, w_{\text{seal}}, p_{\text{set}}$ `; masses, CoM, inertia, stiffness`
end
for $i \leftarrow 1$ **to** N **do**
  inject $\boldsymbol{\xi}_i$ into the Simscape model
  $n_w \leftarrow \textsc{RandInt}(12, 20)$; $\boldsymbol{W} \leftarrow \textsc{LatinHypercube}(n_w, \text{JDOF})$ within $[\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}]$
  assign each segment a speed class (slow / medium / fast / variable); **no** dwell
  $\boldsymbol{q}_{\text{ref}}(t) \leftarrow \textsc{Interpolate}(\boldsymbol{W})$ over 22 s
  simulate with `ode23t`; log on the 0.05 s grid
  store $(\boldsymbol{u}, \boldsymbol{x}, \dot{\boldsymbol{x}}, \Delta\boldsymbol{p}, \boldsymbol{F}, \boldsymbol{q}, \dot{\boldsymbol{q}}, \boldsymbol{q}_{\text{ref}}, \boldsymbol{\xi}_i)$
  if $i \bmod 100 = 0$ **then** checkpoint $\mathcal{D}$ to disk
end

with the mass change, so that the body remains physically realisable rather than acquiring an inertia that its mass could not produce.

The joint stiffness and damping terms model compliance in the bearings and linkage rather than in the fluid. They matter here because they set how much of a pressure transient appears as boom oscillation instead of clean motion, which is the coupling that a purely kinematic controller cannot anticipate.

4.3 Closed-Loop Data Collection

Algorithm 4.1 summarizes the procedure. The parameter table is fixed before simulation, and each run receives a new reference path.

4.3.1 The Controller as a Data-Collection Instrument

Every closed-loop run uses the same integral-backstepping controller at 0.005 s. Its gains remain fixed, so response differences are caused by the plant draw rather than controller retuning.

Joint 2 uses PI control with velocity damping and gravity bias because it carries the boom load. A barrier term enforces the URDF limits. Per-axis command limits are [0.95, 0.95, 0.80, 0.80, 0.90]; the first four are also stored in the inverse-dynamics checkpoint (Section 5.2).

4.4 Open-Loop Excitation Experiments

4.4.1 Limitations of the Closed-Loop Batch

The first forward model achieved one-step $R^2 \approx 0.999$ on closed-loop data but was almost insensitive to valve input: zeroing the input changed long-horizon error by only a few

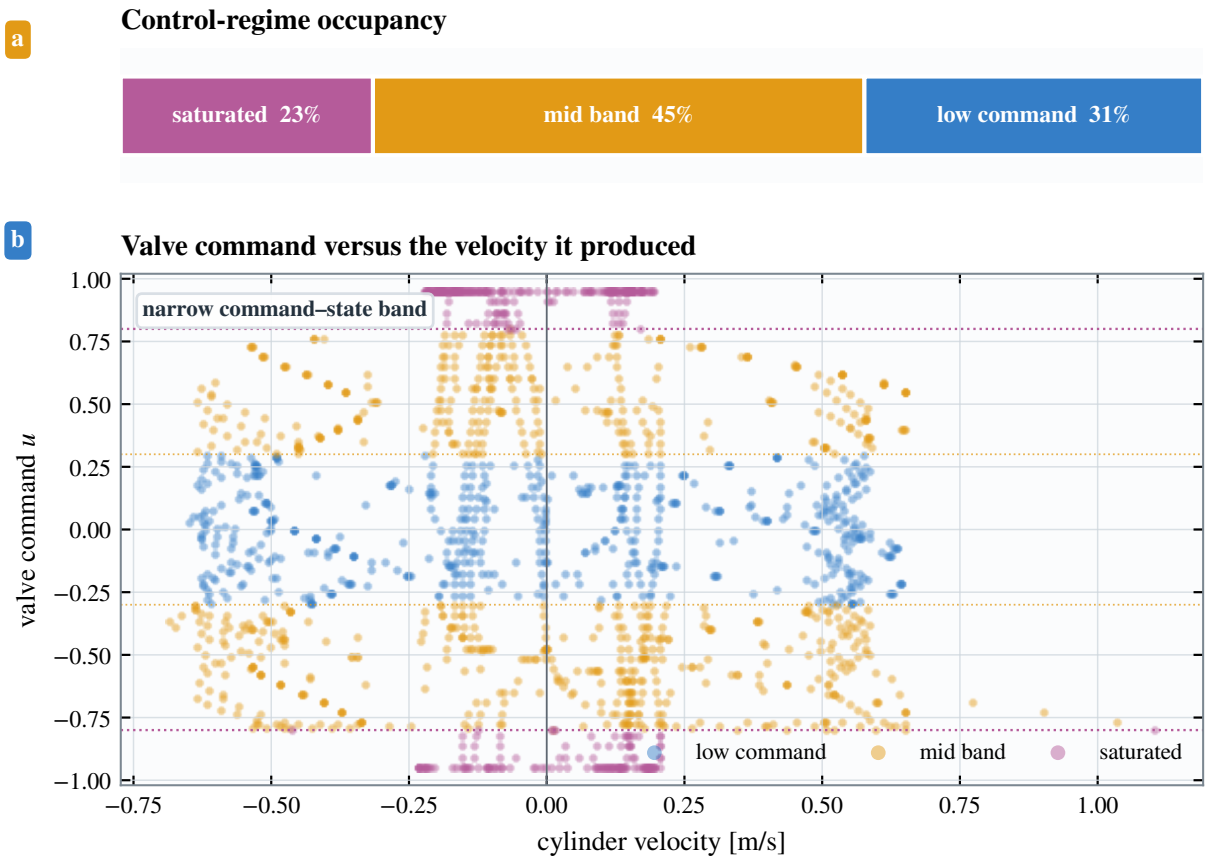

Figure 4.3: The closed-loop batch is diverse in control effort but not identifiable. (a) Fraction of time the valve spends saturated, in the mid band and in low-command creep. (b) The same run in the command–state plane, where the correlation that blocks identification is visible.

percent (Section 5.3.1). In closed loop, command is largely determined by the state history already given to the model, so the data do not separate command causality from state correlation [Ljung 99].

Figure 4.3 confirms that coverage alone is not the issue. The valve is saturated for about one quarter of a pooled run, operates at mid-authority for roughly half, and otherwise remains in low-command creep. Despite this spread, command and cylinder velocity remain concentrated along a narrow band. The data are diverse in control effort but not identifiable, motivating a separate open-loop experiment.

4.4.2 Design of the Excitation Signal

The excitation run lasts 64 s and follows the fixed schedule in Figure 4.2(c). One second of startup is simulated and then discarded, because the plant is settling from its initial condition and those samples describe the initialisation rather than the dynamics. Four sequential slots follow, each giving one cylinder ten seconds of excitation followed by a half-second zero-command gap. The gaps make the boundaries between single-axis segments unambiguous. A twenty-second phase then drives all four channels at once, and a final second of zero command closes the run. Commands are limited to ±0.90 of full authority.

An initial rapidly switching signal met the formal persistence condition but kept each piston near one stroke position. The final design instead uses long full-authority plateaus in the sequential phase and slow, high-authority motion in the simultaneous phase. A slow near-square cycle is combined with distinct medium-frequency channel components, with raised-cosine entry and exit ramps. This provides both input excitation and useful stroke coverage.

4.4.3 Plant Draws and Acceptance

The same excitation is applied to four plant draws taken from the same parameter space as the closed-loop campaign: nominal, randomized hydraulics, randomized mechanics, and fully randomized. Holding the input fixed and varying only the plant is what makes the four runs directly comparable, since any difference in the response is attributable to the parameters.

Each run is checked for stroke coverage before it is accepted. The collector requires at least 10 % of the available stroke on cylinders 1–3 and 8 % on cylinder 4, warns below a target of 30 %, and records the achieved coverage per channel in the run summary. The runs retained here cover between 12 % and 33 % of the stroke depending on the cylinder, which is above the acceptance floor but below the target. Cylinder 4 is the weakest, and improving its coverage is the clearest available next step for this campaign.

An earlier export was rejected because half its runs contained duplicated state trajectories. The acceptance checks in Section 4.10 now detect this failure directly.

4.5 Combining Closed-Loop and Open-Loop Datasets

The inverse model uses only closed-loop data because its target is the controller command for a measured state. The forward model is retrained on the strict open-loop export while retaining closed-loop replay at weight 0.25. Without replay, four excitation runs do not provide enough pose coverage.

Figure 4.4 shows the difference: closed-loop data covers the reachable range, while open-loop data concentrates at identifiable valve plateaus. The forward model therefore uses both.

For ROM training, the inverse and forward exports are joined by run identifier, time index and sample time. The join is accepted only when their sixteen-dimensional cylinder states match sample for sample. It contains 452 400 transitions at 0.05 s from 1 040 runs, split into 936/52/52 complete runs. Valve command is used only to supervise the ROM's internal valve head; it is not a runtime input (Section 5.4).

This aligned nominal-plant export trains the inverse, forward and ROM models. It predates the thesis and its collection script is unavailable, so its measurable contents and audits define its reproducibility boundary. Exact alignment, increment consistency and the identifiability failure were checked directly on the export.

The separate thousand-run randomized curriculum was collected for an earlier adaptation plan. No reported result is trained on or evaluated against it; the adaptation chain uses the IsaacLab collection of Section 4.7. The curriculum is retained as provenance and documented in Appendix B.

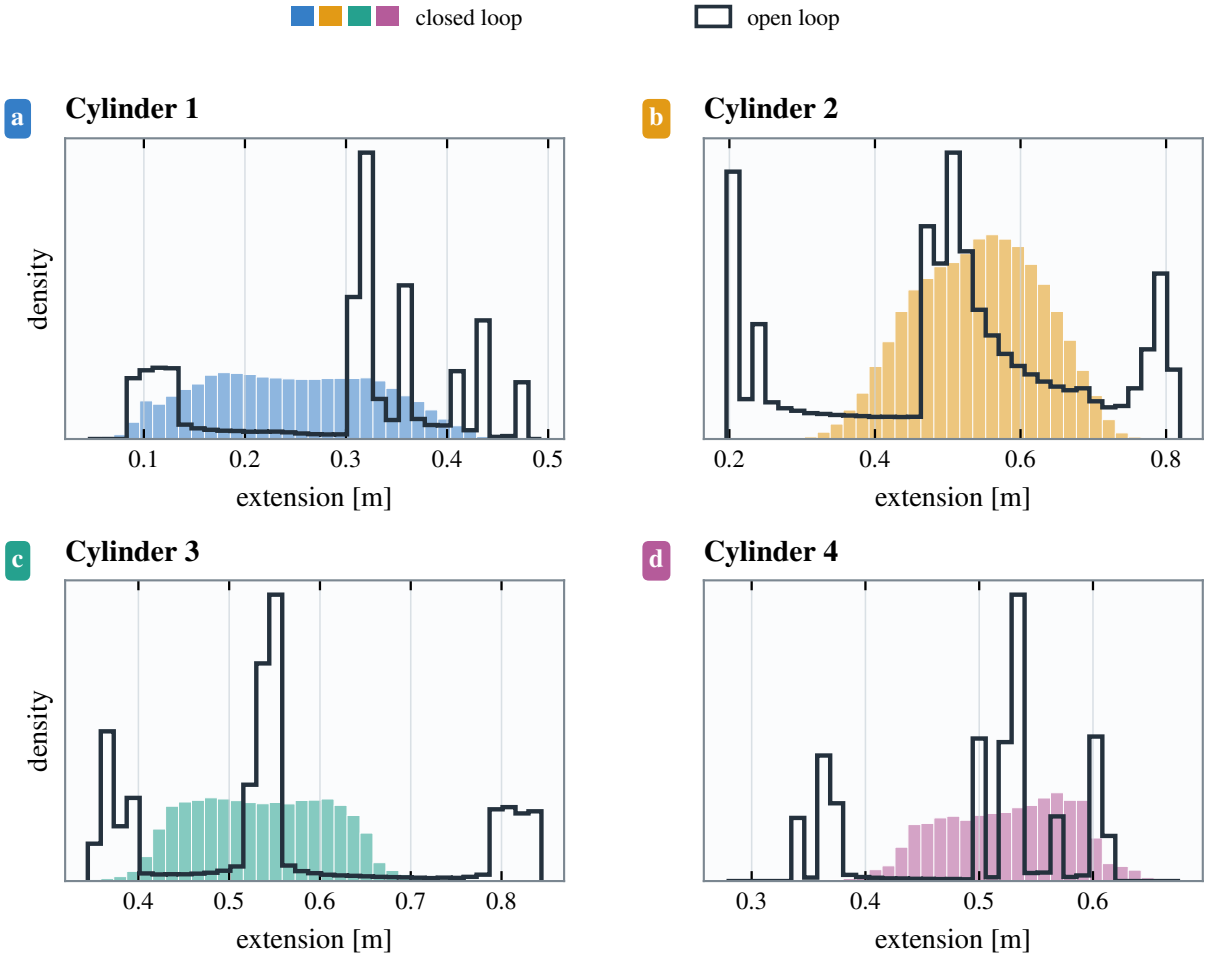

Figure 4.4: Extension coverage of the two campaigns, per cylinder and pooled over runs (closed-loop filled, open-loop outline). The closed-loop data fills the reachable range smoothly; the open-loop data concentrates into the plateaus where the valve is identifiable.

4.6 Closed-Loop Calibration Data for the Command ROM

The ROM drives a PhysX articulation whose compliance and contact response are not present in Simscape. Further Simscape training cannot remove the resulting deployment offset.

The calibration set therefore comes from closed-loop ROM–PhysX operation on the native 0.05 s grid. Its 4 084 transitions store the six-step runtime history and measured PhysX transition.

Only the command-gated correction head is trainable; all other parameters are frozen and the correction is capped at 3 mm. Original-data replay at weight 0.25 preserves wider pose coverage. The gate makes the correction zero at zero command, retaining stationary hold.

The trajectories predate the moving-verge benchmark and exclude cutter–ground contact. The bounded head limits, but does not remove, this coverage gap. Section 3.4.1 explains the consequence for the three-way comparison.

4.7 The IsaacLab Adaptation Collection

4.7.1 Provenance of the Adaptation Data

The models that carry the adaptation argument (the system-dynamics surrogate, the privileged factor encoder, and the AAM) describe the plant that the adaptive controller is closed around. That plant is not the Simscape model. It is the command ROM plus PhysX, the same four-dimensional cylinder command port used at deployment. A surrogate identified on Simscape data would be a model of a model, and the latent it defines would describe variation the deployment plant does not necessarily exhibit. The adaptation dataset is therefore collected *in* the deployment plant, and the frozen ROM, ID and FD checkpoints enter this chain only as fixed components, never as things to be retrained.

4.7.2 The Physical Factor Vector

The randomized quantity is a six-dimensional physical factor vector

$$\mathbf{e} = \Big[e_{\text{flow}},\ e_{\text{compliance}},\ e_{\text{seal}},\ e_{\text{pressure}},\ e_{\text{mass}},\ e_{\text{COM}_z}\Big], \tag{4.1}$$

with the ranges of Table B.3. Four coordinates are hydraulic and two mechanical, which is what makes the hydraulic-only, mechanical-only, and combined domains of Section 4.7.3 separable. Each candidate was admitted only after a one-factor sensitivity screen confirmed that moving it alone produces a measurable change in cylinder extension, velocity, pressure, or end-effector motion. Parameters whose effect could not be measured, or which could not be varied safely, were rejected rather than carried as decoration.

One omission is deliberate. Actuation delay is not a randomized coordinate. Delay in this system is an *emergent* property of flow, compliance, and load rather than a free parameter, so it is measured as a response property of each run (Chapter 7) instead of being supplied to any model as a known factor. Handing an estimator a factor that the real machine would never publish is the same mistake as handing it the true latent.

4.7.3 Domains and Factor Design

Runs are organized into five domains (nominal, hydraulic-only, mechanical-only, combined-training, and an *unseen* combined-test domain), and one factor vector is fixed for a complete run, so that no run mixes plants. Factor configurations are drawn with deterministic scrambled Sobol sequences [Sobol' 67] rather than independent uniform sampling, with explicit boundary anchors so that the corners of the space are represented and not merely approached, and with fixed per-domain seeds so the design is reproducible independently of the data it produced. Combined draws that compound into an excessive response gain are rejected at draw time rather than being collected and discarded later.

The resulting design is auditable: 98.3 % of the factor keys are unique, the active-factor matrix is full rank, every factor quartile is populated, and the factor keys of the training, validation, and test splits are disjoint. That last property is the one that matters most for the claims of Chapter 7: a test run does not merely come from a different episode, it comes from a plant the models have never seen.

4.7.4 Collection Procedure

Every run is collected through the same interface the policy uses. The DLS controller closes the loop on measured cylinder state and issues its command through the four-dimensional action port into the command ROM and PhysX. No joint-target bypass is used, and no PPO policy is used to collect identification data, because a policy trained on the same plant would reintroduce the closed-loop correlation that Section 4.4 spent a campaign removing.

Independent excitation is supplied by dedicated four-channel PRBS windows applied through that same action port, with a small amplitude and a hold of ten control steps, so that the excitation is causally independent of the state without ever leaving the safe envelope. Probe actions are logged separately from controller output, so any analysis can distinguish the two. Each run follows a fixed eleven-segment structure (settle, three postures, three PRBS windows, DLS task motion, reach-and-return, and stow), giving 11 304 control steps and 188.4 s of simulated time per run. The separate open-loop excitation collection of Section 7.2 uses a different schedule and runs to 11 040 steps. Three postures plus line, coupled-sine, and reach-and-return motion were verified in a pilot to give full rank over all four action channels under varied geometry and load.

Figure 4.5 summarizes the IsaacLab collection configuration and its frozen run-level outputs.

Figure 4.5: IsaacLab adaptation-data collection. Each run uses one fixed six-factor plant draw and combines DLS task motion with independent PRBS probes. State, action and factor keys are logged, audited and frozen into disjoint train, validation and held-out test splits.

4.7.5 The Frozen Dataset

The final collection contains 118 complete runs: 2 nominal, 40 hydraulic-only, 12 mechanical-only, 48 combined-training, and 16 unseen combined-test. Collection ran in four vectorized IsaacLab environments with per-environment factors and logging. Slots that tripped a safety or validity check were discarded and retried individually rather than being silently kept. All 118 runs passed the final collection gate, which checks finite physics, exact model-grid alignment, command rank, factor-response authority, state coverage, and saturation limits.

The dataset is then *frozen*: the raw runs are checksummed and the run-level split is fixed at 82 training, 20 validation, and 16 reserved combined-test runs, with zero factor-key leakage between them. The reserved sixteen do not influence history length, checkpoint, hyperparameter or architecture selection. They are scored only after each model is fixed and are treated as a single-use test set. Test results are reported with validation results, not in place of them.

4.7.6 Learning Timebase and Hydraulic State

The learning timebase is the actuator model grid, not the control clock. Samples are exact 50 ms ROM transitions taken at control indices $0, 3, 6, \ldots$: state row t and command row t predict state row $t + 3$. Nothing is interpolated. This is the same alignment rule as Section 3.5.1, and it is enforced rather than assumed, because a model trained across a mismatched interval is wrong by a fixed factor in the quantity it exists to supply.

The hydraulic state used by the system-dynamics models is

$$\boldsymbol{s}_t = \left[\boldsymbol{x}_t^{1:4},\ \dot{\boldsymbol{x}}_t^{1:4},\ \Delta\boldsymbol{p}_t^{1:4}\right] \in \mathbb{R}^{12}, \tag{4.2}$$

with joint position and velocity carried as *context* only, and the prediction target the next 50 ms state delta. Differential pressure is not a measured signal. At runtime it is the actuator model's recursive state, generated by the command ROM or frozen forward model (Section 3.3). The system-dynamics model and AAM may use it because it exists inside the deployed loop, but it inherits actuator-model error. Causal history is therefore retained to represent pressure transients that one sample cannot summarize.

4.8 Feature and Target Construction

4.8.1 The Actuator Models

The three learned actuator models take cylinder-space features, in the ordering and notation fixed in Section 5.1. Their input contracts differ because their questions differ. The inverse model receives the measured cylinder state, the logged rod forces, the desired increments, and the previously applied command, twenty-eight numbers in all. The forward model receives the applied valve command and the measured state, twenty numbers. The reduced-order model receives a six-sample history of state, previously predicted valve, and desired command, twenty-four numbers per step.

Three conventions apply throughout, and they are the same ones the models enforce at inference time.

Increment Targets

Every dynamic target is a within-run one-step increment, $\Delta \boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never an absolute state. This removes the large static component of the signals and, more importantly, makes the trivial persistence solution an explicit baseline that a model can be measured against rather than a hiding place it can settle into.

Causal Alignment

Features at step k contain only quantities available at step k. The extraction stage verifies this by recomputing every logged target from the within-run difference of the states and requiring exact agreement, a check reported in Section 4.10.

Time-Base Conversion

The open-loop campaign is logged at 0.01 s but the models run at 0.05 s, so five consecutive raw samples are combined into a single transition. The valve input attributed to that transition is the command held at the start of the window, which is the causally correct choice. The mean command over the window is also recorded, but only for analysis. Cylinder extension velocity is not logged directly in the closed-loop archive and is obtained by differentiating extension on the output grid, which is consistent with the trapezoidal identity that the strict data satisfies to $R^2 = 0.994$–0.9997 (Equation 5.6).

4.8.2 Observable History of the Adaptation Models

The adaptation models use the frozen IsaacLab collection. A history window carries only quantities produced by the deployed stack at runtime: the five joint positions, the five joint velocities, the four cylinder pressure differences and the four previously applied valve commands, giving the eighteen-value row of Section 5.6. The pressure entry is the learned actuator's own recursive state rather than a measurement (Section 3.3, Table 3.2); it is admitted because the model that generates it runs inside the deployed loop, and the consequence of that choice is examined where the estimator is evaluated. Four classes of signal are excluded by construction, and the exclusion is enforced by an explicit input mask rather than by convention:

- the physical factors $\mathbf{e}$ and the true latent $\boldsymbol{z}$, because recovering them is the estimator's job;
- any future observation, because the estimator is causal;
- the hidden internal state of the command ROM, because the deployment interface does not expose it;
- the desired task reference, so that the latent describes the plant rather than the trajectory being asked of it.

Windows are contained within a single run and never cross a run boundary, and each window carries the stopped-gradient target $\boldsymbol{z} = \mu(\mathbf{e})$ of its own run together with the run identifier, domain, split, and valid history bounds. Applied to the 118 frozen runs this yields 444 624 causal history ticks. What is built from them, how the history length is chosen, and how the result is validated downstream belong to Chapter 5.

4.9 Preprocessing and Data Splits

Splitting is done at the level of complete runs, never of individual samples. A random sample-level split would place transitions from the same trajectory on both sides of the boundary and report a validation score that mostly measures interpolation within a run. For the command-conditioned dataset the 1 040 runs are divided into 936 training, 52 validation, and 52 test runs, giving 407 160, 22 620, and 22 620 transitions respectively, with no run identifier appearing in more than one split. The IsaacLab adaptation collection is split by run *and* by factor configuration (82/20/16, Section 4.7), which is the stronger condition: separating runs alone would still let two runs of the same plant fall on opposite sides of the boundary and turn a claim about unseen plants into a claim about unseen episodes.

The strict open-loop export needs a different treatment because it contains only four runs, and holding one out would remove an entire plant draw from training. Instead each run is divided into twenty temporal blocks, every fifth block is assigned to validation, and a five-sample guard interval is discarded at each block boundary so that no 0.05 s transition window spans the split. A simple last-twenty-percent split was tried first and rejected: it changed the spread of the targets between training and validation by a factor of 1.3 to 2.3, because the final fifth of the run is the simultaneous excitation phase and is dynamically unlike everything before it. That split would have measured extrapolation to a different regime and called it generalisation.

Normalization statistics are computed on the training split only and stored inside each checkpoint, so that a model always normalizes with the statistics it was fitted with. Mixing statistics between checkpoints silently invalidates any comparison, and the loader enforces the stored contract rather than recomputing it.

Finally, samples are carried with an explicit validity flag rather than being silently dropped. In the strict export 93.7 % of samples are marked valid. The remainder are the excluded startup and shutdown seconds and the zero-command gaps. Keeping them in the archive with a flag, instead of deleting them, means the time base stays contiguous and the within-run difference checks remain meaningful.

4.10 Dataset Diagnostics

No dataset in this thesis is used before it passes an explicit audit, for the reason given at the end of Section 4.4. The audit of the strict open-loop export is reproduced in Table B.4.

The two rank figures are the ones that speak directly to requirement R2. The instantaneous command matrix has rank four, so no channel is a linear combination of the others, and the lagged regressor matrix built from the command history has rank eighty, so the excitation is persistently exciting to the order the models need. The condition number of that lagged matrix is about 4.6×10^3, which is finite and workable rather than comfortable. It is a consequence of using long plateaus, and it is the price paid for the stroke coverage those plateaus buy.

The exactness of the three increment checks deserves a word, since a maximum error of exactly zero invites suspicion. It is expected here: the targets are computed from the logged states by the same within-run difference that the check recomputes, so the

check verifies that the extraction stage did what it claims, not that the simulation is accurate. It catches misalignment, off-by-one indexing, and rows that have been reordered or concatenated across run boundaries, which are the failure modes that occur. Physical fidelity is a separate question, addressed by the validation study in Section 3.4.4.

Two limitations of the Simscape material remain. Stroke coverage reaches only 12 % on the head-pivot cylinder, which is also the cylinder the strict forward model predicts least well. The rod-force channel fails a piston-area consistency check on this export, so it carries zero supervised weight in the reduced-order model and no result rests on it.

The IsaacLab collection is audited against different failure modes. Design adequacy is checked before use (unique factor keys, full active-factor rank, disjoint keys across splits), physical validity per run (finite physics, exact model-grid alignment, full command rank, bounded saturation), and leakage structurally, with privileged channels removed by an explicit input mask rather than by trusting the feature list. Its limitation is scale: 118 runs over six factors is a sparse design, which is why the adaptation claims of Chapter 7 are reported with paired bootstrap intervals and per-run detail rather than pooled point estimates.

With the data described and audited, the next chapter turns to what was learned from it: the inverse and forward models that form the IF actuator path, the command-conditioned reduced-order model that replaces them in R, and the system-dynamics, encoder, and adaptation models built on the IsaacLab collection.

5. Learned Models and Online Adaptation

Chapter 4 established how the datasets were generated and why both closed-loop operation and open-loop valve excitation were required. This chapter develops the learned representations that consume them: the inverse-dynamics (ID) and forward-dynamics (FD) networks of the IF actuator path, the command-conditioned reduced-order model (ROM) that replaces them in R, and the system-dynamics surrogate, privileged factor encoder and Arm Adaptation Module (AAM) that together carry the adaptation argument.

Two commitments organise the chapter. The first is that the surrogate is kept strictly separate from the deployed actuator: the system-dynamics model is an instrument for establishing whether a compact dynamics latent exists and is useful, never a component of any control stack. The second is that no model is accepted on a supervised score, because a recurring result of this work is that such scores can be not merely optimistic but uninformative: a forward model reached a one-step R^2 near 0.999 while having learned almost nothing about valve authority (Section 5.3). Every model is therefore presented with its contract, training objective, and the offline evidence that accepted or refused it. All models operate on cylinder-space signals; the mapping to task space belongs to Chapter 3, the control integration to Chapter 6, and deployment-level results to Chapter 7.

5.1 Modelling Overview and Common Notation

All learned models act on the four hydraulic cylinders of the MULAG arm, indexed $i \in \{1, \dots, 4\}$ in the fixed order slew cylinder (*Drehzapfen*), boom cylinder I (*Ausleger I*), boom cylinder II (*Ausleger II*) and head pivot cylinder (*Messerkopf-Schwenk*). The passive fifth joint is not actuated and enters no model input. At the learned-model sample time $T = 0.05\,\mathrm{s}$ the discrete index k refers to the cylinder extension $\boldsymbol{x}_k$, the extension velocity $\dot{\boldsymbol{x}}_k$, the chamber pressure difference $\Delta\boldsymbol{p}_k$ and the logged rod force $\boldsymbol{F}_k$, each in $\mathbb{R}^4$. The valve command $\boldsymbol{u}_k \in [-1, 1]^4$ is the normalised spool command of the four proportional valves. The desired cylinder command produced by the task controller is the pair

$$\boldsymbol{c}_k = \left[\Delta\boldsymbol{x}_k^*,\ \Delta\dot{\boldsymbol{x}}_k^*\right] \in \mathbb{R}^8, \tag{5.1}$$

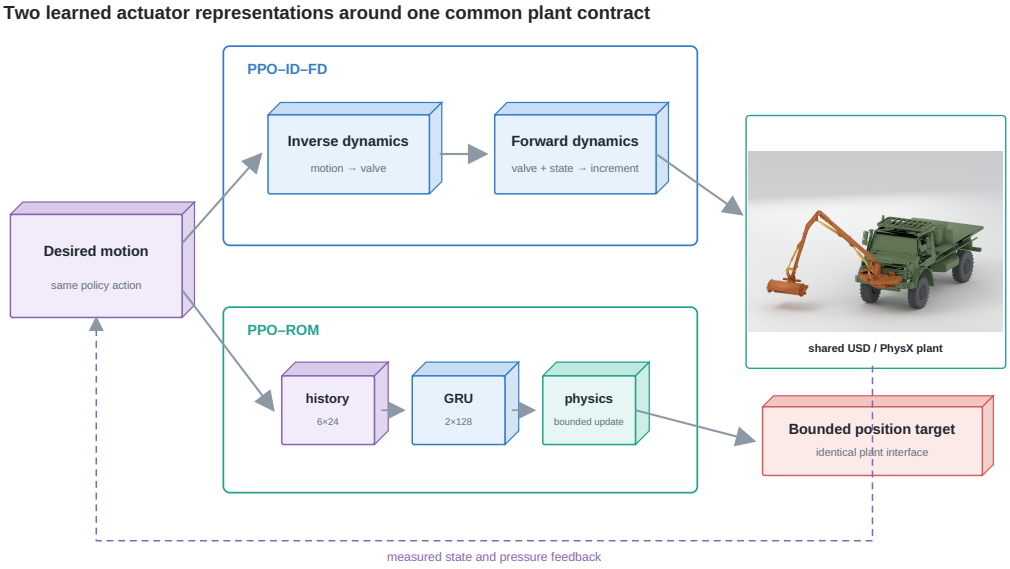

Figure 5.1: The two learned actuator representations as parallel lanes around one shared contract. Both consume the same desired cylinder command and the same measured cylinder state, and both produce the predicted increment that becomes the bounded PhysX target. The IF lane composes two separately trained networks; the R lane realises the command inside a single grey-box model whose valve signal is internal. The experimental variable between the two stacks is the actuator representation alone.

the extension and extension-velocity increments to be realised over the next sample. Predictions carry a hat, and $\Delta(\cdot)_k = (\cdot)_{k+1} - (\cdot)_k$ denotes a within-run one-step increment throughout.

Figure 5.1 places the two actuator paths side by side. In IF, the ID network maps the hydraulic state and a desired increment to a valve command and the FD network maps a valve command and the state to the predicted increment, so their composition $f_{\mathrm{FD}} \circ f_{\mathrm{ID}}$ predicts what a commanded increment will do. In R, one command-conditioned ROM replaces that composition: it receives the same $\boldsymbol{c}_k$, predicts the valve internally, and propagates pressure, velocity and extension through explicit physical updates. Both paths terminate in the same bounded PhysX cylinder target inside IsaacLab [Mittal 23] and are driven by the same policy under the same action contract [Schulman 17].

Three conventions apply to every model. All dynamic targets are within-run increments rather than absolute states, which removes the dominant static component and makes the persistence solution $\Delta\hat{\boldsymbol{x}} = \mathbf{0}$ an explicit, testable baseline rather than a hidden local minimum. Every checkpoint stores and applies the normalisation statistics of its own training data, never those of another checkpoint, because mixing them silently invalidates any comparison. And inputs at step k contain only quantities available at step k, including the previously applied command, with targets the measured transition to step $k+1$.

Two dataset families from Chapter 4 are consumed. The *closed-loop* family is the aligned Simscape export [The MathWorks, Inc. 24a] used for supervised ID and FD training and, after exact alignment, for the command-ROM dataset: 452 400 transitions at $T = 0.05\,\mathrm{s}$ from 1 040 runs, split by complete runs into 936 training, 52 validation and 52 test runs. The *open-loop* family is the strict valve-excitation export used to repair the FD model:

four 64-s runs at 0.01 s in which the same rank-4 multi-input excitation is applied to four distinct plant draws.

5.2 Inverse Dynamics Model

The ID network answers the question the low-level controller asks: given the current hydraulic state, which valve command realises a desired cylinder increment? Its contract is

$$\Delta \boldsymbol{u}_k = f_{\mathrm{ID}}\Big(\boldsymbol{x}_k,\, \dot{\boldsymbol{x}}_k,\, \Delta \boldsymbol{p}_k,\, \boldsymbol{F}_k,\, \Delta \boldsymbol{x}^*_k,\, \Delta \dot{\boldsymbol{x}}^*_k,\, \boldsymbol{u}_{k-1}\Big) \in \mathbb{R}^4, \tag{5.2}$$

a 28-dimensional input mapped to a valve-command *change*. The deployed command is the accumulated $\boldsymbol{u}_k = \mathrm{clip}(\boldsymbol{u}_{k-1} + \Delta \boldsymbol{u}_k,\, -\boldsymbol{u}_{\max},\, \boldsymbol{u}_{\max})$ with per-axis limits $\boldsymbol{u}_{\max} = [0.95,\, 0.95,\, 0.8,\, 0.8]$ stored in the checkpoint. Predicting the change rather than the absolute command conditions the problem on the previous operating point of the valve, which matters because proportional valves are direction- and history-dependent around their dead zone (Section 2.1.2). The desired increments in the training data are the *achieved* transitions of the logged runs, used as a proxy because no independent cylinder-space reference was logged.

The network is a multilayer perceptron with an input projection to width 512 followed by four residual blocks [He 16] with SiLU activations [Elfwing 18], layer normalisation [Ba 16] and dropout [Srivastava 14], implemented in PyTorch [Paszke 19]. Training minimised the smooth-L_1 loss [Huber 64] on normalised targets using Adam [Kingma 15], with samples in valve saturation up-weighted so the rare saturated regime is not ignored.

The supervised result was, on its face, excellent: per-valve validation R^2 between 0.9973 and 0.9996. That number means the network reproduces the logged valve commands from the logged states and achieved transitions. It does not mean the composite map has authority over the plant, because the achieved transitions in closed-loop data are themselves produced by the logged commands, so a model can score highly by exploiting that correlation without ever being asked to *cause* a transition absent from the data (Section 2.4.2).

5.2.1 Paired Control Fine-Tuning

The deficiency became measurable when the two networks were composed. For control, the relevant object is the *paired differential response*, the difference between what the FD predicts under the ID's command for the actual target and under its command for a zero target,

$$\begin{aligned} \boldsymbol{u}^{\mathrm{tgt}}_k &= f_{\mathrm{ID}}(\cdot,\, \boldsymbol{c}_k,\, \boldsymbol{u}_{k-1}), \qquad \boldsymbol{u}^{0}_k = f_{\mathrm{ID}}(\cdot,\, \boldsymbol{0},\, \boldsymbol{u}_{k-1}), \\ \Delta \hat{\boldsymbol{x}}^{\mathrm{pair}}_k &= f_{\mathrm{FD}}\Big(\boldsymbol{u}^{\mathrm{tgt}}_k, \cdot\Big) - f_{\mathrm{FD}}\Big(\boldsymbol{u}^{0}_k, \cdot\Big). \end{aligned} \tag{5.3}$$

Before fine-tuning this quantity did not track the requested increment at all: the normalised control error of the composite was 1.00, exactly the score of predicting nothing, with per-axis regression slopes of at most 0.004 (Table 5.1). Both networks were therefore fine-tuned jointly on $\mathcal{L}_{\mathrm{ctl}} = \|\Delta \hat{\boldsymbol{x}}^{\mathrm{pair}} - \Delta \boldsymbol{x}^*\|^2_{\mathrm{norm}} + \lambda_{\mathrm{ID}}\mathcal{L}_{\mathrm{ID}} + \lambda_{\mathrm{FD}}\mathcal{L}_{\mathrm{FD}}$, retaining the supervised terms to limit regression from the original data contracts. Crucially, the target blocks were shuffled across valid states during this stage, so the networks could no longer succeed

Table 5.1: Effect of paired control fine-tuning on the composite differential response (5.3), per cylinder axis on validation data. Before fine-tuning the composite had essentially no control authority despite near-perfect supervised scores of both networks; the composite normalised control error fell from 1.0011 to 0.1086.

	Regression slope		Correlation	
Cylinder axis	before	after	before	after
1 (slew)	0.003	0.818	0.135	0.920
2 (boom I)	0.002	0.928	0.090	0.967
3 (boom II)	0.001	0.849	0.037	0.933
4 (head pivot)	0.004	0.912	0.249	0.958

by reconstructing the logged trajectory and had to respond to targets the logged controller never issued in that state. After 30 epochs the composite control error fell to 0.109 and all four axes reached slopes above 0.8 and correlations above 0.92. The resulting checkpoint is the ID model deployed, frozen, in every IF and AIF experiment.

5.2.2 Transfer of the Frozen Inverse Model

The adaptation architecture of Sections 5.6 and 5.7 freezes this model and places a small correction around it. That design is only defensible if the frozen model is itself robust across the plant family, so the question was tested directly rather than assumed. The frozen ID was evaluated on a separate 64-run IsaacLab collection, made through the same deployment plant and spanning the same factor manifest (Section 4.7.2): 2 nominal, 16 hydraulic-only, 12 mechanical-only and 34 combined, the last being that collection's own 14 training, 4 validation and 16 unseen runs. It is not a subset of the 118-run adaptation collection and shares no run with it. All 64 are held out in the strict sense, because the ID is trained on Simscape exports alone and has never seen an IsaacLab transition, so the entire collection lies outside its training distribution by construction; the internal split is named only to show the result does not rest on one part of it. Runs were teacher-forced at each ROM tick and scored on the quantity the model actually produces, the valve command.

Table 5.2 reports the outcome. Against a recorded valve signal of root-mean-square magnitude 0.4350, the ID reproduces the command with an error of 0.1739, so it accounts for roughly 84 % of the valve-command variance. More importantly for the architecture, that accuracy is almost flat across domains: the spread between the best and worst domain mean is a factor of 1.09, and the ordering does not follow plant severity, since the randomised-hydraulics domain scores better than the nominal one. Robustness here rests on all 64 runs rather than on a ratio to the nominal domain, which contributes only two runs and is far too thin to anchor one.

Three limitations qualify that conclusion and are carried forward rather than buried. First, judged as a one-step state predictor rather than as an inverse model, the ID is only marginally better than a trivial baseline: the mean extension error of 1.002 mm compares with 1.143 mm for a zero-order hold over the same 50 ms step, an improvement of 12 %, and on velocity the zero-order hold is in fact 8 % better. This is not a contradiction but a

Table 5.2: Transfer of the frozen inverse model across deployment domains, 64 runs, teacher-forced at the ROM tick. The error is on the valve command, the model's own output. The recorded valve signal has RMS magnitude 0.4350. The spread across domains is a factor of 1.09; nominal is not the easiest domain.

Domain	runs	valve RMSE	s.d.	worst run
combined	34	0.1739	0.0311	0.2917
hydraulic	16	0.1672	0.0170	0.2174
mechanical	12	0.1828	0.0129	0.2082
nominal	2	0.1829	0.0155	0.1984

statement about which metric is meaningful for an inverse model, and both are reported so the reader can judge. Second, recursive open-loop rollout of the composite is unstable in 48 of the 64 runs. Divergence is bimodal and is discriminated by valve saturation, whose mean occupancy is 0.989 in diverging runs against 0.223 in stable ones, and not by domain; it is a property of open-loop integration into the stroke limit and does not bear on the one-step, closed-loop use the controller makes of the model. Third, supplying the measured pressure difference instead of the training mean makes the model *worse*, 1.907 mm against 1.311 mm, so the mean-pressure deployment contract is a deliberate choice rather than an omission.

5.3 Forward Dynamics Model

The FD network predicts the one-step cylinder response to a valve command,

$$\left[\Delta\hat{\boldsymbol{x}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\widehat{\Delta\boldsymbol{p}}_k\right] = f_{\mathrm{FD}}\Big(\boldsymbol{u}_k,\ \boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \boldsymbol{F}_k\Big) \in \mathbb{R}^{12}, \tag{5.4}$$

using the same residual trunk as the ID network. Supervised pretraining on the closed-loop export, with an auxiliary recursive rollout loss [Nagabandi 18], reached grouped one-step R^2 near 0.999 for the extension increments.

5.3.1 Assessment of the Supervised Score

That score was misleading in a way worth stating precisely, because it is the clearest instance of the chapter's organising caution. A model intended to give a policy gradient information about actuation must respond to the *valve*, not merely to the state history. Zeroing the valve input of the pretrained FD barely changed its long-horizon rollout error, a valve-zero error ratio of about 0.94 at a 0.5 s horizon: removing the input the model exists to explain made it very slightly *better*. The network had learned to extrapolate the smooth kinematics of closed-loop operation, in effect a sophisticated persistence model, because in feedback data the controller makes command and state history nearly redundant. Ablations on the source exports confirmed that the valve was not causally identifiable from the closed-loop distribution at all, so no amount of retraining on that distribution could repair it. The data had to change.

The repair dataset is the strict open-loop excitation export. All four valves receive a rank-4 multi-input sequence of long, full-authority plateaus, exciting one cylinder at a

time in the first four segments and all four simultaneously in the last, applied to four distinct plant draws. Validation established the properties the closed-loop export lacked: all four state trajectories are distinct, valve history alone predicts the pressure response with $R^2 \approx 0.80$–0.85, the valve regressors have variance-inflation factors of only 7–10, and pressure-relief saturation occupies 9–22 % of samples, so the saturated regime is represented but not dominant. An earlier excitation export was rejected outright because half its runs duplicated state trajectories and the valve was again not identifiable, a reminder that excitation data must itself be audited rather than assumed informative [Isermann 11].

Starting from the paired-control checkpoint, strict retraining minimised

$$\mathcal{L}_{\mathrm{FD}} = \mathcal{L}^{\mathrm{strict}}_{\mathrm{Huber}} + 0.25\,\mathcal{L}^{\mathrm{MAT}}_{\mathrm{Huber}} + 2.0\,\mathcal{L}^{(10)}_{\mathrm{roll}} + 10.0\,\mathcal{L}_{\mathrm{kin}}, \tag{5.5}$$

combining the one-step loss on the strict data, a replay term on the original closed-loop data, a 0.5 s recursive rollout loss, and a kinematic consistency term enforcing the trapezoidal identity

$$\Delta \boldsymbol{x}_k = T\,\dot{\boldsymbol{x}}_k + \tfrac{1}{2}\,T\,\Delta\dot{\boldsymbol{x}}_k, \tag{5.6}$$

which holds in the measured strict data with $R^2 = 0.994$–0.9997 and ties the predicted extension and velocity increments to each other. One initially attractive regulariser was evaluated and rejected: anchoring the model to a constant valve Jacobian. Measured valve authority varies by a factor of 15–75 across pressure quartiles and can change sign, so pinning a single global gain produced a pleasing mean response while collapsing one-step accuracy. This is consistent with the strongly operating-point-dependent valve–flow–pressure characteristics of hydraulic servo systems [Merritt 67] and motivates the pressure-dependent structure adopted in the ROM.

Figure 5.2 shows the outcome. Strict retraining reduces the 0.5 s extension rollout RMSE from 75.2 mm to 12.4 mm and the 1.0 s RMSE from 157.9 mm to 27.3 mm, and, decisively, raises the valve-zero error ratio from 0.94 to 1.81: removing the valve input now roughly doubles the rollout error, which is the sensitivity a control-relevant forward model must exhibit. Per-axis one-step extension R^2 improved from the range $[-5.7,\ 0.60]$ to $[0.76,\ 0.98]$. Two honest limitations remain. At horizons below 0.25 s the zero-order hold is still competitive, simply because extension does not move far in 50 ms; the learned model earns its value at the horizons over which a target must be committed. And the velocity-increment outputs remain poorly predicted on two axes, dominated by relief transients a memoryless 50 ms model cannot resolve. The deployed controller therefore integrates only $\Delta\hat{\boldsymbol{x}}$, the accurately modelled and directly validated quantity, and treats the velocity output as diagnostic. That restriction becomes important again in Section 5.5.

5.4 Reduced-Order Model

The IF path needed three repair stages before its composition had authority. The R path instead replaces the composition with a single command-conditioned reduced-order model trained end to end on the quantity the controller consumes. Two decisions distinguish it from a generic sequence model. It is *grey-box*: pressure, force, velocity and extension are propagated through explicit physical updates with trainable effective parameters, and neural components supply only what the physics cannot, namely the valve realisation, the unmodelled load and bounded residuals [Karniadakis 21, Lutter 19]. Encoding the

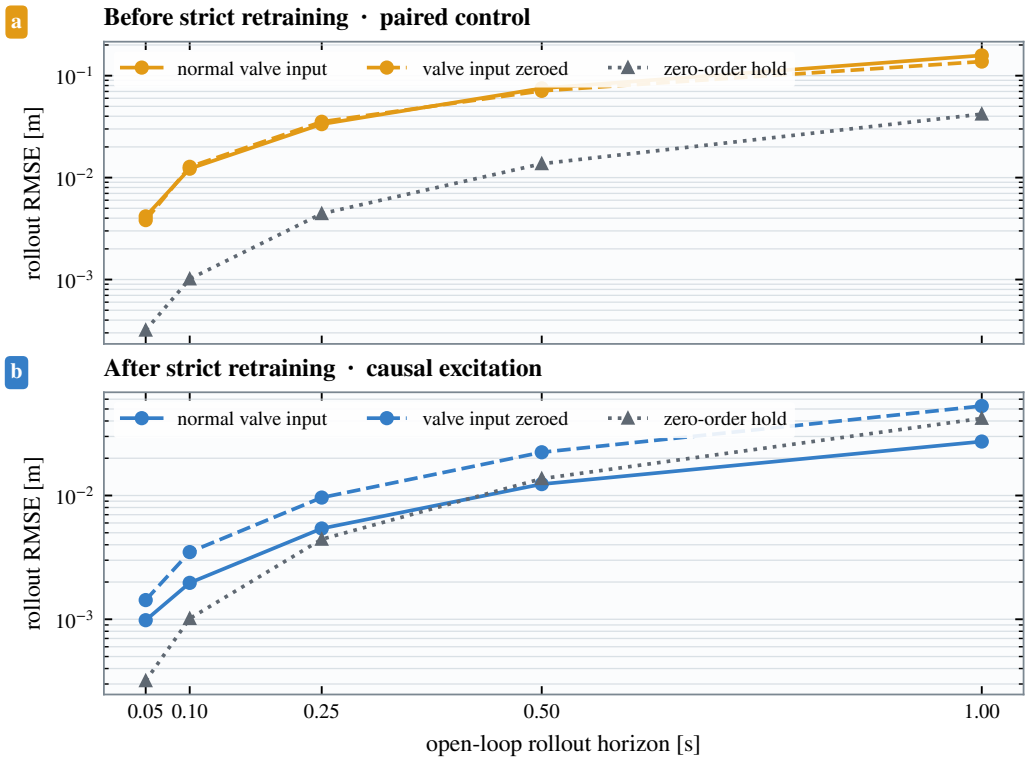

Figure 5.2: Open-loop extension rollout RMSE on held-out strict-excitation data against rollout horizon, before (orange) and after (blue) strict retraining. Dashed curves show the same model with the valve input zeroed, so the gap between solid and dashed is the model's usable valve authority. The paired-control FD is insensitive to its valve input, whereas the strict FD degrades by a factor of 1.8–1.9 when the valve is removed and outperforms the zero-order hold baseline (grey) beyond 0.5 s.

orifice-flow and continuity structure [Merritt 67] makes the pressure dependence of valve authority architectural rather than something to be rediscovered from data. It is also *recurrent*: a GRU encoder [Cho 14] over 0.25 s of history replaces the memoryless state input of the FD, in the spirit of learned actuator networks [Hwangbo 19], because hydraulic response depends on recent pressure and command history that a single sample cannot represent. The model consumes six samples of a 24-dimensional vector comprising the cylinder state, the *previously predicted* valve command fed back as recurrent state, and the current desired command; the actual valve appears only as a training target for the internal valve head, and at runtime no valve signal enters the model.

Figure 5.3 shows the computation. The encoder latent and the command feed an actuation head whose output passes through a learnable smooth dead-zone element with per-cylinder trainable width and asymmetric gains, reflecting the dead band and direction-dependent gain of proportional valves. Model-based designs treat the same nonlinearity by inverting an identified dead band analytically [Yang 26]; training its parameters per cylinder instead removes the need for a separate identification experiment. The resulting internal actuation

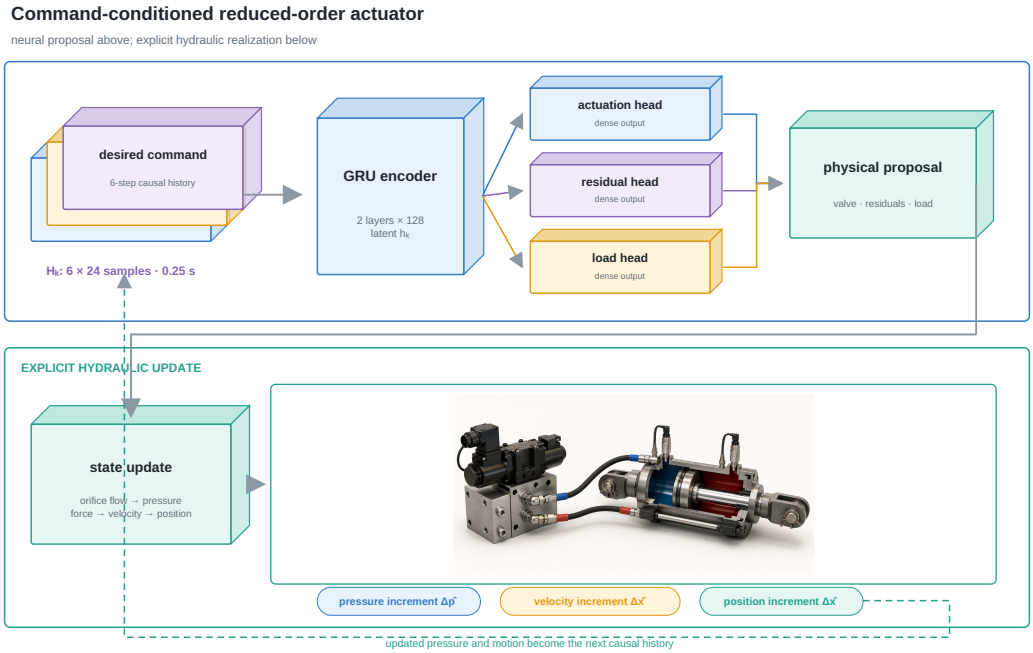

Figure 5.3: Structure of the command-conditioned reduced-order model. The 0.25 s history is normalised and passed through the recurrent encoder, whose hidden state carries pressure and command transients across the six steps. The final latent feeds three heads: the actuation head realises the command as an internal valve signal, the residual heads supply bounded corrections, and the load head the configuration load. The state is then propagated through the explicit pressure, velocity and position updates with trainable effective hydraulic parameters, so the neural parts propose and the physics disposes.

$\hat{a}_i \in [-1, 1]$ drives an orifice-flow and continuity update per cylinder,

$$q_i = \hat{a}_i \, k_{q,i} \sqrt{\tfrac{1}{2}\big(p_{s,i} - \operatorname{sgn}(\hat{a}_i)\, \Delta p_i\big)_+},$$
$$\Delta\hat{p}_{i,k+1} - \Delta p_{i,k} = \frac{4\beta_i}{V_i(x_i)}\Big(q_i - A_i \dot{x}_i - k_{\ell,i}\, \Delta p_i\Big)\, T \;+\; r_{p,i}, \tag{5.7}$$

with effective bulk modulus β_i, supply pressure $p_{s,i}$, flow gain $k_{q,i}$, leakage coefficient $k_{\ell,i}$, piston area A_i and extension-dependent chamber volume $V_i(x_i)$; the smoothed pressure margin $(\cdot)_+$ keeps the square root differentiable near the supply limit. Pressure then produces force and force produces motion,

$$\hat{F}_i = \Delta\hat{p}_{i,k+1} A_i + r_{F,i}, \qquad \ddot{x}_i = \frac{\hat{F}_i - c_{v,i}\, \dot{x}_i - \ell_i(\boldsymbol{h}_k)}{m_i},$$
$$\dot{\hat{x}}_{i,k+1} = \dot{x}_{i,k} + \operatorname{sat}\Big(T\, \ddot{x}_i + r_{v,i}\Big), \qquad \hat{x}_{i,k+1} = x_{i,k} + \tfrac{T}{2}\Big(\dot{x}_{i,k} + \dot{\hat{x}}_{i,k+1}\Big) + r_{x,i}, \tag{5.8}$$

where $c_{v,i}$ is a viscous friction coefficient, m_i an effective modal mass and $\ell_i(\boldsymbol{h}_k)$ a neural load head capturing the configuration-dependent gravitational and coupling load. All physical parameters are trained in log-space with hard clamps to plausible ranges, so they adapt to the data without leaving their meaning behind. The neural residuals are tanh-bounded to 10 % of the pressure step limit, 10 % of the velocity step limit and 1 cm respectively: large enough to absorb model error, too small to let the network bypass the

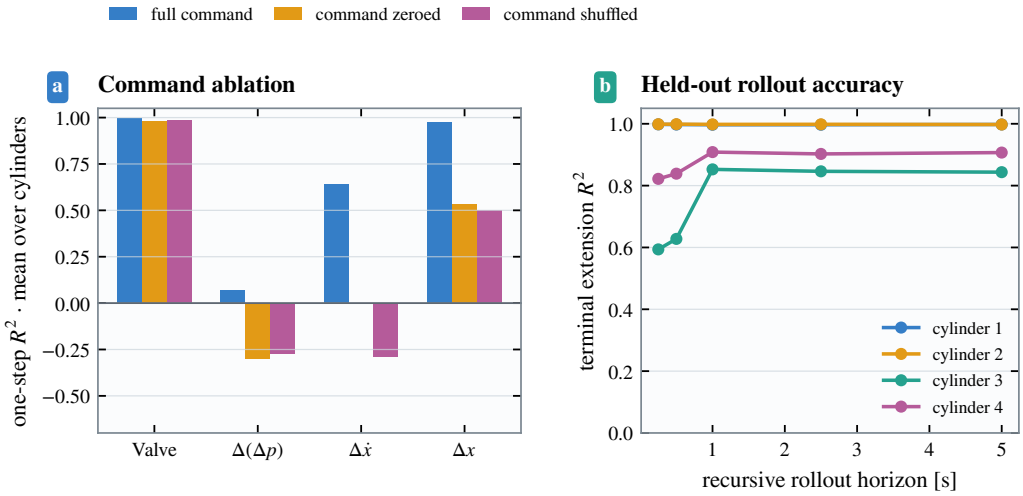

Figure 5.4: Offline validation of the command ROM on held-out runs. (a) One-step R^2 per output group, averaged over the four cylinders, with the full desired command, with the command zeroed, and with the command shuffled across the batch. The collapse of the velocity and extension scores without the true command demonstrates command authority. (b) Terminal extension R^2 of recursive rollouts up to 5 s per cylinder.

physics. This bounded-residual layering is the central robustness argument of the grey-box design, because even a badly trained residual cannot produce an unphysical state jump.

Training follows a four-stage rollout curriculum with horizons $H \in \{1, 5, 20, 50\}$, each stage optimising weighted one-step losses plus a recursive rollout loss at its horizon, with scheduled sampling ramping from teacher-forced to 75 % self-fed [Bengio 15]. The residual heads are penalised explicitly so that explanatory burden stays on the physical terms, and the logged force channel receives zero supervised weight because it failed a piston-area consistency audit; supervising a head on a label that violates the model's own structure would corrupt the pressure chain.

Three findings from the offline acceptance report (Figure 5.4) matter for the control experiments. The model has *command authority*: with the true command, mean one-step R^2 is 0.998 for the internal valve, 0.64 for velocity and 0.97 for extension, while zeroing or shuffling the command collapses the velocity score below zero and extension to about 0.5; the 4×4 Jacobian of predicted extension increment with respect to the commanded increment is diagonally dominant with condition number 2.14. It is *stable* in rollout, with terminal extension R^2 of 0.998, 0.998, 0.84 and 0.91 per cylinder at 5 s, so R training rollouts do not ride on a diverging model. And it has an *honest weakness*: one-step pressure-increment R^2 is only 0.06–0.09, so the internal pressure state is best understood as a latent that supports the velocity chain rather than as a calibrated pressure estimator. That weakness recurs in Section 5.5 and turns out to matter there.

5.4.1 Closed-Loop Calibration Against the Deployment Plant

Everything above identifies the ROM against Simscape, but at runtime the model does not drive Simscape. It drives a PhysX articulation whose contact behaviour, solver and effective compliance are its own, and a small systematic difference between the extension

the ROM commits to and the extension PhysX realises is invisible from Simscape data by construction. The promoted checkpoint therefore adds a bounded, command-gated correction to the extension update,

$$\hat{x}_{i,k+1} \leftarrow \hat{x}_{i,k+1} + \delta_i(\boldsymbol{h}_k, \boldsymbol{c}_k), \qquad |\delta_i| \le 3\,\mathrm{mm}, \qquad \delta_i\Big|_{\boldsymbol{c}_k=\boldsymbol{0}} = 0, \tag{5.9}$$

trained on 4 084 closed-loop transitions collected in IsaacLab on the five plant-reachable reference shapes, with the Simscape dataset retained as a replay term. Every other parameter, the whole flow, continuity, force and integrator chain, is frozen during this stage, so the identified hydraulics are preserved and only a deployment offset is placed on top of them. The gating keeps the stationary hold contract intact: at zero command the correction is exactly zero, so a commanded hold remains a hold.

This is why the command ROM, and not the ID–FD pair, serves as the deployment plant of Section 3.4.1. It is also a genuine asymmetry between the two actuator paths, since the ROM is the one model that has seen the plant it is deployed in, and it must be restated whenever their tracking numbers are compared in Chapter 7. The command ROM is frozen for the final comparison and conditioned on no latent; the adaptation mechanism below acts on the inverse model of the IF path and leaves this model untouched.

5.5 The Privileged Latent and Its Limits

The three actuator models above are alternatives for the same job. The system-dynamics model is not: it is an *instrument*, a learned approximation of the deployment plant itself, the command ROM followed by PhysX, trained from scratch on the IsaacLab collection of Section 4.7 and never used as the actuator of any stack. Its purpose is to answer whether a compact factor-dependent context exists at all. If it does not, there is nothing for an online estimator to recover and the adaptation argument stops.

The contract is a 50 ms transition. From the observable state, the applied action and a causal history the model predicts the next increment of the twelve-dimensional hydraulic state, with joint position and velocity as context. The action is the pair of consecutive valve commands $[\boldsymbol{u}_{k-1}, \boldsymbol{u}_k]$, which is what the command ROM itself consumes at a tick, so the model is asked the same question the plant is. It carries a belief module over a six-tick history, because a single sample cannot summarise the pressure state that drives the next transition, and it is trained with a five-transition unroll so that it is optimised for recursive use. The matched control SD_0 is identical except that no latent is supplied.

The privileged encoder $\boldsymbol{z} = \mu(\boldsymbol{e})$ compresses the six-dimensional physical factor vector of Section 4.7.2 into a two-dimensional latent. It is trained *jointly* with the latent-conditioned model SD_z, so the representation is shaped by what helps predict rollout rather than by a reconstruction objective. At a horizon of 20 ticks, one second of plant time, SD_z is evaluated with the true latent against five controls constructed from the training pool alone: zero, shuffled, random, the mean encoded latent, and the nominal factor setting. The test is paired per run and reported with bootstrap confidence intervals on validation and on the sixteen reserved test runs separately.

On the pooled state error the true latent wins on both pools. Validation rollout error falls by 7.20×10^5 with a 95 % interval of $[-1.03 \times 10^6, -4.30 \times 10^5]$ over 20 runs, and

Table 5.3: Per-channel decomposition of the paired latent advantage at a 20-tick horizon, pooled over 36 runs. Values are the mean paired difference between the latent-conditioned model and the no-latent control, with a 95 % bootstrap interval; a negative value favours the latent for the pooled state error, and a positive value favours it for the per-channel errors as signed here. The advantage is confined to the pressure channel.

Channel	mean paired difference	95 % interval	excludes zero
extension	5.2×10^{-5}	$[-3.6 \times 10^{-5},\ 1.4 \times 10^{-4}]$	no
velocity	-2.4×10^{-4}	$[-7.1 \times 10^{-4},\ 2.4 \times 10^{-4}]$	no
pressure	1.75×10^{5}	$[2.1 \times 10^{4},\ 3.3 \times 10^{5}]$	yes

the reserved test runs fall by 4.41×10^5, interval $[-7.61 \times 10^5,\ -1.74 \times 10^5]$ over 16 runs. Neither interval crosses zero.

That result, taken alone, would overstate the case, and the per-channel decomposition in Table 5.3 is the reason. The pooled statistic weights extension, velocity and pressure equally in normalised units, but the raw pressure magnitudes are around six orders larger, so the pooled figure is effectively a pressure statistic. Decomposed by channel, the advantage of the true latent lives entirely in pressure. On extension and velocity, the two channels the deployed controller actually integrates (Section 5.3), the paired intervals straddle zero on both pools, and the velocity mean difference even favours the no-latent control.

What this establishes must therefore be stated narrowly. A two-dimensional privileged context does carry information about held-out hydraulic rollout that the state and action do not, which justifies freezing the representation and attempting online inference. It is not evidence that the AAM works, because it uses the true $\boldsymbol{z}$ rather than an estimate, and it is not evidence of any control improvement. Most importantly, the information it carries is about the channel the ROM already predicts worst and the controller deliberately does not integrate. Section 5.7 returns to this, because it is the best available explanation for what the closed-loop experiments later find.

The representation is then frozen. The encoder, SD_z, the belief module, the normalisers, the input mask, the split and the rollout protocol are written to a frozen store with a recorded protocol stamp, and everything downstream reads that copy.

5.6 Arm Adaptation Module

The AAM estimates the latent from history that exists at deployment, $\hat{\boldsymbol{z}} = \mathrm{AAM}(\boldsymbol{H}_t)$. Its input row is eighteen-dimensional, comprising five joint positions, five joint velocities, four cylinder pressure differences and the four previous valve commands. Twenty consecutive 50 ms rows form the window, so the estimator reads 1.0 s of past and 360 inputs in total, and the promoted architecture is a causal one-dimensional convolutional network of width 64 and depth 4 over that window.

The pressure entry deserves a precise description, because it carries most of the hydraulic factor information and it is not a measurement. The four pressure differences are the learned actuator's own recursive pressure state, produced by the command ROM in R and by the frozen forward model in AIF, and they are available at every deployment step because the actuator model that generates them runs inside the deployed loop. Nothing

privileged enters: the factor vector, the true latent, the desired reference, future rows and the model's hidden recurrent state are excluded by construction, and the previous rather than the current valve command is used so that the row is causal at the instant the estimate is needed. History length was selected from a predeclared set of 5, 10, 20 and 40 ticks on validation runs only, taking the shortest history that remains statistically comparable with the best downstream rollout. The selected checkpoint has a validation latent RMSE of 0.802 and a reserved-test RMSE of 0.734, a substantial reduction against a constant-prediction baseline on runs the estimator never saw.

Deployment behaviour is fixed by the online runtime. Inference runs every three control steps, placing it on the 20 Hz model-tick grid inside a 60 Hz control loop, and the first valid estimate appears only after twenty complete rows have accumulated; until then the adapter is bypassed and the frozen inverse model's action is returned unchanged. A past-only rolling mean over five predictions smooths the output, adding 250 ms of lag. That smoother is causal by construction, unlike an earlier variant that scored the estimator using a full-run mean of held-out predictions, which is not a deployable estimator because it uses the whole run to predict any instant within it. Every number reported here comes from the corrected causal score, and keeping the superseded verdict visible and labelled is deliberate: the failure mode is easy to reintroduce.

5.6.1 What the Latent Actually Encodes

A two-dimensional latent cannot represent a six-dimensional factor vector, and it is worth measuring what it does represent rather than assuming a distributed code. Linearly decoding the six physical factors from the per-run mean estimate gives $R^2 = 0.956$ for the seal-condition factor and little else: 0.448 for compliance, 0.114 for flow, 0.081 for supply pressure, 0.056 for the centre-of-mass offset and -0.008 for the mass scale. The deployed estimator is, to a good approximation, a seal-condition fingerprint. It is also *static*: the estimate is stable within a run, which the temporal consistency term encourages, but it does not refine as evidence accumulates.

Two consequences follow. The first is that no latent dimension is named after a physical parameter anywhere in this thesis on the basis of a scatter plot; the one relationship asserted here is the one that was measured. The second is that this is a recognised failure mode rather than an idiosyncrasy of this system: history-based adapters that compress many factors into a single low-dimensional latent are known to lose robustness precisely where the factors act through different channels [Liu 25]. The original rapid-motor-adaptation argument treats the low-dimensional projection as a virtue, on the grounds that parameters which covary with identical effects on observables need not be separated and perfect system identification is not required [Kumar 21]. That defence holds only when the surviving direction is the one the controller needs. Here the surviving direction is seal condition, which acts through pressure, and Section 5.5 has already shown that pressure is the channel the controller does not integrate. Work that instead estimates named physical quantities in a belief filter, including seal leakage and valve gain degradation, retains that interpretability by construction and pays for it with an explicit plant model [Hu 26].

5.7 Latent-Conditioned Inverse-Model Adapter

The adapter is where adaptation reaches the plant. The frozen inverse model produces a base command $\boldsymbol{u}_{\text{base}} = \text{ID}(\boldsymbol{s}, \boldsymbol{r})$ and the adapter adds a small bounded residual conditioned on the estimated latent,

$$\boldsymbol{u} = \text{clip}\Big(\boldsymbol{u}_{\text{base}} + g_\psi(\boldsymbol{s}, \boldsymbol{r}, \boldsymbol{u}_{\text{base}}, \hat{\boldsymbol{z}})\Big). \tag{5.10}$$

The inverse model, the forward model and the policy stay frozen *while the adapter is trained*. The deployed policy of AIF is itself conditioned on the estimated latent and is trained separately, so the stack does not reduce to IF when the residual is disabled. The residual is bounded by construction and scaled by a declared factor, so a warm-up period or an invalid latent zeroes the correction and restores the non-adaptive actuator path rather than producing a third, untested behaviour. The latent-conditioned policy is unchanged by that fallback, so the resulting system is IF in its actuator path but not in its actor; recovering IF exactly requires switching in the 66-input baseline policy as well. What the fallback buys is a single-variable ablation *within* the adaptive checkpoint, which is the controlled comparison used in Chapter 7.

5.7.1 Open-Loop Evidence

Isolated from the control loop, the adapter does benefit from the latent, and the effect is small but consistent. Three information-matched arms were trained under identical conditions, differing only in what occupies the latent input: zeroed, the AAM estimate, and the privileged true latent. Across five seeds the privileged latent improves validation loss over the zeroed arm in 5 of 5 seeds, with a mean paired improvement of 0.0037, and the AAM estimate does so in 5 of 5 seeds with a mean paired improvement of 0.0118. The differences amount to a few tenths of a percent, so the sign, reproduced across every seed in a paired design, is the result rather than the magnitude.

A companion experiment tested the natural hypothesis that the latent should help more once the observation is made insufficient, by masking the pressure and force features so that plant identity can no longer be read off the input directly. It did not survive reseeding: under masking the privileged latent helps in only 2 of 5 seeds and the seed-to-seed spread grows by roughly an order of magnitude. A single-seed pilot had shown a clean reversal of the ordering, and it is recorded here as noise rather than as a finding, because the paired five-seed design does not reproduce it.

5.7.2 Closed-Loop Evidence, and Why It Is Null

Inside the control loop the picture changes. Evaluation-time ablations on one trained checkpoint compared the full stack against a zeroed latent, a shuffled latent, a frozen latent, an in-distribution mean latent and a sham plant change, which together separate whether any effect is due to the latent's presence, its content, or its online updating. No contrast produced a consistent improvement: the differences in lateral error, cut-height error and impact counts are small relative to run-to-run variation and change sign across the comparisons.

The explanation is a property of the plant rather than of the estimator, and it was found by measuring before drawing a conclusion. Across the scored window the first cylinder's valve

command is pinned at its limit for roughly 92 % of the episode, the second for 67–82 % and the fourth for 50–57 %, each measured against its own per-axis limit rather than a common one. Control-effort ratios before and after the scheduled plant change lie between 1.005 and 1.009, confirming that no headroom is released. A bounded additive correction cannot express itself through an actuator that is already at its limit, so the experiment as posed could not have detected adaptation regardless of the quality of $\hat{\boldsymbol{z}}$. This is the classical interaction between adaptation and input saturation, in which an adaptive law loses the authority it needs exactly when the plant most requires it and dedicated anti-windup compensation becomes necessary [Sofrony 24].

One further measurement shows that the saturation is not an artefact of a poor setting. Reducing the lead cap to relieve the windup does lower the first cylinder's saturation from 0.91 to 0.14, but degrades cut height by a factor of three: the actuator is pinned because the task sits close to the plant's capability limit (Section 3.6.2), not because a gain was chosen badly.

5.7.3 The Chain Verdict

The chain gate asks whether the deployable estimate retains a worthwhile share of the advantage the true latent buys, by comparing $SD_0(\boldsymbol{s}, \boldsymbol{u})$, $SD_z(\boldsymbol{s}, \boldsymbol{u}, \boldsymbol{z})$ and $SD_z(\boldsymbol{s}, \boldsymbol{u}, \hat{\boldsymbol{z}})$ under the frozen protocol with paired bootstrap intervals. It does not pass. On extension and velocity neither the true latent nor the estimate beats the no-latent control with an interval excluding zero, on either pool. The estimate is not worse than the true latent, and the retained fraction of the true advantage is positive on both channels, so the failure is not that the AAM estimates badly; the AAM recovers the latent about as well as the latent can be used. The failure is upstream, and Table 5.3 located it: the advantage the estimator faithfully recovers is a pressure-channel advantage.

Taken together, the three subsections above describe a chain that is sound component by component and inconclusive end to end. The inverse model is robust across the plant family (Section 5.2.2); the latent is recoverable from causal history alone; the adapter uses it when it is trained in isolation. What is missing is a demonstrated path from a pressure-channel latent to the extension the controller integrates, through an actuator with enough headroom to act on it. Chapter 7 reports the deployment measurements in full, and Chapter 8 takes up what would have to change for the question to be answered rather than deferred.

5.8 Training and Model-Selection Protocol

All adaptation-chain models share one split. Runs are divided 82 / 20 / 16 into training, validation and reserved test, by complete run *and* by factor configuration, so no factor key appears on both sides of a boundary, and windows never cross run boundaries. The sixteen reserved runs entered no decision at any stage: they are recorded as unused in the adapter's manifest, and no history length, checkpoint or hyperparameter was selected on them. They are scored, once, after each model is fixed, so every reserved-test number in this chapter is reported next to its validation counterpart rather than instead of it (Section 4.7). Selection is by held-out criterion rather than training loss throughout: SD_z

Table 5.4: Provenance of each model in the adaptation chain. Frozen components are loaded unchanged and never receive gradients in this chapter.

Model	Status	Selected checkpoint
Control ID	frozen	paired-control, Section 5.2
Strict FD	frozen	strict-excitation, Section 5.3
Command ROM	frozen	PhysX-calibrated, Section 5.4.1
SD_0 (no latent)	scratch	epoch 58
SD_z + encoder	scratch	epoch 48, frozen after the gate
AAM	scratch	epoch 73, 1.0 s causal CNN
Latent adapter	scratch	epoch 116

and SD_0 on validation rollout, the AAM on downstream rollout quality rather than on latent error alone, and the adapter on validation command error.

One selection hazard is worth recording because it cost a false conclusion. The adapter of Section 5.7 minimises next-state error through the frozen forward model, whereas the stage-level promotion gate scores the predicted valve command. The two objectives disagree, and scoring an adapter on the gate's objective produced a uniformly negative verdict that says nothing about the adapter's own training goal. Each model is scored on the objective it was trained for, and Table 5.4 states for every component whether it is frozen or trained from scratch, since the comparison of Chapter 7 depends on that answer being unambiguous.

6. Control Framework and Simulation Integration

Chapter 5 left three actuator representations sitting on a bench, each with its own contract and its own offline evidence. This chapter puts a controller in front of them and turns them into three experiments.

Section 6.1 describes the classical controller, because it is where the competence comes from even though it never runs at deployment. Section 6.2 specifies the policy: what it sees, what it emits, what stops it from emitting something dangerous, and what it is paid for. Section 6.3 then explains how the first becomes the second, and reports a result about that transfer which changed the training protocol of this thesis. The individual stacks appear only after those three are fixed, because they differ in one place only.

6.1 The DLS Teacher and Its Two Roles

6.1.1 Scope and Role of the Controller

The classical controller occupies a narrower position in this architecture than its prominence suggests. There is no damped-least-squares layer underneath the policy at runtime. Nothing is added to the policy's output. If the policy emits zero, the arm does not move.

The DLS controller is nevertheless the reason the policies work at all, and it appears twice, both times at training or collection time and never at deployment (Figure 6.1). It is the *privileged teacher* whose behaviour the policies are distilled from (Section 6.3), and it is the closed-loop controller that drove the identification runs of Section 4.7. Calling it a nominal layer would misdescribe the deployed system and, worse, would make every reported result a property of a controller pair rather than of the policy.

6.1.2 Privileged Information

The teacher is *privileged* rather than better tuned: it reads two quantities that no deployable controller on this machine could obtain. The first is the exact terrain height directly under

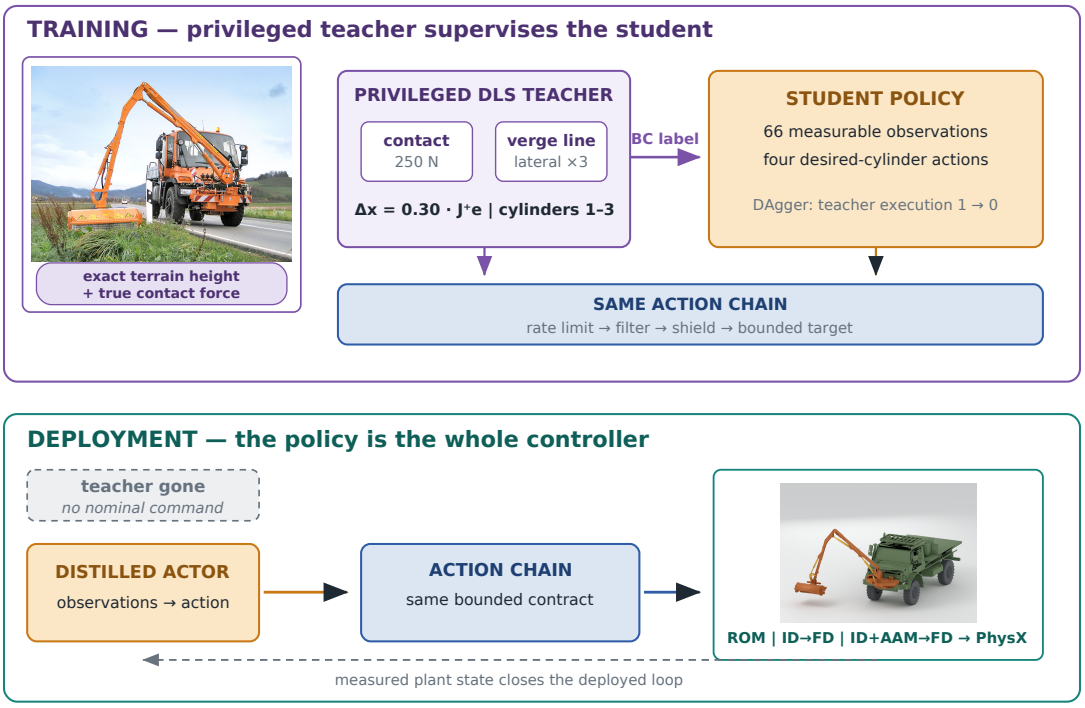

Figure 6.1: Where the classical controller lives. At training time the privileged teacher reads quantities the deployed system cannot (the exact terrain height under the cutter and the true contact force) and converts them into a desired cylinder target, which is then pushed through *the student's own* action chain and plant. At deployment the teacher is gone entirely: the policy is the whole controller, and no nominal command is summed with its output. The distilled actor is the only thing that crosses the boundary.

the cutter, queried from the simulator rather than inferred from a forward- looking scan. The second is the true cutter–ground contact force. Together these turn the hardest part of the task, knowing where the ground actually is, right now, underneath a 1.2 m flail head that is occluding it, into a lookup.

The teacher uses that privilege in a specific way. Rather than commanding a height, it runs a contact regulator: it maintains a vertical offset relative to the ground and adjusts that offset to hold the measured contact force near 250 N within a 100 N deadband, pressing down below the band, backing off proportionally above it, searching downward when contact is lost and retreating immediately on a force spike. The accumulated offset is clamped to $[-0.30, -0.05]$ m so the regulator can neither drive the head into the ground nor lift it clear.

The teacher's advantage is therefore not kinematic. Its inverse is the same DLS solution available to anything; what it has is contact sensing. That is the quantity the student must infer from a noisy forward scan and its own contact history, and it is a large part of why distillation is a nontrivial transfer rather than a formality.

6.1.3 From Task Error to Cylinder Command

The rest of the teacher is conventional and is stated for completeness. A goal point is maintained in the truck frame: laterally it tracks the verge centre line, vertically it follows

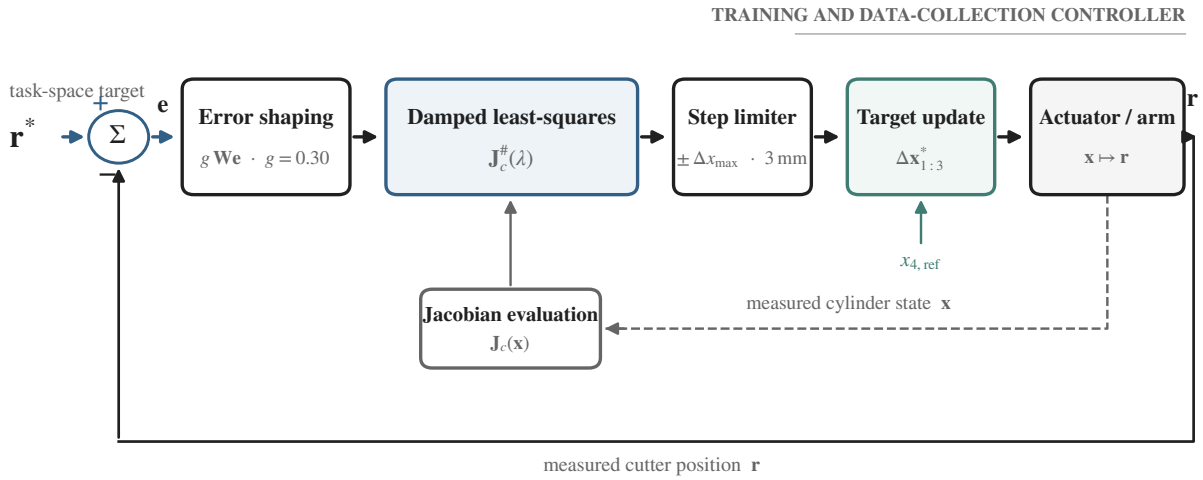

Figure 6.2: Feedback realization of the damped least-squares teacher of (6.1). The weighted task error is scaled by g, mapped to the first three cylinder coordinates by the measured damped inverse $\boldsymbol{J}_c^{+}$, and clipped to $\pm\Delta x_{\max}$ per tick. Cylinder 4 is held at its head-down reference. Cutter position closes the task-space loop and the measured cylinder state updates $\boldsymbol{J}_c$ at every step. The loop is used only for teaching and data collection, never at deployment.

the measured ground height plus the contact regulator's offset. The task error is the difference between that goal and the measured cutter position, with the lateral component weighted by a factor of three, because on this machine lateral error is what the acceptance gate measures and vertical error is largely handled by contact.

That error is mapped to cylinder space through the damped least-squares inverse of the measured cutter Jacobian $\boldsymbol{J}_c$ of Section 3.2.2,

$$\Delta \boldsymbol{x}^{*}_{1:3} = \operatorname{clip}\Big(g\, \boldsymbol{J}_c^{+}\, \boldsymbol{W} \boldsymbol{e},\ \pm \Delta x_{\max} \Big), \qquad g = 0.30, \qquad \Delta x_{\max} = 3\,\mathrm{mm}, \tag{6.1}$$

with $\boldsymbol{W}$ the lateral weighting and the per-step clamp bounding how far any cylinder may be asked to move in one tick. Only cylinders 1–3 are solved. Cylinder 4 is held at the reference extension that keeps the head down, for the reason given in Section 3.1. The gain g is small on purpose: this is a stiff, heavy mechanism in contact, and a resolved-rate controller that closes quickly on the task error simply excites it.

6.1.4 Execution Through the Student Interface

One implementation detail determines whether distillation converges to something usable. The teacher's output $\Delta \boldsymbol{x}^{*}$ is a desired cylinder target, and it is *not* written to PhysX directly. It is converted into the same normalized four-value action the student emits and pushed through the student's entire action chain and into the same plant (Section 6.2), so every label the student is asked to imitate is by construction an action the student can produce. A teacher writing targets directly to the plant would generate labels outside the student's reachable set, giving the imitation error a floor no amount of training could remove. That failure mode was measured here: when an earlier action-rate limit sat below the teacher's own structural bound, 0.43 % of labels became unreachable by construction and acted as exactly such a floor.

Table 6.1: The privileged DLS teacher, measured on the stage-6 verge task through the command-ROM plant (128 environments, 480 steps, all eleven path families, seed 17). It is competent, not perfect, and several acceptance thresholds sit deliberately just outside what it achieves.

Quantity	Teacher	Gate
Mean $\lvert e_{\text{cut}} \rvert$	0.0234 m	≤ 0.040 m
p95 $\lvert e_{\text{cut}} \rvert$	0.1524 m	—
Mean $\lvert e_{\text{lat}} \rvert$	0.1221 m	≤ 0.300 m
Ground-contact fraction	0.7807	≥ 0.700
p01 head alignment	0.9638	≥ 0.900
Minimum head alignment	0.8317	—
p95 tool acceleration	64.9 m/s^2	≤ 120 m/s^2

6.1.5 Teacher Performance

A teacher is not an oracle, and quoting its performance is necessary because distillation cannot exceed it and the gates of Section 3.9 were anchored to it. Table 6.1 reports the privileged teacher measured on the stage-6 verge task through the ROM plant, over 128 environments and 480 control steps.

Two readings follow. First, the teacher is a competent but visibly imperfect controller: it loses ground contact about a fifth of the time and its cut-height tail is six times its mean. Second, and this is why Section 3.9 re-derived several thresholds, a gate set above these numbers would be a gate that the ground-truth controller itself cannot pass, which carries no information about the policy.

6.2 Policy Formulation

6.2.1 Policy Definition

The deployed controller is a memoryless Gaussian multilayer perceptron. It maps a 66-value observation to four normalized actions, one desired motion command per hydraulic cylinder, and that is the entire control law. Actor and critic are separate networks with hidden widths 256–128–64 and ELU activations, over empirically normalized observations. AIF is the one exception: its actor reads 68 values, the same 66 plus the two channels of $\hat{\boldsymbol{z}}$, so the latent conditions the policy as well as the inverse model. That is an architectural difference between the stacks rather than a tuning difference, and Section 6.9 states how the comparison accounts for it.

Three things it is *not* must be stated, because each was a design option that was considered and rejected. It is not a residual: there is no nominal command to correct. It is not recurrent: temporal inference in this architecture belongs to the AAM of Chapter 5, and giving the policy its own memory would confound the two. And it does not command the vehicle: the truck's forward motion is scripted, so the policy cannot solve a tracking problem by driving differently.

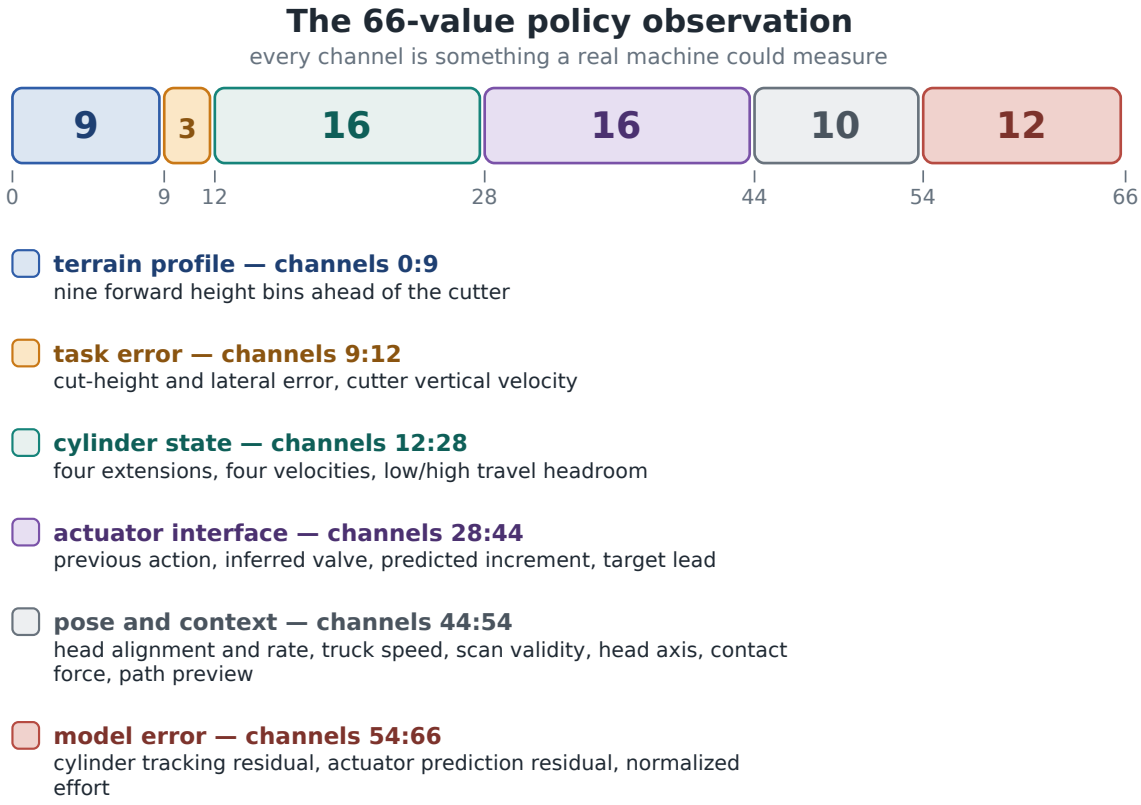

Figure 6.3: The 66-value policy observation, drawn to scale by group. Every channel is available or computable at runtime on the target machine: some are measured, some are derived from measurements, and the model-error group is estimated by the actuator models rather than sensed. No trajectory-family label, no physical factor vector and no true latent enters the actor. AIF appends the two estimated latent channels to this vector, giving 68, and is reported as the different architecture that it is.

6.2.2 The Observation Vector

Figure 6.3 lays out the 66 channels by group. The grouping tells the story of what this controller has to reason about: nine terrain-height samples ahead of the cutter, three task-error channels, sixteen channels of cylinder state and remaining travel, sixteen describing the actuator interface, ten of pose and context, and twelve of model error.

The last group deserves an argument, because twelve of sixty-six channels is a substantial fraction of the observation spent on something that is not the task. Those channels carry the actuator's standing error: the cylinder tracking residual, the actuator prediction residual, and the normalized cylinder effort. They exist as a direct consequence of the policy being memoryless. Forming an error requires differencing position across two steps and comparing against a prediction issued on the previous step, and a feedforward network can construct neither from a single observation, so without these channels the model error the policy is asked to compensate is not merely hard to see but absent from the input. This was measured rather than reasoned about: across ten training blocks the p01 head alignment sat at 0.79–0.80 and did not respond to changes in the feedforward source or

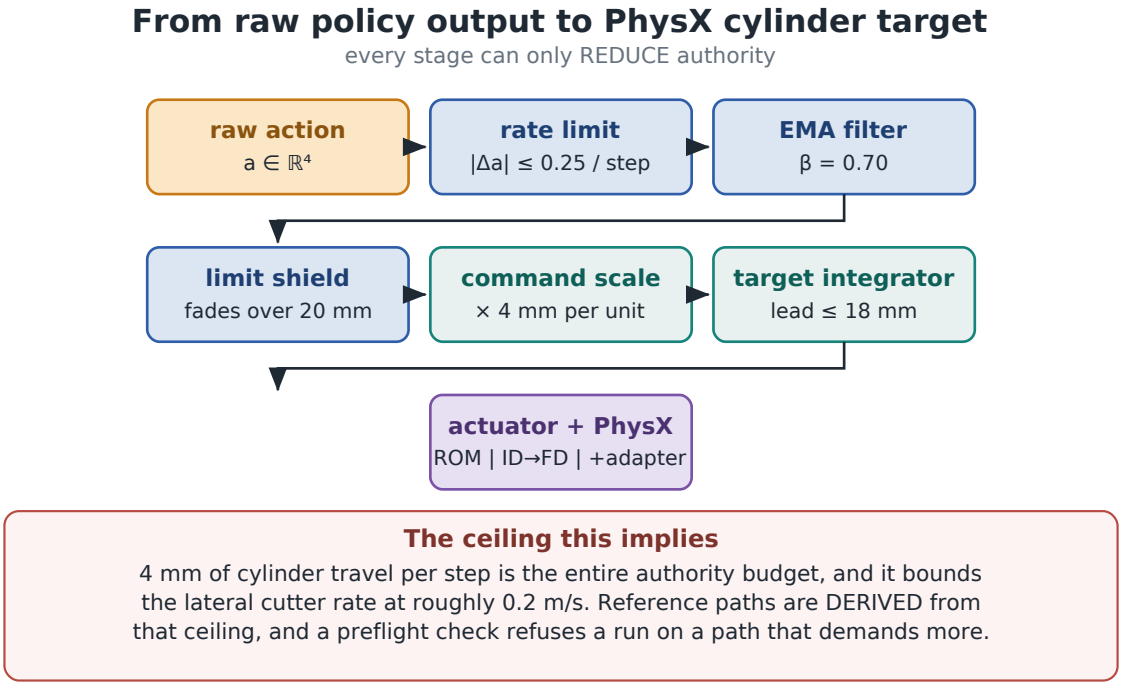

Figure 6.4: The action chain. With no nominal controller to explore around, the contract itself is the safety argument: a rate limit, an exponential filter, a limit shield that fades authority as travel headroom is consumed, a small per-step command scale, and an anti-windup cap on how far the integrated target may lead the measured extension.

the levelling gain, because the policy had no observable evidence of the error it was being asked to correct.

This group is also what makes the three stacks comparable. Under the ROM the tracking residual is near zero, because the ROM reaches what it is told. Under a learned ID–FD path it is the model's standing error. The same observation contract therefore gives each policy an honest view of its own actuator's shortcomings, which is what a fair comparison requires.

6.2.3 The Action and Its Bounds

The policy emits $\boldsymbol{a} \in \mathbb{R}^4$, sampled from its Gaussian at training time and taken at the mean for evaluation. Between that sample and the PhysX cylinder target sit five stages, shown in Figure 6.4. Every one of them can only *reduce* authority. None can introduce motion the policy did not request.

The rate limit bounds $|\Delta \boldsymbol{a}|$ at 0.25 per control step and is held *constant* across every curriculum stage. It is not a curriculum knob, and the reason is the reachability argument of Section 6.1: the teacher's own command stream has a structural per-step bound near 0.20, so a student limit below that makes part of the label set unreachable. The exponential filter ($\beta = 0.70$ at the final stage) smooths the command. The limit shield scales the command down, and only down, and only in the direction that consumes headroom, over the last 20 mm of cylinder travel, which is the soft replacement for the teacher's hard clamp into the learned range. The command scale converts the filtered action into 4 mm

of cylinder travel per unit action. Finally the target integrator accumulates the increments in reference space, with the integrated target permitted to lead the measured extension by at most 18 mm.

That last cap is an anti-windup device and it is load-bearing. Relaxing it from 18 mm to 50 mm halved the lateral error on the two families demanding a sustained standing offset (`lane_change_hold` from 0.4833 to 0.2310, `boom_sweep` from 0.3745 to 0.1602), showing that the clamp was bounding the standing effort those paths need, while the same relaxation on cylinder 4 drove head alignment from 0.9507 to 0.4636. The final task therefore resolves the trade per axis, and this is the configuration every reported R result was produced under: 50 mm on cylinders 1 to 3 and 20 mm on cylinder 4.

Two properties of this chain follow. It is what replaces the residual band, since exploration is bounded by how much motion a single action can command rather than by proximity to a baseline. And it fixes the plant's capability ceiling: four millimetres of cylinder travel per step is the entire authority budget, which produces the 0.240 m/s lateral rate limit of Section 3.6.2 from which the reference paths are derived.

6.2.4 The Reward Function

The reward of the final R task has ten bounded terms, listed in Table 6.2. Three are positive and describe the task. Seven are penalties and describe how the machine must behave while doing it. All are bounded, and all retain a live gradient beyond the acceptance boundary, so that a policy already inside a gate still has a direction to improve in.

It replaces an inherited formulation carrying 53 weighted reward fields. Isaac Lab's own locomotion task uses eleven. The reduction was not cosmetic. At 53 terms the optimum is not something a designer can reason about, and three separate terms turned out to be working against the task without being noticed for thirteen training campaigns. The three rules below were each adopted after a measured failure.

No Gaussian Shaping

The earlier reward priced tracking as $w \exp[-(e/s)^2]$, which is effectively flat beyond about $2s$. Measured on the lateral term, at an error of 0.50 m the gradient is -2.07 at $s = 0.30$ but -0.004 at $s = 0.15$: a factor of 520 weaker where the failing path families live. Every shaping term is therefore linear or Huber in the error. Writing $x = |e|/s$, the two shapes used are

$$B(e; s, c) = \min(x,\, c), \qquad H(e; s, c) = \min\Big(c,\, \begin{cases} \tfrac{1}{2}x^2, & x \le 1 \\ x - \tfrac{1}{2}, & x > 1 \end{cases}\Big). \tag{6.2}$$

B has the same gradient at 0.5 m of error as at 0.05 m, so a policy far from the reference is still told which way to move; H buys quadratic precision near zero where cut height needs it, without a vanishing far field.

Bounded Terms

A term that can grow without limit can dominate the sum and destabilize the critic. Each contribution is clipped, which gives the reward a published budget: $6 + 12 + 4 = 22$ for the task terms against -24 for the safety terms. A non-finite value is replaced by -25, just below that worst case, so a numerical fault can never outscore legitimate behaviour.

Table 6.2: The ten reward terms of the final R task, with weights and shaping scales. B and H are the bounded shapes of Equation (6.2). The lateral and head terms each combine a coarse and a fine scale, so far-field correction and near-field precision are priced separately.

Term	Weight	Shape and purpose
Cut-height tracking	+6	H, scale 0.04 m, cap 2.5; the primary mowing-quality term
Lateral tracking	+12	B, coarse 1.50 m and fine 0.60 m mixed 0.6/0.4; the primary line-following term
Ground contact	+4	pays for the cutter being in contact
Head alignment	−12	B on the deficit below 0.98, fine scale 0.02 and coarse scale 1.00 mixed evenly
Contact loss	−3	charges directly for loss of contact
Action rate	−1	B on the mean absolute command change, scale 0.25; penalizes chatter
Tool acceleration	−1	B, scale 300 m/s^2; mechanical-wear proxy on the cutter
Contact force	−1	B on force above 30 kN, scale 150 kN
Cylinder headroom	−2	B on encroachment inside a 0.02 m margin, scale 0.10 m
Hard impact	−4	indicator on the impact event

Fixed Sign Convention

The three task terms are non-negative and the seven safety terms non-positive. This is what makes worst-case (CVaR) weighting safe to add later: scaling a non-positive term by a factor ≥ 1 can only lower the return, whereas up-weighting a *positive* reward for the worst environments pays the policy to be bad. Measured in the earlier stack, weighting the positive contact reward four-fold for the worst quarter of environments made a contact quality of 0.49 pay 7.84 against 2.00 for a quality of 0.50, and collapsed worst-family contact within a single training block.

Three terms of the earlier formulation are absent, and the reasons are results rather than preferences. The *action-magnitude* penalty on mean(a^2) was removed outright: a terrain action is a desired cylinder motion rather than a pose offset, so what the term actually charged for was the sustained offset that `lane_change_hold`, `offset_return` and `boom_sweep` structurally require. The policy correctly learned to return to neutral, which is the one thing "hold an offset" forbids. Removing it produced the first training block in a full session that did not degrade, improving the held-out score from 1.2058 to 0.3678. The *head barrier* was replaced, because in its earlier form the head group dwarfed lateral tracking and line following could never compete for the optimizer's attention. And the *clearance barrier* was disabled, since measured contact, cylinder headroom and cut-height error proved more reliable signals of the same condition.

One apparent inconsistency is deliberate. The tool-acceleration term saturates at 300 m/s^2 while the corresponding gate sits at 120 m/s^2, anchored to the measured physical floor of the stage-6 environment near 100 m/s^2. The reward scale is a saturation scale for a

bounded penalty rather than a target, so the gate remains the binding criterion and the reward's role is only to keep a gradient against gross excursions. The action-rate term is held mild for a related reason: cylinder 4 owns head pitch, so a rate tax falls hardest on the fast corrections that keep the head down as boom attitude shifts, and under a heavier tax the optimizer chose head drift over paying it.

Every term is logged individually and by name during training. The earlier stack logged an aggregate and a partial subset, with the two largest lateral terms not logged at all, which is part of why a reward term opposing the task survived thirteen campaigns. Reward weights on this system are measured trades rather than preferences, and several natural-looking choices are actively harmful. That is also why the reward is *not* the result criterion: promotion uses the eight physical gates of Section 3.9, and a policy may raise total reward while failing the thing the reward was written to encourage.

6.2.5 Optimization

Policies are optimized with clipped-surrogate PPO over generalized advantage estimation, in the form of Section 2.7. The maintained configuration uses clip parameter $\epsilon = 0.2$, discount $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, five learning epochs over four minibatches per update, an adaptive learning-rate schedule targeting a KL divergence of 0.02 from a base rate of 3×10^{-4}, gradient-norm clipping at 1.0, and a clipped value loss with coefficient 1.0. The terrain task collects 32 steps per environment per update. At 512 environments that is 16 384 transitions per update, chosen because the inherited 128-step horizon exceeded two minutes per update on the available GPU without adding useful batch diversity.

The final fine-tuning blocks depart from that configuration because they run under the reference anchor of Section 6.4 rather than an adaptive learning-rate schedule: clip $\epsilon = 0.1$, discount $\gamma = 0.97$, three learning epochs, fixed learning rates of 10^{-5} for the actor and 10^{-4} for the critic, entropy coefficient 0.001, an exploration standard deviation fixed at 0.15 and excluded from the gradient, and frozen observation normalizers.

Three of those departures are structural rather than tuning. The discount is reduced because the episode is long in simulation steps but locally controlled, so an action should be credited for correcting the terrain state it faces rather than for a return hundreds of steps later that the critic cannot resolve. The actor learning rate sits an order of magnitude below the critic's so that the actor is deliberately the slow variable and trust-region budget is not spent on steps driven by a value function that has not yet converged. And the adaptive schedule is dropped because the anchored formulation already constrains the update through an explicit divergence bound; two competing step-size controllers would make neither attributable.

The exploration settings changed for a reason specific to this architecture. A residual policy explores around a controller that already tracks, whereas here a zero action means the arm does not move at all, so the initial standard deviation was raised from 0.05 to 0.30 and the entropy coefficient with it, to let the policy discover the command directions before refining them. On the terrain task, where distillation supplies a competent starting point, both are reduced again and the standard deviation is *held fixed* rather than initialized, for a further reason taken up in Section 6.4. Exact values for any reported run must be read from that run's frozen manifest, because they changed during development.

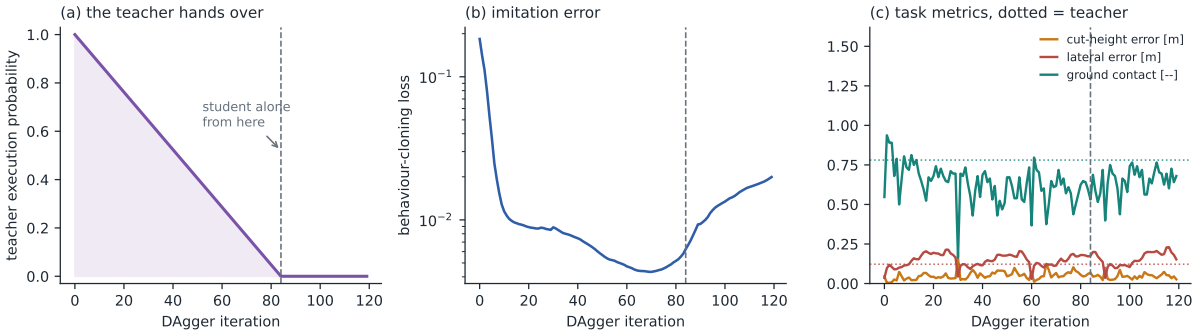

Figure 6.5: A measured distillation run for the R stack, from the stored record. (a) The teacher's execution probability is annealed to zero over the first 70 % of iterations. (b) Behaviour-cloning loss falls by over an order of magnitude under teacher execution, then rises once the student is alone, the distribution shift that DAgger is designed to surface, not a regression. (c) Closed-loop task metrics during distillation. Dotted lines are the teacher's own measured values from Table 6.1.

6.3 Training Protocol and the Distillation Finding

6.3.1 Distillation by DAgger

Policies are initialized by imitation, not by reward. The privileged teacher of Section 6.1 is rolled out in the student's own environment, and the student is fitted to the teacher's raw action by regression, in the DAgger scheme of Ross et al. [Ross 11]: the teacher's probability of *executing* the action, as opposed to merely labelling it, is annealed from one to zero over the first 70 % of iterations, so the later part of training collects states the student itself visits rather than states the teacher would have visited. Labels are accumulated in a replay buffer of 262 144 transitions and refitted over three epochs per iteration in minibatches of 8 192.

Figure 6.5 shows a complete distillation run for the ROM stack, taken from the stored record. Panel (a) is the handover schedule. Panel (b) is the imitation error: the behaviour-cloning loss falls by more than an order of magnitude while the teacher is still driving, reaches its minimum around the handover, and then *rises* once the student is alone. That rise is not a failure. It is the distribution shift DAgger exists to expose: the student is now visiting states the teacher never chose, and the teacher's advice on those states is new information rather than a repetition of what has already been fitted. Panel (c) confirms that the closed-loop task metrics stay in the neighbourhood of the teacher's own, without matching them.

6.3.2 Objective Mismatch in Unconstrained Fine-Tuning

The natural next step is to treat the distilled actor as an initialization and improve it with PPO. That is what the residual literature does, it is what the plan for this thesis originally assumed, and in unconstrained form, on this task, it does not work. The finding is reported here in full because it is what motivates the anchored formulation of Section 6.4, which is how the final policies are trained.

PPO fine-tuning was run repeatedly from distilled checkpoints, and the outcome was consistent enough to be treated as a property of the problem rather than as bad luck.

The critic was healthy throughout, with explained variance in the 0.85–0.95 band, so the failure is not a value-estimation failure. Total reward rose. And the primary tracking objective degraded, in the worst case by a factor of 12.9.

The mechanism follows from placing the reward and the gates side by side. The reward pays for ground contact and cut height as well as lateral tracking, and the optimizer discovers that it can buy the first two by spending the third. That trade increases total reward and moves the policy away from the gate that decides acceptance. The reward's optimum is simply not where the tracking gate is. Four earlier fine-tuning campaigns on a related configuration failed the same way in a different currency: every block degraded head alignment, with p01 falling from 0.9854 to the 0.48–0.78 range while safety terminations rose from 1.3 % to 3.6–7.7 %.

This is a claim about two functionals over the same policy space rather than about the optimizer. Training maximizes the expected return $J(\vartheta)$, while acceptance is decided by the vector of physical gates $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$ of Section 3.9. Writing $\mathcal{F} = \{\vartheta : \boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star\}$ for the feasible set, the measured content of the finding is

$$\arg\max_{\vartheta} J(\vartheta) \notin \mathcal{F}, \qquad \text{while} \qquad \vartheta_{\text{distilled}} \in \mathcal{F}. \tag{6.3}$$

Distillation lands inside $\mathcal{F}$ because the teacher was constructed to satisfy the gates, not to maximize J. Unconstrained ascent on J from that point therefore leaves $\mathcal{F}$ by construction. This is the optimizer working correctly on the wrong functional, and it is why the remedy is a constraint on the ascent rather than a repair of the reward.

Two consequences follow, and both are protocol rather than interpretation. Unconstrained fine-tuning is inadmissible, and both the pre- and post-fine-tune checkpoints are reported in every case; the matching rules that this imposes on the three-stack comparison are stated in Section 6.9. And reward decomposition, critic explained variance, policy entropy and the gate metrics themselves are logged *during* training rather than only at the end, because the whole point is that total reward went up while the system got worse. A training curve reporting only the scalar reward would have shown a success.

The general form of the result applies beyond this machine. When a competent teacher exists and the acceptance criterion is not the reward, reward-based fine-tuning is not automatically an improvement step: PPO optimizes the written objective, that objective is a proxy, and the teacher was constructed to do the task. Distillation transfers the intent; PPO then optimizes the proxy, and here the two point in measurably different directions. Since that divergence grows with distance travelled from a policy where proxy and intent were measured to agree, the remedy is not to abandon reward optimization but to bound the distance explicitly, which is the subject of the next section.

6.4 Reference-Anchored PPO

Equation (6.3) says that reward ascent leaves the feasible set. It does not say that reward ascent is useless, only that it is unbounded. The construction of this section keeps PPO and constrains how far it may travel from a policy already known to satisfy the gates. Writing π_{ref} for a frozen reference policy and J for the expected return, each training block solves

$$\max_{\vartheta} J(\vartheta) \quad \text{s.t.} \quad \mathbb{E}_{\boldsymbol{o}\sim d_{\pi_\vartheta}}\Big[\mathrm{KL}\Big(\pi_{\text{ref}}(\cdot \mid \boldsymbol{o}) \,\|\, \pi_\vartheta(\cdot \mid \boldsymbol{o})\Big)\Big] \le \delta_{\text{ref}}, \tag{6.4}$$

with the expectation taken over the states the policy itself visits.

6.4.1 Critic Warm-Up

A resumed checkpoint carries an actor ϑ and a critic ψ. After distillation the actor already behaves sensibly, but the critic has never been fitted to the long terrain return, so the first PPO updates would move a useful actor using a value estimate that is not yet meaningful. The actor is therefore frozen for 150 iterations while only the value loss is optimized, under the asserted invariant $\vartheta_{\text{after}} = \vartheta_{\text{before}}$. The observation normalizers are frozen at the same point: without that, the same raw observation is normalized differently from one iteration to the next, and the actor's behaviour changes while its weights do not.

6.4.2 The Reference Anchor

Let π_{ref} be a frozen copy of the accepted warm actor. It is queried only on states the live policy visits and never supplies an executed action, so PPO remains the sole controller of the plant. The penalty is the *forward* divergence, which for diagonal Gaussians is available in closed form,

$$D_{\text{ref}} = \mathrm{KL}\big(\pi_{\text{ref}} \,\|\, \pi_\vartheta\big) = \sum_i \left[\log \frac{\sigma_{\vartheta,i}}{\sigma_{\text{ref},i}} + \frac{\sigma_{\text{ref},i}^2 + (\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2\sigma_{\vartheta,i}^2} - \frac{1}{2}\right], \tag{6.5}$$

at the cost of one additional forward pass and no sampling. Because both scales are fixed at $\sigma = 0.15$, this collapses to a pure quadratic on the drift of the commanded action,

$$D_{\text{ref}} = \sum_i \frac{(\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2(0.15)^2}, \tag{6.6}$$

which is the second reason for fixing σ: with a free standard deviation, entropy collapse shrinks D_{ref} while the commanded action moves, so the constraint would report compliance during the drift it exists to prevent.

The direction is load-bearing. Forward KL is mass-covering: it charges π_ϑ wherever π_{ref} has density and π_ϑ does not, and therefore penalizes the *abandonment* of reference behaviour. The reverse direction is mode-seeking and would permit the policy to collapse onto a subset of reference behaviour at no cost, which is the observed failure, in which one competence was dropped to buy another. It follows that the term is a behaviour-cloning anchor evaluated on the policy's own state distribution, rather than the reverse-KL regularizer of soft reinforcement learning. Its lineage is demonstration-regularized policy gradient [Rajeswaran 18], not the Boltzmann-optimum family.

The actor objective is

$$\mathcal{L}(\vartheta) = \mathcal{L}_{\text{PPO}} + \mathcal{L}_V - \beta\,\mathcal{H}(\pi_\vartheta) + \lambda_{\text{ref}}\,\mathbb{E}_k\Big[\mathrm{KL}(\pi_{\text{ref}}\|\pi_\vartheta)\Big], \qquad \lambda_{\text{ref}} = 0.10. \tag{6.7}$$

6.4.3 Trust Regions and Rollback

A penalty is not a constraint: at a fixed coefficient, a sufficiently large advantage always buys its way past. Feasibility is therefore also verified on the update taken. Two bounds are imposed,

$$D_{\text{update}} = \mathrm{KL}(\pi_{\text{old}}\|\pi_\vartheta) \le 0.01, \qquad D_{\text{ref}} \le 0.05, \tag{6.8}$$

and they do different work. D_{update} is the classical step-size control that keeps the surrogate a valid local model of J. D_{ref} is the new object: because the anchor is *fixed* within a block rather than moving with the policy, it bounds cumulative drift. A sequence of updates each individually satisfying D_{update} can still walk arbitrarily far from feasibility, and that is the trajectory the earlier campaigns took, monotonically, with the reward rising the whole way. Only a constraint referred to a fixed feasible point can observe it.

Enforcement is in two stages. During minibatch optimization the epoch is early-stopped as soon as D_{update} exceeds its bound. After the update both divergences are recomputed over the full rollout. If either bound fails, or any quantity is non-finite, the actor and its optimizer state are restored *atomically*. On a divergence rejection the critic fit is retained when finite, since the bounds constrain the policy and not the value function, and rewinding valid critic fits computed on the same on-policy rollout was wasting most batches near the boundary. On a non-finite update both networks are restored. In the final block, five of one hundred logged updates were rejected, all of them on the reference bound and none on the update bound. That is, the drift caught was of the kind that ordinary PPO clipping does not see.

6.4.4 The Block Ratchet

The reference is not fixed forever. Within a block it is frozen. When a checkpoint is scored and passes the gates, it is promoted to reference,

$$\vartheta_{\text{ref}}^{(m+1)} = \vartheta^\star, \qquad \vartheta^\star \in \mathcal{F}, \quad \vartheta^\star \in B_{\delta_{\text{ref}}}\big(\vartheta_{\text{ref}}^{(m)}\big), \tag{6.9}$$

and the actor's optimizer moments are reset, because momentum accumulated relative to the old anchor is not meaningful relative to the new one. The critic's are retained: it is fitting the same return, and its calibration is expensive to rebuild.

This is the structural point of the construction. The policy is at all times within a bounded divergence of a checkpoint that has been verified against $\boldsymbol{G}$, and the anchor advances only through verified checkpoints. The acceptance criterion therefore enters the training loop instead of appearing only at the end. The optimizer still ascends the proxy. It simply can never travel far from the last point at which proxy and intent were measured to agree. In this construction distillation is no longer the endpoint: it is $\vartheta_{\text{ref}}^{(0)}$.

Algorithm 6.1 states the construction in full.

6.4.5 Theoretical Status

The components are standard and citable. The bound on D_{update} is conservative policy iteration and its trust-region descendants [Kakade 02, Schulman 15]; the atomic rollback is that literature's backtracking line search reduced to a single trial point; the penalty in Equation (6.7) is a Lagrangian relaxation of the constrained problem (6.4) evaluated at a fixed multiplier, with the rollback supplying the feasibility enforcement a fixed multiplier cannot.

What does not follow is a certificate. Bounding the divergence from a verified policy does bound gate degradation in form: the chain rule of KL over trajectories, Pinsker's inequality and the boundedness of every gate metric give $|\mathbb{E}_{\pi_\vartheta}[g] - \mathbb{E}_{\pi_{\text{ref}}}[g]| \le 2G_{\max}\sqrt{H\delta_{\text{ref}}/2}$ for a

Algorithm 6.1: Reference-anchored PPO for the R verge policy. The teacher appears only in layer 1 and the reference only as a divergence target: at every step of layer 3 the plant is driven by π_ϑ alone.

input : privileged teacher π_T; gates $\boldsymbol{G}$ with thresholds $\boldsymbol{g}^\star$; bounds $\delta_{\text{upd}} = 0.01$, $\delta_{\text{ref}} = 0.05$; anchor weight $\lambda_{\text{ref}} = 0.10$; fixed $\sigma = 0.15$; $\epsilon = 0.1$, $\gamma = 0.97$, $\lambda_{\text{GAE}} = 0.95$; $\eta_{\text{actor}} = 10^{-5}$, $\eta_{\text{critic}} = 10^{-4}$
output : accepted policy $\vartheta^\star$ with $\boldsymbol{G}(\vartheta^\star) \preceq \boldsymbol{g}^\star$

`// Layer 1: a physically feasible student`
$\vartheta \leftarrow \arg\min_\vartheta \mathbb{E}[\|\mu_\vartheta(\boldsymbol{o}) - \boldsymbol{y}^T\|_2^2]$ on DAgger states `// on the mean, not a sample`
discard π_T `// never in the loop again`

`// Layer 2: critic warm-up, actor frozen`
freeze ϑ and the observation normalizers
for 150 *iterations* **do**
  $\psi \leftarrow \psi - \eta_{\text{critic}} \nabla_\psi \mathcal{L}_V(\psi)$
end
assert $\vartheta_{\text{after}} = \vartheta_{\text{before}}$
$\vartheta_{\text{ref}} \leftarrow \vartheta$ `// block-0 anchor`

`// Layer 3: anchored PPO in blocks`
for *block* $m \leftarrow 0, 1, 2, \ldots$ **do**
  for *each iteration of block* m **do**
    collect an on-policy rollout with π_ϑ as sole controller
    compute GAE advantages from $\gamma, \lambda_{\text{GAE}}$; normalize
    snapshot $(\vartheta, \psi,$ optimizer$)$
    for 3 *epochs, each over the minibatches* **do**
      if $\text{KL}(\pi_{\text{old}} \| \pi_\vartheta) > \delta_{\text{upd}}$ **then break**
      $\mathcal{L} \leftarrow \mathcal{L}_{\text{PPO}} + \mathcal{L}_V - \beta\mathcal{H} + \lambda_{\text{ref}} \text{KL}(\pi_{\text{ref}} \| \pi_\vartheta)$
      step Adam on $\mathcal{L}$ with $\eta_{\text{actor}}, \eta_{\text{critic}}$
    end
    $D_{\text{upd}} \leftarrow \text{KL}(\pi_{\text{old}} \| \pi_\vartheta)$, $D_{\text{ref}} \leftarrow \text{KL}(\pi_{\text{ref}} \| \pi_\vartheta)$ `// over the whole rollout`
    if D_{upd} *or* D_{ref} *non-finite* **then**
      restore ϑ, ψ and the optimizer
    else if $D_{\text{upd}} > \delta_{\text{upd}}$ **or** $D_{\text{ref}} > \delta_{\text{ref}}$ **then**
      restore ϑ and the actor optimizer group; **retain** the critic fit `// the bounds constrain the policy, not` V
  end
  score the block's checkpoints on ≥ 3 held-out terrain meshes; aggregate $\boldsymbol{G}$ to the worst mesh, and to the worst path family for the tracking gates
  if *some candidate* $\vartheta^\star$ *has* $\boldsymbol{G}(\vartheta^\star) \preceq \boldsymbol{g}^\star$ **then**
    $\vartheta_{\text{ref}} \leftarrow \vartheta^\star$; reset the *actor* optimizer moments, retain the critic's
  else
    keep ϑ_{ref}; the block is not promoted
  end
end

metric g with $\|g\|_\infty \leq G_{\max}$ over a horizon H, a bound available only because the reward of Table 6.2 is bounded term by term and one that an exponential term would destroy. At the production horizon of 2808 steps and $\delta_{\text{ref}} = 0.05$, however, the right-hand side exceeds

eight, which for a metric bounded in $[0, 1]$ is vacuous. There is also a measure mismatch, since the constraint is taken under the live policy's state distribution while the chain rule requires the reference's.

The honest claim is therefore narrower. The construction is a principled restriction of reward ascent to a neighbourhood of a policy where proxy and acceptance criterion were measured to agree, assembled from standard trust-region and demonstration-regularized results, with feasibility re-established empirically at every block boundary. The gates, not the theory, certify each promotion.

6.5 R: Reference Actuator Baseline

R is the reference condition of this thesis. The policy emits desired cylinder motion, the frozen command ROM converts it to a cylinder position target, and PhysX closes the loop. There is no online adaptation anywhere in the stack. The truck is scripted over a 26 m strip at one constant speed drawn per episode from 2 to 7 km/h, the eleven path families of Section 3.6 are distributed across environments within a single run, and the terrain mesh is drawn per environment so that no policy can memorize one ground surface.

Its status is that of a reference condition rather than a competitor, as developed in Chapter 7: the command ROM is not a pure Simscape artefact but a Simscape-trained model with a bounded closed-loop calibration against IsaacLab and PhysX, and ROM-plus-PhysX *is* the deployment plant. R's actuator therefore coincides with the plant. Its tracking advantage over IF is consequently not evidence that a reduced order model approximates the actuator better than an identified ID/FD pair: one of the two defines the target.

The policy is produced by the three-layer construction of this chapter: distillation from the privileged DLS teacher to a feasible student, critic warm-up with the actor frozen, and reference-anchored PPO in blocks under Equations (6.7) and (6.8). The final block produced checkpoints 280 to 329. The last iterate was not taken, because PPO can raise mean return while damaging a single path family. Candidates were scored instead on held-out meshes against the eight gates, aggregated to the worst mesh and, for the tracking-critical gates, the worst path family.

The gate table for that selected checkpoint is deliberately not printed here, and the reason is a limitation rather than an omission. The numbers on record were measured on the superseded 16 m rollout, and the checkpoint's own selection record marks them provisional for exactly that reason; that checkpoint's own full-length six-mesh revalidation was never run. Quoting a certified gate result for R on the strength of a shorter rollout would be the kind of claim this thesis is written to avoid, so R is described here as a reference *condition* whose acceptance evidence is pending, and Chapter 7 reports the revalidated figures for all three stacks under one protocol.

Three separate records of R acceptance exist in the project, and because they disagree it is worth stating plainly which one this thesis relies on. The pipeline's own documentation reports all eight gates passing across the 2 to 7 km/h range on three meshes, which is a selection-time record for a different checkpoint than the one described above. An earlier six-mesh evaluation of the rebuilt pipeline reports seven gates passing and *ground contact failing*, at 0.6486 against the 0.700 threshold and a margin of -0.0514, which

was at the time the one genuine performance gap in the stack. Neither record certifies the checkpoint this chapter selects, and neither is the evidence used in Chapter 7: the closed-loop comparison there re-scores a single checkpoint per stack under one protocol, at one speed, over the full path, on the same meshes and the same eleven families. The earlier ground-contact failure is still the right expectation to carry into that chapter. It is measured on a worst-mesh, worst-family aggregation, which is the statistic that the pooled per-cell means of Table 7.8 do not report, and it identifies ground contact on the ROM plant as the gate most likely to bind, a prediction Chapter 7 is able to check rather than assume.

6.6 IF: Non-Adaptive Learned Baseline

IF subclasses the R task and changes exactly one thing. The policy still emits desired cylinder motion, but that command is now realised by the frozen inverse model of Section 5.2, which converts it to a valve command, followed by the frozen strict forward model of Section 5.3, which converts the valve command to a predicted extension increment. PhysX receives the same bounded cylinder target as before. Task, terrain families, meshes, observation contract, action chain, reward, gates and the PPO update are inherited unchanged, so the actuator representation is the only free variable between the two stacks.

The stack matters to the argument twice over. It is the non-adaptive learned baseline against which AIF must be measured, since the two differ only by the AAM and the adapter, and it is the honest control for R, whose actuator coincides with the deployment plant.

Two properties of the composed path shape its behaviour, and both were established offline. The valve command is accumulated rather than issued absolutely and is clipped to the per-axis limits $[0.95, 0.95, 0.80, 0.80]$, so the actuator path carries its own saturation boundary independently of the action chain. And only the predicted extension increment is integrated, since the forward model's velocity output failed validation on two axes (Section 5.3) and is carried as a diagnostic. This stack's standing model error is therefore visible to the policy through the twelve model-error channels of Section 6.2, which is what makes the shared observation contract fair across actuators rather than merely identical.

The promoted policy's run manifest records all eight gates passing across the 2 to 7 km/h speed range on three held-out meshes. That is the selection-time evidence on which the checkpoint was promoted, not a certified result: it was measured on three meshes rather than the six the final claim uses, and it is superseded by the single-protocol revalidation of Chapter 7, where the per-stack tracking numbers belong. One protocol point belongs here: the reported policy was obtained under the reference-anchored construction of Section 6.4 with a lower actor learning rate than R and a fixed exploration standard deviation of 0.05, which is read from the run manifest rather than assumed to match.

6.7 AIF: The Adaptive Stack

AIF adds online adaptation to the IF path and changes nothing else about the plant. At every model tick the AAM of Section 5.6 reads the observable history window and produces $\hat{\boldsymbol{z}}$, which enters the stack in two places, shown in Figure 6.6: it is appended to the

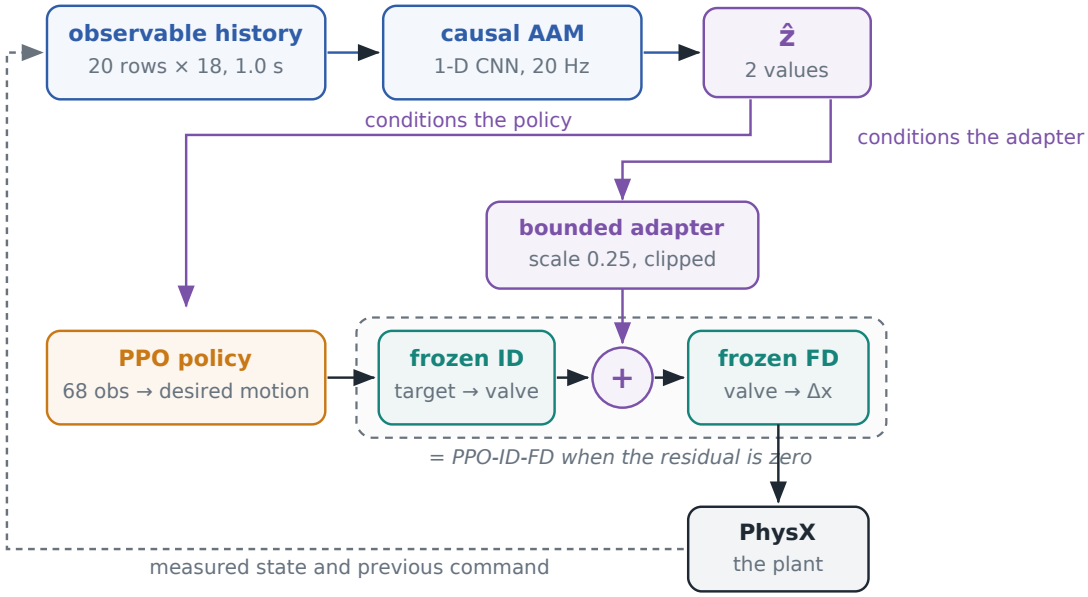

Figure 6.6: The AIF stack. The causal AAM reads only the observable history window and estimates $\hat{z}$, which conditions both the policy observation and the bounded adapter around the frozen inverse model. The forward model and the plant are unchanged from IF, and during warm-up or on an invalid estimate the correction is exactly zero, so the actuator path reverts to IF while the latent-conditioned actor stays in place.

policy observation, and it conditions the bounded adapter that corrects the frozen inverse model's valve command. The forward model, the base inverse model and the command ROM are untouched.

Conditioning the policy as well as the adapter is a deliberate design choice and it is the one place where this stack is not a drop-in substitution. The research question is whether an inferred description of the current plant improves closed-loop behaviour, and a latent that reached only the inverse model could change the realised valve command but never the intent behind it. The cost is that the actor is a 68-input network rather than a 66-input one, which is an architectural difference and is treated as one in Section 6.9.

The adapter itself is deliberately weak. It adds a residual to the base valve command, the residual is bounded by construction and further scaled by a declared factor of 0.25, and the result is clipped to the same per-axis valve limits as IF. Three fallbacks make the degradation path exact rather than approximate. Until twenty complete history rows have accumulated the adapter is bypassed and the frozen inverse model's command is returned unchanged; a non-finite latent triggers the same bypass; and the residual scale can be set to zero at evaluation time without touching any other part of the stack. In all three cases the *actuator path* is IF exactly, which is what makes the residual ablation a single-variable comparison. The actor is not restored by any of the three: it remains the 68-input latent-conditioned network. This stack therefore never reduces to IF as a whole, and the across-stack contrast in Chapter 7 is read with that in mind.

The estimate is not instantaneous, and the latency is part of the contract rather than an implementation detail. The AAM updates on the 20 Hz model-tick grid and its output

passes through a past-only rolling mean over five predictions, so a plant change reaches $\hat{z}$ no earlier than 250 ms after it becomes visible in the history window, on top of the 1.0 s window itself. Section 5.6.1 showed the deployed estimate to be a near-static seal-condition fingerprint, so this latency bounds a quantity that does not, on the evidence available, move much within an episode.

What this section does *not* claim is that the added machinery improves the task. The stack is complete, its interfaces are verified, and its manifest records the eight gates passing on six held-out meshes at selection time; whether the adaptation path contributes anything measurable is a separate question, settled in Chapter 7, and Section 5.7 has already reported that the closed-loop contrasts are null and why.

6.8 IsaacLab Integration and Timing

Four rates run concurrently and Table 3.4 lists them: PhysX integrates at 120 Hz, the policy acts at 60 Hz under a decimation of two, the learned actuator models advance on a 20 Hz grid of 0.05 s model ticks, and the AAM updates on that same grid. No single one of these is "the control frequency", and reporting a scalar rate for this system would misdescribe every one of the three stacks.

The relationship between the 60 Hz and 20 Hz layers is where integration errors hide, so it is fixed by construction rather than by interpolation. A learned actuator model is evaluated on every third control step and its predicted increment is held across the two intervening steps. The action presented to the model is the action of the tick on which it fires, not an average, and the target integrator accumulates in reference space so that a held prediction cannot re-anchor to a sagging measured position. Chapter 3 records why that reference-space integration is mandatory: integrating against the measured extension turns a static hold into a slow ratchet.

Order within a tick is fixed and asserted at run time. The AAM consumes the history window ending at the previous tick and produces $\hat{z}$ before the inverse model runs, so no quantity computed at the current tick can influence its own estimate. History buffers are per-environment and cleared on episode boundaries, which matters because the vectorised environments reset asynchronously and a shared buffer would leak one episode into the next. Both properties are checked rather than assumed: a runtime check replays a trace and asserts bit-exact agreement between the online estimate and an offline causal recomputation, and truncating the future of the input leaves the prefix of the output unchanged. Checkpoint loading then compares the stored normalisation statistics, feature contract and history length of every learned component against what the environment expects, and aborts on a mismatch rather than degrading silently.

6.9 Constraints and Fair-Comparison Rules

6.9.1 Limits and Safeguards

Authority is bounded at four independent places, and the redundancy is deliberate because each bound protects against a different failure. The action chain of Section 6.2 limits the rate of change of the command, fades authority as cylinder travel headroom is consumed, and caps how far the integrated target may lead the measured extension. The actuator

path clips the valve command to its per-axis limits. The environment terminates an episode on the safety conditions of Section 3.9 rather than letting it accumulate reward in a state an operator would stop. And every learned component falls back to a defined behaviour on a non-finite output.

Preflight runs before any training or evaluation and refuses to start on a violated precondition. The check that does the most work asserts that the commanded reference stays inside the measured actuator authority ceiling of Section 3.6.2: the cylinders move the cutter laterally at most 0.240 m/s, so a path family demanding more is infeasible by construction and no reward, critic or optimiser change could recover it. Terrain is derived from that ceiling, and the preflight assertion keeps the two from drifting apart.

6.9.2 Matching Rules for the Three-Stack Comparison

The comparison is only informative if the three stacks differ where the hypothesis says they differ and nowhere else. The following are held identical across R, IF and AIF: the verge task and its eleven path families, the terrain generator and the held-out mesh set, the reward function of Table 6.2, the action chain and its authority budget, the eight acceptance gates and their aggregation to the worst mesh and worst path family, episode length, actor and critic capacity, and the three-layer training construction of distillation, critic warm-up and reference-anchored PPO. This is the rule the comparison was designed to follow.

Four departures from that rule exist and are declared rather than smoothed over, because each of them weakens a comparison that would otherwise be clean.

The Reference Stack Contains the Plant

R's actuator is the command ROM, and the command ROM plus PhysX *is* the deployment plant (Section 5.4.1). Its tracking advantage over IF is therefore not evidence that a reduced-order model approximates the actuator better than an identified pair, because one of the two defines the target. R is a reference condition, not a competitor.

The Adaptive Stack Has a Larger Observation

AIF's actor reads 68 values against the others' 66. Any difference in its favour is therefore attributable to the latent's content, to the two extra inputs, or to the adapter, and these are separated by evaluation-time ablation rather than by assumption: the zeroed, shuffled, frozen and mean-latent arms of Section 5.7 all keep the architecture fixed and change only what occupies those channels.

The Training Runs Are Not Matched

The rule above governs the design; the runs that produced the three deployed checkpoints do not satisfy it, and the deviation is recorded in Section 7.1 from the run manifests rather than inferred. R and IF agree on every recorded training field, but the adaptive run differs in three: seed (41 against 17), actor learning rate (3×10^{-6} against 3×10^{-4}) and frozen-actor warm-up (0 against 150 iterations). The truck-speed schedule of the two reference runs is not recorded at all. Only one PPO training run exists per stack, so training variance is not estimated and the 33 evaluation cells measure scenario variation rather

than optimiser variability. The across-stack result is therefore conditional on the selected checkpoints, and causal attribution to the architecture comes from the within-checkpoint latent substitutions instead.

Fine-Tuning Budgets Must Be Matched or Reported

Either all three stacks are distilled with identical teacher data and budget and then evaluated, or all three receive the same bounded fine-tuning budget, with both the distilled and the fine-tuned checkpoints reported. Comparing a distilled baseline against a separately optimised adaptive policy and attributing the difference to the AAM is not permitted, for the reason established in Section 6.3.2: reward ascent moves a policy off the acceptance criterion, so an unmatched budget is itself a treatment.

7. Experimental Evaluation and Discussion

7.1 Hypotheses and Fairness Controls

The chapter tests four hypotheses. The first two are settled at model level in Sections 7.2–7.4; the last two require the control loop and are taken up from Section 7.7 onwards.

H1 The inverse model trained on randomised Simscape data is sufficiently robust to the gap between Simscape and the IsaacLab reduced-order hydraulic system that it can stay frozen. Support: a state predictor conditioned on an estimated latent, trained without the inverse model in the loop, closes the residual gap and beats a matched no-latent predictor on the same data.

H2 The value of the estimated latent lies in its variation over time, not in a static plant identity. Support: substituting a per-episode constant latent costs materially more than substituting a wrong constant.

H3 The closed-loop benefit of plant-identity conditioning is resolved from zero under a common evaluation protocol. This is a resolution question, not an equivalence claim: failing to resolve a difference is not evidence that none exists, and no practical-equivalence margin was predeclared, so the experiment cannot be read as one.

H4 That bound is obtainable before an estimator is built, from what a privileged latent read from the true factors achieves.

Each hypothesis is paired with its rejection criterion before the experiment runs: H1 fails without a majority of seeds, H2 if the constant-latent and wrong-latent substitutions cost the same, H3 if the paired bootstrap interval spans zero, and H4 if a privileged latent does not outperform an uninformative one. H3 is reported below as failed: what the experiment yields instead is the width of the interval, which is what a later equivalence test would need.

Fairness is enforced by construction rather than by argument, and the guarantee differs by experiment. All model-level and within-checkpoint ablations are *paired*: the two arms share the dataset, the split, the training objective, the model-selection criterion and the random seed, and differ in exactly one declared respect. The across-stack control comparison is matched on evaluation conditions but not on PPO training realisation, for the reasons set out above. Where a baseline could have been quoted from an earlier collection it is instead retrained on the current data, because a different collection carries a different state distribution and a different normaliser, and the resulting numbers are not comparable even when they measure the same quantity. Closed-loop comparisons additionally share trajectory family, truck speed schedule, terrain mesh, episode length, scored window and safety stops across all three stacks.

Several asymmetries survive that control and are stated rather than absorbed, read from the three run manifests rather than assumed. The R and IF checkpoints come from runs that agree with each other on every recorded field (seed 17, terrain seed 17, 256 environments, stage 6, actor learning rate 3×10^{-4}, entropy coefficient 0.001, frozen normaliser, and a 150-iteration frozen-actor warm-up) and differ only in the actuator. The adaptive stack is represented by a different run, for the reason given in Section 7.6, and that run differs from the matched pair in three recorded respects: a different seed (41, with terrain seed 41), an actor learning rate two orders of magnitude lower (3×10^{-6}), and no frozen-actor warm-up at all (0 against 150 iterations). Its observation is 68 columns against 66, which is the latent itself and is the intended difference rather than a confound.

All three manifests use the same schema and record the same eleven trajectory families. One field is missing where it would be most useful: the R and IF manifests do not record the truck-speed schedule at all, while the adaptive run records 2 to 7 km/h sampled per episode. The training speed schedule of the two reference stacks therefore cannot be verified from the manifests, and it is the reference pair rather than the adaptive run that is undocumented in this respect.

What is identical for all three is the evaluation: same meshes, same families, same speed, same window, same seed. The comparison is therefore fair as a statement about the three deployed controllers and not as a statement about three equally-tuned training procedures. The learning-rate and warm-up differences in particular mean that a deficit in the adaptive stack could be a deficit of its training run, and this is one reason the paired latent substitutions of Section 7.9, which hold the checkpoint fixed and change only the latent supplied to it, carry more weight here than the across-stack comparison does.

The shared estimand for every model-level experiment is the free-running accumulated normalised state error over a five-tick horizon, with no teacher forcing. The model integrates its own predicted increments and is charged for where it actually lands, which is the quantity a controller depends on and is the criterion the state predictor selects on. Reporting a one-step error instead would flatter every arm equally and hide exactly the compounding behaviour that distinguishes them.

7.2 Excitation Design and the Identifiability Envelope

What the data makes identifiable determines what any estimator can recover, so it is measured first. Excitation gathered under a task controller does not separate plants: with the command computed from the state, command and response are nearly redundant.

Table 7.1: What each excitation tier makes identifiable, and why the higher tiers do not. Decode R^2 is held out over 36 reserved runs; drift is the across-plant spread of cylinder extension at matched ticks, expressed as a fraction of the stroke the probe itself produces. Plant identity survives the first two tiers and is gone by the third, while command saturation never exceeds ten per cent.

Amplitude	seal R^2	mean R^2	drift / stroke	command sat.
0.05	0.703	0.109	0.18	0.11%
0.15	0.505	0.099	0.28	0.35%
0.35	−0.018	−0.040	0.39	3.26%
0.70	0.043	−0.021	0.50	9.98%

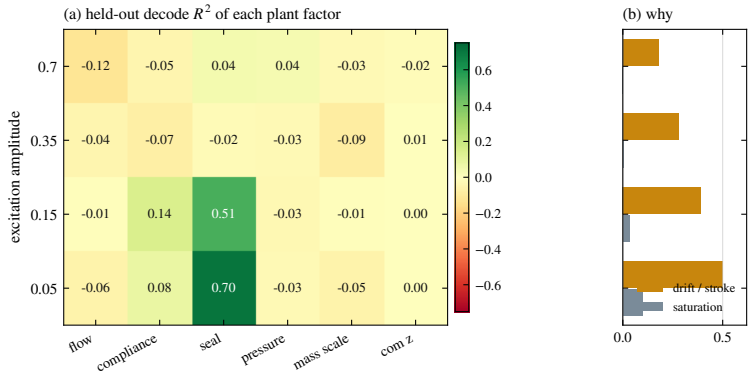

Figure 7.1: The identifiability envelope. (a) Held-out decode R^2 for each of the six plant factors at each excitation level. Only the seal condition is recoverable, and only in the two lowest tiers. (b) The reason: operating-point drift, as a fraction of the stroke the probe produces, rises steadily with amplitude while command saturation stays low throughout and cannot account for the collapse.

The reason is structural. A task controller computes its command from the measured state, so command and response are close to redundant and little is left for a plant descriptor to explain; the input must be rich enough for the regressor to stay well conditioned [Ljung 99], which makes excitation design a first-class decision [Schoukens 19]. The collection used throughout this chapter therefore removes the controller entirely and applies open-loop zero-mean square-wave excitation directly, at the cylinder-command interface because the command reduced-order model accepts cylinder extensions and not valve signals by design (Section 5.4). This collection, `rom_openloop_v1`, comprises 118 plant draws of 11 040 control steps each, split 82/20/16 by run into training, validation and reserved test sets, and its Hankel matrix reaches full rank 40 of 40 at a condition number of 17.8.

The excitation sweeps four amplitude tiers. Table 7.1 reports what each tier makes identifiable, decoding the six true plant factors from the per-tier mean latent estimate and scoring on held-out runs. The seal-condition factor is the positive control throughout, since it is the one factor the estimator is known to represent (Section 5.6.1); if it collapses, the decode is broken and no other entry in the row carries information.

The obvious explanation for the collapse is actuator saturation, and it is wrong. Saturation reaches only 9.98% of steps even at the largest amplitude, which cannot account for a

decode falling from $R^2 = 0.703$ to zero. The measurement that does account for it is operating-point drift. Every run executes an identical excitation schedule, so at a matched tick the only thing that differs between runs is the plant, and the across-run spread of cylinder extension is a direct reading of how far the plants have desynchronised in position. That spread grows from 1.5 mm at the start of the episode to 54.2 mm at the end, reaching half of the stroke the probe itself produces. Once runs are that far apart, a one-second observation window cannot separate a response that differs because the seal is leaking from one that differs because the cylinder is sitting five centimetres lower. The confound is an accumulated history effect, and it is not one the estimator can be blamed for.

Two consequences follow, and both constrain later sections. First, all adaptation experiments in this chapter are run below amplitude 0.35, inside the band where plant identity is present at all; an experiment run above it would be testing the estimator on data that contains no answer. Second, the excitation plan ramps amplitude monotonically across the episode, one tier per quarter, so amplitude and episode time are the same axis in this collection. No within-episode trend may be read from it. A pilot analysis that reported decode quality decaying over the episode was measuring the amplitude ramp, and is recorded here as an artefact rather than a finding. Interleaving the tiers and re-homing the cylinders between them would remove both problems and is the recommended design for any successor collection.

7.3 Sufficiency of the Frozen Inverse Model

The inverse model is trained on randomised Simscape data and deployed against the IsaacLab reduced-order plant, and the architecture of Chapter 6 depends on it staying frozen. H1 asks whether that is defensible. The direct measurement of inverse-model transfer error is reported in Section 5.2.2; what is tested here is the weaker but more consequential proposition that whatever residual gap remains is learnable *without touching the inverse model at all*.

The test removes the inverse model from the loop entirely. A state predictor consumes the current state, the applied command and an estimated latent inferred end to end from $[\mathbf{q}, \dot{\mathbf{q}}, \Delta p, \mathbf{u}]$, and predicts the next state. The baseline is the same predictor with no latent input, retrained on the identical data, split and objective, at five matched seeds. Figure 7.2 reports the pairs.

The estimated latent improves on the no-latent baseline in 5 of 5 seeds, with a mean paired change of -24.7% and a paired bootstrap 95% confidence interval of $[-34.2\%, -18.2\%]$ that excludes zero (Figure 7.3a). One qualification belongs with the number rather than in a footnote: seed 20260823 produced an unusually poor baseline at 0.03190 against roughly 0.024 elsewhere, and its -43.3% is as much a statement about that baseline as about the latent. The median change of -22.3% is the figure to quote, and the sign, reproduced in every seed of a paired design, is what supports H1.

The same comparison on the earlier closed-loop collection is what makes this result interpretable. There the ordering reversed: the no-latent predictor scored 0.17278, the end-to-end latent 0.17596 and the privileged latent 0.18515, so the latent appeared to be worthless or worse. The architecture was identical in both cases. What changed was the excitation, exactly as Section 7.2 predicts. Reporting only the closed-loop result would

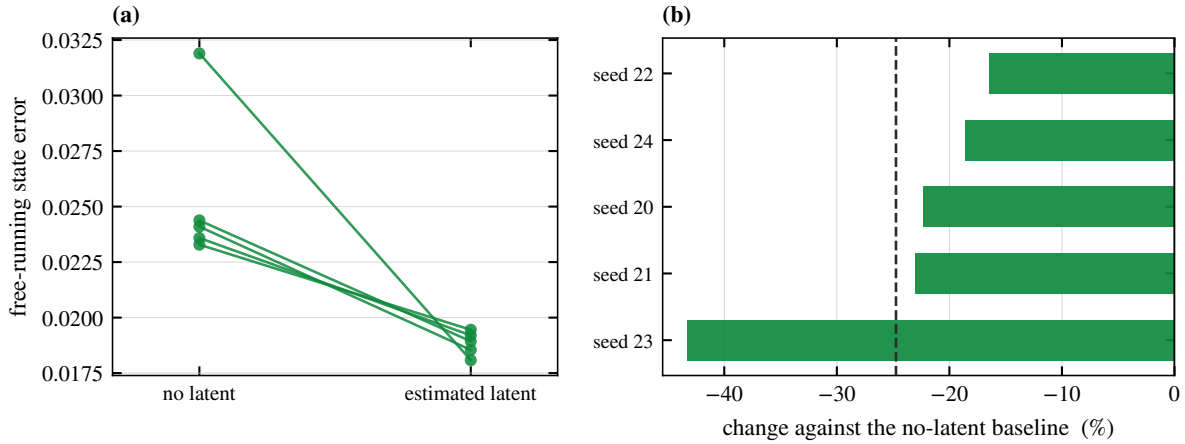

Figure 7.2: Latent conditioning against a matched no-latent baseline, five seeds. (a) Each line is one seed. No pair crosses, which is the result; the mean is not. (b) The same data as paired changes, with the mean dashed. Seed 23 carries the mean because its baseline is an outlier, so the median −22.3 % is the figure to quote.

Table 7.2: Sufficiency of the frozen inverse model, five matched seeds. The estimated latent lowers free-running state error in every seed. Seed 20260823 has an anomalously poor baseline and inflates the mean, so the median of −22.3% is the representative figure.

Seed	no latent	estimated latent	change
20260820	0.02438	0.01893	−22.3%
20260821	0.02410	0.01855	−23.0%
20260822	0.02328	0.01945	−16.5%
20260823	0.03190	0.01808	−43.3%
20260824	0.02359	0.01920	−18.6%
mean	0.02545	0.01884	−24.7%

have supported a confident negative conclusion about the architecture that the open-loop data contradicts.

H1 is therefore supported, with its scope stated plainly. The experiment does not measure inverse-model error, and it is not evidence that the inverse model is accurate in any absolute sense. It shows that the residual mismatch between the Simscape training distribution and the deployment plant is recoverable by a latent-conditioned state model, which is what licenses the simpler architecture in which the inverse model is never fine-tuned online.

7.4 Anatomy of the Adaptive Latent

That the latent helps does not establish *why*, and the two available explanations carry different consequences for the thesis. Under the first, the estimator recognises which plant it is operating on and the policy is conditioned on a plant identity. Under the second, the estimator tracks a time-varying operating condition and its value lies in the estimate moving. H2 separates them with a substitution experiment that changes nothing but the contents of the latent slot.

Table 7.3: Where the latent's value lies, five seeds, one trained model each, scored on the reserved validation runs. Freezing the estimate at its correct per-episode value costs 13.6%; making that frozen value wrong costs a further 0.43%. Both effects have the same sign in every seed.

Latent supplied to the state model	mean error	vs. online	seeds
online estimate $\hat{\mathbf{z}}(t)$	0.018842	—	—
per-episode constant, correct identity	0.021402	+13.6%	5/5
per-episode constant, wrong identity	0.021492	+14.1%	5/5

One trained model is scored three ways on the same data. In the *online* condition the latent is read from the observable history at every model tick, as in deployment. In the *constant* condition it is replaced by that run's own mean estimate, which preserves the correct plant identity exactly and destroys all variation. In the *shuffled* condition it is replaced by another run's mean, preserving the distribution of latent values while making the identity wrong. The gap between online and constant measures what the variation is worth; the gap between constant and shuffled measures what the identity is worth.

The decomposition is unambiguous and tight across seeds (Figure 7.3b). Removing the variation costs 13.6%, with a per-seed range of 11.9 to 14.3%. Removing the identity on top of that costs a further 0.43%, with a per-seed range of 0.32 to 0.84%. The second effect is consistently signed in all five seeds and so is real rather than noise, but it is roughly thirty times smaller than the first. Essentially all of the latent's value lies in the estimate moving, and almost none of it in the estimate being correct about which plant this is. H2 is supported.

This licenses one claim and forbids another. The claim it forbids is that the module identifies the plant it is operating on: at 0.43%, that reading is very nearly false, and the thesis does not make it. The claim it supports is that the module tracks a time-varying operating condition, and that the tracking is what earns the performance.

This is the ordinary form of the claim rather than a weak one. The module estimates from a fixed one-second causal window and carries no state between ticks, so it re-estimates rather than accumulates: recursive estimation with a forgetting factor, the standard construction when the estimated parameters are themselves time-varying [Åström 95, Ljung 99]. A converging estimator would lock onto a past operating condition and stop responding [Kumar 21].

Two boundaries are worth stating before the closed-loop chapters. The first is that this analysis concerns the jointly fine-tuned estimator trained on open-loop excitation, which is a different model on different data from the deployed estimator characterised in Section 5.6.1; the two disagree about stability, and the disagreement is informative rather than contradictory. The deployed estimator, trained under a task controller, is static within a run, whereas this one varies within an episode by roughly nine times the spread between plants. Excitation decides which of the two behaviours the estimator exhibits. The second boundary is that everything here is measured on a dynamics model rather than in the control loop; H4 repeats the substitution experiment on the closed-loop stack in Section 7.8, and only that result can carry the adaptation claim into the task.

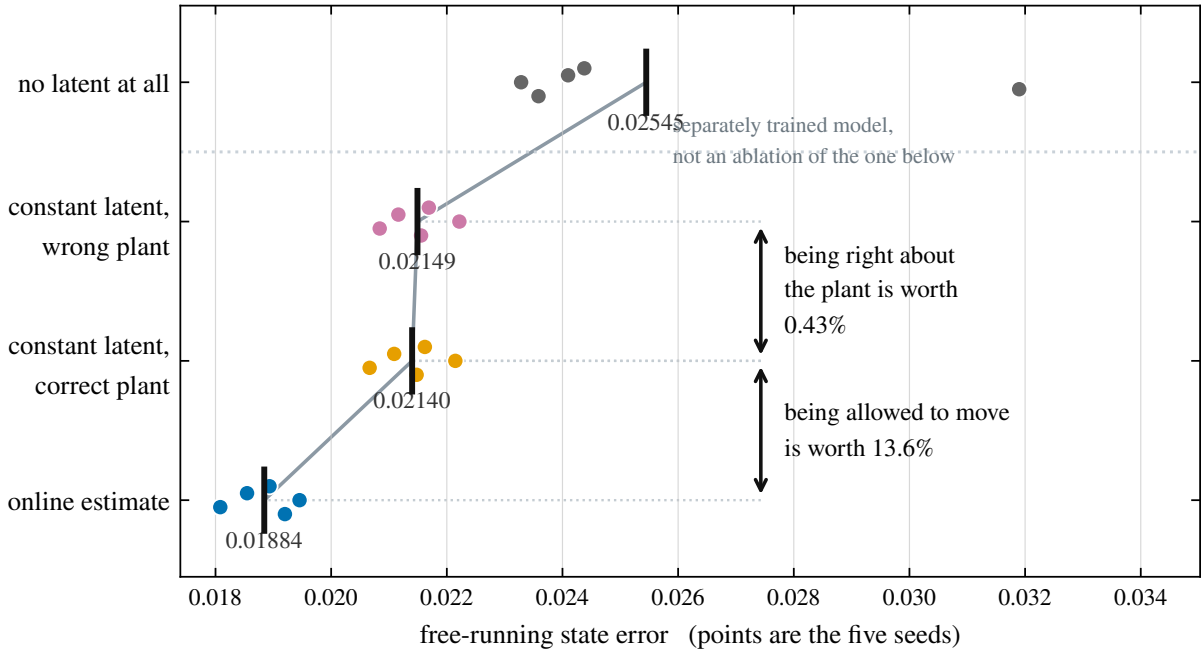

Figure 7.3: The model-level result as a single sequence, worst at the top. Each point is one of the five seeds and the heavy bar is the mean. The *gaps* are the claim: making the constant latent correct about the plant rather than wrong is worth 0.43%, while allowing that same estimate to vary in time is worth 13.6% – a factor of thirty-two. The topmost row is a separately trained no-latent model rather than an ablation of the three below it, so the step down to it is not attributable to the latent alone and is divided off.

7.5 Latency, from the Valve to the Tool

Latency enters this system in two quite different places, and separating them is the point of this section. The hydraulics impose a transport delay that no controller can remove; the control stack then adds its own, and only the second is an architectural choice. Section 7.5.1 measures the first on open-loop excitation, where it can be isolated cleanly. Section 7.5.2 asks whether the three stacks differ in how quickly they drive the plant. Section 7.5.3 asks the question that matters for the task – how long each takes to answer a change in what it is asked to do – and finds that the answer is dominated by the control side rather than the hydraulic one.

7.5.1 Dead Time in the Plant

Delay is measured here rather than assumed, and it is worth separating from an earlier negative result. Attempts to identify the valve's dynamic *model* from the available datasets did not succeed, but that is a parameter-fitting outcome and says nothing about whether the command-to-response *latency* is observable. It is, and the open-loop collection of Section 7.2 is close to an ideal instrument for it: the excitation is a train of zero-mean square waves with known edge times, which is what one would design for a step-response measurement.

The estimator locates each command edge and records the first sample at which the response leaves the pre-edge noise floor by four standard deviations. A cross-correlation

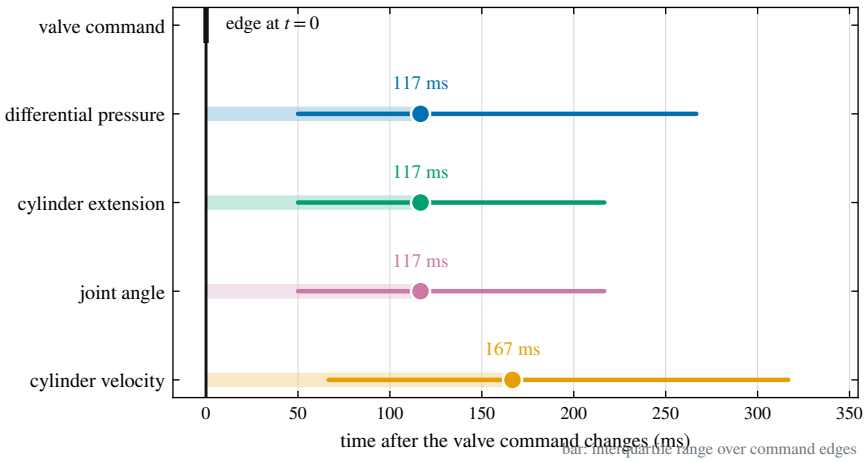

Figure 7.4: How a command propagates through the actuation chain, at excitation amplitude 0.15. The valve command changes at $t = 0$; each row below sits at the delay measured before that stage begins to move, with the shaded lead-in showing the wait and the bar giving the interquartile range over command edges. Pressure, cylinder extension and joint angle all begin at 117 ms and cylinder velocity at 167 ms. That extension and joint angle coincide is the control: the linkage between them is static and must contribute no lag. Each edge is aligned on the command's own onset rather than its steepest sample, since aligning on the slope places the origin after the cause has already acted and the response then appears to precede it.

peak was rejected because it conflates dead time with rise time. Only the pressure channel is drawn as a trace in Figure 7.4: cylinder and joint velocity are differentiated quantities whose edge-averages leave a transient at the command instant that reads as a response preceding its own cause.

The dominant term is the command-to-pressure lag of 117 ms, stable across three of the four amplitude tiers. The kinematic control is exact: the command-to-extension and command-to-joint onsets agree to within 0.0 control steps at every amplitude, so the linkage contributes nothing and the estimator is not manufacturing lag.

The medians do not compose – the first two links sum to 16 control steps while the direct measurement gives 7 to 11 – because the per-edge distributions are wide and the median of a sum is not the sum of medians. Each link is supported individually; the chain decomposition is not.

This matters for the architecture rather than merely describing the plant. A dead time of 117 ms is 2.3 model ticks at the 20 Hz estimator rate and roughly a tenth of the AAM's one-second observation window. A controller reading only the instantaneous state cannot compensate for it, because the information required is not present in a single slice; this is the standard argument for history-based estimation [Ljung 99], and it holds here on measured rather than assumed grounds.

7.5.2 The Chain Compared Across the Three Stacks

The measurement above characterises one plant on open-loop excitation. The three control stacks reach that plant through different actuator paths, and whether those paths differ in latency is a question about the architectures rather than about the hydraulics. It is answered by recording the same chain during a closed-loop drive on each stack.

Table 7.4: Transport latency along the actuation chain, median over 24 runs at 60 Hz control, in milliseconds. The last two rows are the control: the two onsets must coincide, and they do at every amplitude. Amplitude-invariance is what distinguishes a transport delay from a threshold effect.

	excitation amplitude			
Link	0.05	0.15	0.35	0.70
command $\rightarrow \Delta p$	117	117	117	150
$\Delta p \rightarrow \dot{\mathbf{s}}$	150	150	133	133
command $\rightarrow \dot{\mathbf{s}}$	183	167	133	117
command $\rightarrow \mathbf{s}$	117	117	100	100
command $\rightarrow \mathbf{q}$	117	117	100	100
linkage contribution	0	0	0	0

Table 7.5: Actuation-chain latency during a closed-loop drive, median cross-correlation lag over four cylinders and sixteen environments. The last row is the control: cylinder and joint are coupled by a static linkage, so it must read the resolution floor of one control step, and it does so on every stack.

Link	R	IF	AIF
command $\rightarrow$ pressure	450.0	166.7	158.3
pressure $\rightarrow$ cyl. velocity	0.0	166.7	58.3
command $\rightarrow$ cyl. velocity	283.3	283.3	**133.3**
cyl. $\rightarrow$ joint velocity	16.7	16.7	16.7

All entries in milliseconds.

One methodological difference must be stated before the numbers, because it prevents a false comparison. A closed-loop command varies continuously and has no step edges, so onset detection cannot be used and the estimator is instead a cross-correlation lag, which conflates dead time with rise time. The values in Table 7.5 are therefore valid for comparing stacks, where the bias is identical across the three, and are *not* comparable with the open-loop dead times of Table 7.4. The two tables do not measure the same quantity.

Three readings follow. The adaptive stack reaches cylinder motion in less than half the time of the other two, 133 ms against 283 ms, and the difference is in the hydraulic half of the chain rather than the mechanical one. R records no lag at all from pressure to cylinder velocity, which is a structural fact rather than a fast plant: the command reduced-order model predicts pressure and velocity together from one internal state, so there is no transport between them to measure, and its long apparent command-to-pressure lag of 450 ms should be read in the same light, since the valve signal on that path is produced inside the model rather than applied to it. The two stacks that drive the plant through the inverse and forward models, by contrast, show a genuine two-stage propagation with comparable lag in each stage.

The mechanical end is identical everywhere, at one control step, which is the resolution

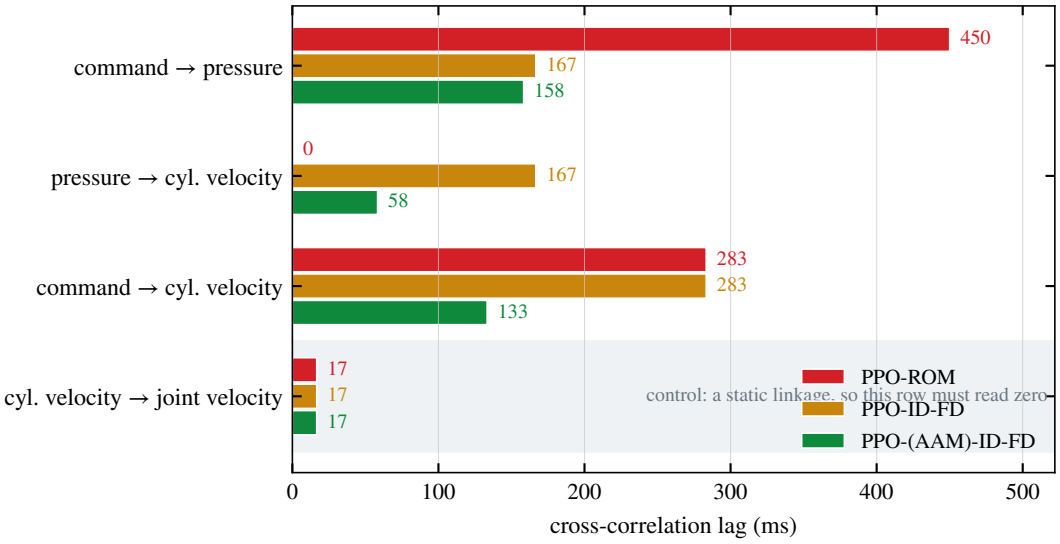

Figure 7.5: The same comparison drawn. The mechanical link at the bottom is the control and reads one control step on all three stacks, which is the resolution floor; the hydraulic links above it separate the architectures.

floor: the architectures differ in how they drive the hydraulics and not in how motion propagates through the linkage.

7.5.3 Step Response to a Commanded Change

The dead time measured so far belongs to the hydraulics and is the same for every stack. What differs is how each control stack copes with it, and that cannot be recovered from an ordinary drive: reference and achieved path are coherent only below about 0.3 Hz, where phase reflects the controller's own lag dynamics rather than transport, so no delay extracted there is a dead time (Section 7.14). The delay is therefore injected rather than inferred. A step of 100 mm is added to the *commanded* line partway through the scored window, and each stack is scored on the same mesh, seed and speed. The step is applied at the single point where the tracking line is computed, so it reaches the policy's own preview of the path as well as the error term: a step visible to the metric but not to the controller would measure only the metric's discontinuity.

The result is that **the hydraulics are not what makes these stacks slow**. Every stack begins to answer the step three to five times later than the 117 ms the plant needs, and none settles inside two seconds. Part of that is a hard bound rather than a control deficiency: at a command rate limit of 0.004 m per control step the fastest possible completion of a 100 mm step is 417 ms, which already exceeds the hydraulic dead time by a factor of three. The remainder is control-side, and settling times of 2.2 to 3.2 s are five to eight times that bound.

Between the stacks the ordering is modest and not uniform. IF settles soonest at 2.18 s, AIF is slowest to begin at 558 ms and slowest to settle at 3.23 s, and R overshoots least. The adaptive stack is therefore not faster to answer a change in the command, which is worth stating plainly alongside the finding of Section 7.5.2 that it reaches cylinder motion

Table 7.6: Response to a 100 mm step in the commanded line, median over sixteen environments. The rise time is 10 to 90% of the step and settling is the first time the error stays within 20 mm for half a second. The underlying trajectory is subtracted before measurement, so the only content left is the response to the step itself.

Stack	dead time	rise	settled	overshoot
R	333	617	2633	57%
IF	350	717	**2183**	73%
AIF	558	625	3233	78%
plant dead time, open loop	117	—	—	—
command rate limit	—	417	—	—

All times in milliseconds.

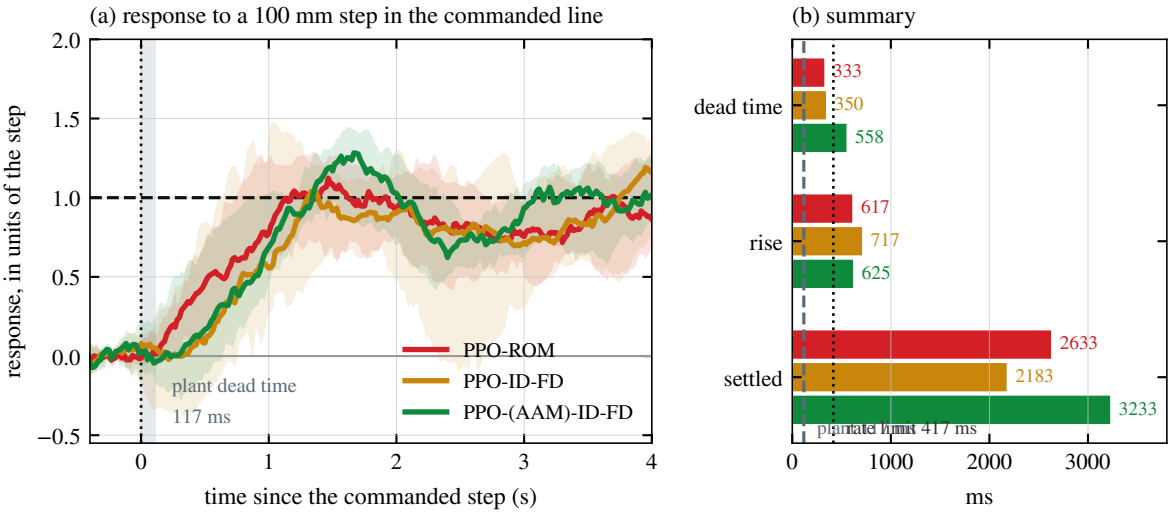

Figure 7.6: (a) Median response of each stack to the commanded step, in units of the step, with the interquartile band over environments; the shaded strip at the origin is the 117 ms the hydraulics alone require. (b) The same summarised, against two references: the plant's dead time and the 417 ms floor the command rate limit imposes on a 100 mm step.

in less than half the time of the others: a shorter path from command to actuator does not translate into a quicker response at the tool.

The dead-time column compares when a response becomes *detectable* rather than pure transport, because it is measured against each stack's own pre-step band and the stacks oscillate differently during normal tracking; the rise and settling columns carry the more trustworthy comparison.

7.5.4 The Latency Budget of the Closed Loop

The three results are easy to misread in isolation, and together they say something the chapter needs later. The plant costs 117 ms from valve command to differential pressure, and that figure is amplitude-invariant with an exact kinematic control, so it is a property of the hydraulics rather than of any controller. The stacks differ in how quickly they

Table 7.7: Candidate adaptive checkpoints re-scored over the full 26 m path, worst-family values on one held-out mesh. The incumbent is among the weakest of the seven. Only `headroom_priced_v1`, whose reward prices cylinder headroom explicitly, clears the ground-contact gate.

Run and iteration	contact	cut height	lateral
`headroom_priced_v1`/299	**0.799**	**0.017**	0.326
`contact_focus_scale025`/625	0.503	0.054	0.092
`boom_focus_scale025_v2`/610	0.403	0.060	0.089
`contact_focus_scale025`/600	0.393	0.061	0.073
`stage31_promotable_v2`/375	0.360	0.068	0.482
`train_final`/599	0.346	0.073	0.107
`speed_range_2_7_contact_safe_v2`/700	0.344	0.076	0.540
gate	≥ 0.700	≤ 0.040	≤ 0.300

traverse that chain, and the adaptive stack traverses it fastest, reaching cylinder motion in 133 ms against 283 ms for both others. Yet when the commanded line moves, the adaptive stack is the *slowest* of the three to respond and to settle.

A short path from command to actuator therefore does not produce a quick response at the tool, and the reason is that the dominant term is neither of the two. A 100 mm step cannot be completed faster than 417 ms at the command rate limit, which already exceeds the hydraulic dead time threefold, and the stacks settle in 2.2 to 3.2 s, five to eight times that bound. The actuation chain is not the bottleneck; the commanded rate and the policy's own dynamics are.

This bears directly on the adaptation question. An estimator that updates every 50 ms and a plant that answers in 117 ms are both fast relative to a loop that needs seconds to move the tool to a new line. Whatever the latent contributes, it contributes it to a controller whose response is set elsewhere, which is one candidate explanation for the gap between the clear model-level effect of Section 7.3 and the closely matched closed-loop arms of Sections 7.7 and 7.8. Section 7.13 returns to it.

7.6 Checkpoint Selection and the Scoring Horizon

The closed-loop results depend on which checkpoint represents each stack, and that choice turned out to be load-bearing in a way worth reporting rather than resolving silently.

Checkpoints are selected over the full scored path. The project's selection utility defaults to 480 control steps, which at the 4 km/h evaluation speed covers 8.9 of the 26 m course, so a candidate can satisfy it on the opening third alone. Every candidate was therefore re-scored over the complete path against the same eight-gate contract.

Two measurements fix what that selection is sensitive to. The adapter's residual scale is not the operative variable: setting it to zero moves ground contact by 0.001. And the adaptive stack does not reduce to IF when the residual is disabled, because its policy is separately trained and conditioned on the estimated latent. The two stacks differ in the policy as well as in the adapter, which is why each is selected on its own terms.

The adaptive stack is therefore represented in Section 7.7 by `headroom_priced_v1` at iteration 299, which passes seven of the eight gates on three meshes and fails only the lateral gate, at 0.326 against a threshold of 0.300. The name records the mechanism: headroom was priced into that run's reward. This is the same lesson the project has met before, that a quantity which is gated but not priced will be traded away, and it is reported here as a finding about the acceptance contract rather than buried in a checkpoint path.

Two consequences follow for the rest of the chapter. First, every closed-loop result names its checkpoint and its scoring horizon, because a gate that observes a third of an episode certifies a third of an episode. Second, the 480-step default was also used to select the R and IF checkpoints, so the same caveat is checked for all three rows rather than applied only where it was first noticed; those two stacks hold ground contact above 0.95 over the full path and are unaffected.

A stored evaluation of an earlier checkpoint, `stage31_idfd_aam_adapt_v1`/24, records all eight gates passing on six held-out meshes and appears to beat the checkpoint promoted here. It was scored under an easier protocol: 2 km/h rather than 4 km/h, and eight of the eleven trajectory families, omitting three of those that Figure 7.8 shows carrying the pooled means. A worst-family gate evaluated over a subset that excludes the hard families is a weaker test by construction.

The comparison is therefore not available. No checkpoint of any stack has been shown to pass all eight gates under the protocol used in this chapter, and a record obtained under an easier protocol is not evidence about the harder one. The 7/8 checkpoint scored correctly is reported here in preference to the 8/8 checkpoint scored under conditions the rest of the chapter does not use.

7.7 End-Effector and Mowing Performance

This is the chapter's only measurement of control, and hypothesis H3 stands or falls on it. All three stacks are scored on three held-out terrain meshes and eleven trajectory families under identical conditions: the same pinned truck speed of 4 km/h, the same 26 m scored window, the same 32 environments, distributed over eleven trajectory families, and the same evaluation seed. The unit of analysis is the (mesh, family) cell, which gives 33 paired conditions per stack; Section 7.12 explains why the evaluation seed is not a unit of replication here.

Figure 7.7 shows what the aggregate numbers summarise, on the three families the stacks track best. On these the commanded line is followed closely by all three, and the visible differences are oscillations about the reference rather than departures from it. Those oscillations are a property of the closed loop and not of the plot: they have a wavelength of one to two metres, which at 4 km/h is a period of one to two seconds, so they are slower than any filter that could be applied without removing the signal itself. Their amplitude is what the 26 to 28 mm mean absolute error records, and reducing it is a control problem rather than a presentation one. That is the honest picture of the typical case, and it is worth stating plainly because the aggregate means are not dominated by it: the suite also contains families such as the double S-curve and the Fourier meander, on which errors are an order of magnitude larger, and those cells carry the means reported below.

Table 7.8: Closed-loop performance over 33 paired (mesh, family) cells. Lateral and cut-height error are the mean over families rather than the worst family, which is a maximum dominated by a single hard trajectory. Impacts, head alignment and tool acceleration exist only per mesh and are therefore $n = 3$.

	R	IF	AIF
lateral error, mean (m)	0.1026	0.0703	**0.0692**
lateral error, median (m)	0.0392	**0.0281**	0.0342
lateral error, p95 (m)	0.4664	0.2665	**0.2155**
cut-height error, mean (m)	0.0084	**0.0063**	0.0097
cut-height error, median (m)	**0.0013**	0.0031	0.0020
ground contact	**0.9623**	0.9529	0.9490
hard impacts	0.0947	**0.0905**	0.1204
head alignment	0.7438	0.2247	**0.9763**
tool acceleration p95 (m/s^2)	39.90	**39.02**	51.45

The distinction matters for how Table 7.8 should be read. The stacks differ in the *tail* of the error distribution rather than in their typical behaviour, and the worst excursion over all cells is roughly 0.25 m for IF against about 1 m for the adaptive stack and 2 m for R.

The paired contrast closest to isolating adaptation is AIF against IF. The two share the frozen inverse and forward models and the evaluation protocol, but they are not a single-variable pair: the adaptive actor reads two extra observation channels and was trained from a different seed and learning-rate schedule (Section 6.9). What follows is a comparison of two deployed controllers, not of two training procedures.

On the three tracking metrics the two are separated by at most a few per cent and no difference resolves. Lateral error differs by -1.6% with the adaptive stack ahead in 9 of 33 cells, cut-height error by $+53.8\%$ ahead in 25 of 33, and ground contact by -0.4% ahead in 25 of 33. Every one of those three paired bootstrap intervals spans zero, so **H3 fails on its own criterion**: the sign of the effect is not resolved.

Failing to resolve a difference is not the same as showing there is none, and the useful quantity is therefore the interval width rather than the point estimate. The paired intervals are $[-0.044, +0.040]$ m on lateral error against a IF mean of 0.070 m, $[-0.0034, +0.0108]$ m on cut height against 0.0063 m, and $[-0.024, +0.015]$ on ground contact against 0.953. Read as a bound, only the last is informative: ground contact differs by less than about 2.4 percentage points either way. On the two tracking errors the interval is comparable to or wider than the baseline mean itself, so the experiment does not constrain the effect to anything smaller than the effect that was being looked for. Establishing equivalence would require a predeclared margin and a sample large enough to fit inside it; neither is available here.

Mean and median disagree on cut-height error and ground contact: the adaptive stack is ahead in roughly three quarters of the cells yet behind on the mean, the signature of a few poor cells rather than a uniform deficit. Both statistics therefore appear in Table 7.8.

Three differences stand out from the null result above, and two of them do resolve. IF scores 0.2247 on head alignment, including -0.855 on one mesh, the collapse under

unconstrained fine-tuning measured over the full path; the adaptive stack is the only one that holds alignment, at 0.9763; and it pays for its ground contact in impacts (+33%, interval [+0.013, +0.043]) and tool acceleration (+32%, interval [+2.8, +21.3] m/s^2). Those last two intervals exclude zero, and they are the only closed-loop contrasts in this chapter that do, both of them against the adaptive stack. They are also the weakest evidence in the table: impacts and tool acceleration aggregate per mesh rather than per cell, so the bootstrap resamples three units and the interval should be read as a sign indication rather than a calibrated bound.

Figure 7.9 places these numbers against the eight-gate contract and adds one result that the per-cell table cannot show. On worst-mesh, worst-family statistics the lateral gate is failed by every stack, and by R most severely. A threshold that no configuration satisfies is a statement about the hardest trajectory family in the suite, not a discriminator between architectures, and it should not be read as a defect of the adaptive stack alone.

That result has a consequence for the acceptance contract itself, and it should be stated rather than left for the reader to assemble. Under the protocol of this section (eleven families, 4 km/h, the full 26 m path) *no* stack satisfies all eight gates, because all three fail the lateral gate on the worst family. The contract as specified in Section 3.9.1 is therefore not demonstrated by this work for any architecture, and no claim of certified acceptance is made for any of them. Two readings of that are available and the evidence distinguishes them: either every stack is inadequate, or the lateral threshold is mis-set for the widened path library. The second is better supported. The threshold of 0.300 m was anchored before the trajectory families were widened, the two families that fail it are precisely those the amplitude widening made hardest, and the same stacks hold lateral error near 0.03 m on the other nine, an order of magnitude inside the gate. A gate failed by every candidate on two of eleven families is measuring those two families.

The eight-gate contract should accordingly be understood as the project's working acceptance instrument rather than as a validated specification, and two of its eight gates are now known not to discriminate. The lateral gate is failed by all three stacks and the hard-impact gate is passed by a do-nothing policy (Section 3.9.1). The remaining six do separate the stacks, head alignment most sharply, at 0.9763 for the adaptive stack against 0.2247 for IF, and it is on those that the comparative statements of this chapter rest.

7.7.1 Control Effort

Tracking error ranks outcomes but not costs, and two stacks with comparable error are not equivalent if one commands twice the actuation to get there. Three measures are taken from the same drives, chosen so that none restates another: the root-mean-square valve command, its rate of change, and the number of times per second the command reverses the direction of motion. The last is deliberately scale-free, because a stack can command little in total and still chatter, and chatter is what wears the machine.

The measure separates two failure modes that the error column alone conflates. R uses the *least* command of the three and tracks worst, which on its own would read as a conservative controller; its reversal rate of 40 per second, four times either other stack, says instead that its actuation is largely spent fighting itself. That also gives a mechanical account of its tool-acceleration and impact figures in Section 7.7, which otherwise appear as separate observations.

Table 7.9: Control effort and the tracking it buys, on held-out mesh 1337. Effort read without the error beside it would rank R best, which is the opposite of the truth.

Stack	RMS command	rate RMS	reversals/s	lateral error
R	0.586	7.55	40.0	120.1
IF	0.827	10.68	10.6	**26.8**
AIF	0.783	9.41	**9.5**	59.1

Lateral error in millimetres; command in normalised units.

The two stacks driving the plant through the inverse and forward models behave in the opposite way: they command more, reverse a quarter as often, and convert the additional effort into tracking. Between them the adaptive stack commands slightly less and reverses slightly less often than IF while tracking about twice as poorly, so on this evidence it is smoother rather than cheaper.

One limitation bounds what this table can be used for. It compares three distinct policies, so a difference in effort cannot be attributed to any single component of the architecture. In particular it does not test the explanation offered in Section 7.9, that the estimated latent disturbs the policy through its variation: that claim concerns different latents supplied to *one* policy, and the arms in Section 7.9 were scored without per-step traces. Measuring effort across those arms is the direct test and remains outstanding.

7.8 Response to a Mid-Episode Plant Change

This experiment measures the disturbance envelope of the deployed stack: how large a change in the plant it absorbs without loss of performance. The result is a specification, and it states when a fixed controller suffices.

Design

The plant's valve effectiveness is scaled by 0.6 at a known instant inside the scored window. All arms share the policy, the mesh, the seed and the schedule, and differ only in what occupies the latent slot.

The *online* arm is the deployed system and the *frozen* arm latches a pre-change value, so it is identical up to the latch and then stops tracking; beating it is the adaptation result. Three controls bound the reading: *shuffled* carries the right latent distribution and the wrong plant, which rules out any benefit from extra input dimensions; *mean* is in distribution and carries no plant information; and *sham* repeats the online arm with the schedule firing and the plant left alone. IF is scored under the same schedule as a non-adaptive reference.

A zeroed latent is deliberately not used as the no-information arm. Measured on a real trace both channels sit at means of -1.19 and -2.34 with no positive samples, so zero lies two to three standard deviations outside the distribution; it would be an out-of-distribution arm wearing the label of an ablation. The *mean* arm does that job instead.

Table 7.10: Response to a mid-episode plant change, over 33 paired (mesh, family) cells. Every paired bootstrap interval straddles zero and no sign count approaches unanimity.

Latent supplied	lateral (m)	cut height (m)	ground contact
online estimate	0.07964	0.01159	0.94012
frozen, pre-change	0.07867	0.01314	0.93742
shuffled	0.06822	0.00934	0.94655
mean	0.07595	0.01156	0.94361
online, plant unchanged	0.07974	0.01382	0.93505
IF reference	0.11537	0.00966	0.93865
online vs. frozen	+1.2%	−11.8%	+0.3%
better in	17/33	17/33	17/33

Result

The arms sit within a few per cent of one another on every metric, and the sham arm explains why: the perturbation they were meant to respond to left the task untouched.

Effect of the Perturbation

Three measurements settle it, and they must be read in order.

1. **The perturbation is real.** The recorded plant parameter falls from 0.893 to 0.536, a ratio of 0.600 exactly as configured, at 9.0 s inside a window ending at 23.4 s.

2. **The task does not notice it.** Over the post-change window the perturbed run records 95.3 mm of lateral error against 98.1 mm unperturbed: a difference of −2.9%, in the wrong direction, and well inside the spread between cells.

3. **The estimator does not detect it.** $\hat{\mathbf{z}}$ shifts by 0.496 across the step when the plant changes, and by 0.473 when it does not.

A 40% **reduction in valve effectiveness is absorbed.** Tracking, cut height and ground contact all hold through a change of that size. Within this envelope no adaptation is required, which is a statement about the deployed controller and not about the estimator.

Locating the Edge of the Envelope

A perturbation reveals the edge of the envelope only if it degrades performance. Transport delay does not qualify: the 117 ms dead time is present throughout training, so the policies already accommodate it. A step in the *commanded* line produces a measurable response on every stack (Section 7.5.3) and is the instrument for that measurement.

Table 7.11: Closed-loop performance under per-episode plant variation, over 33 paired (mesh, family) cells. The privileged arm reads the true factors, so it bounds what an estimator could contribute *through this conditioning architecture*.

Latent supplied	lateral (m)	cut height (m)	ground contact
privileged (oracle)	0.07204	0.01082	0.94150
mean, no plant info	0.07456	0.01133	0.94060
shuffled, wrong plant	0.07021	0.00878	0.94551
estimated (deployed)	0.09002	0.01579	0.93200
R reference	0.08121	0.01034	0.96138
IF reference	0.09754	0.00824	0.94183

7.9 An Oracle Reference for Plant-Identity Conditioning

One question decides whether building an estimator will pay: would perfect knowledge of the plant help this closed loop? It is answerable directly, by supplying the true factors through the privileged encoder, and the answer is obtained without training an estimator. The result is an oracle reference *within this encoder, adapter and policy*, not a universal bound on what plant knowledge could ever be worth: a different representation or a different downstream consumer could use the same information differently.

The plant is drawn per episode from the held-out factor set and is not perturbed mid-episode, so plant identity is a real and stable property of each run. Four latents are compared on the adaptive stack: the deployed estimate, a *privileged* latent read from the true factors by the frozen encoder, an in-distribution *mean* carrying no plant information, and a *shuffled* latent with the right distribution and the wrong plant. R and IF are scored under the same factor domain.

The absolute errors in Table 7.11 are larger than those in Table 7.8 for the same stacks, and the reason is the condition rather than the measurement: Section 7.7 scores the nominal plant, whereas every arm here faces a plant drawn from the held-out factor set. Held-out plants are harder, and the two tables should be read against their own rows rather than against each other.

The Oracle Result

Knowing the plant exactly is worth -3.4% in lateral error and -4.5% in cut height, and neither survives its sign test: the oracle beats the no-information arm in 18 and 19 of 33 cells respectively, which is indistinguishable from chance. A latent read directly from the true factors cannot outperform a latent carrying no plant information at all.

Within this conditioning architecture, estimator accuracy is not the limiting factor: a latent read from the true factors produces only a small improvement that the sample cannot resolve, so effort spent improving the AAM would not have been repaid here. Run before an estimator is trained, this measurement redirects that effort. Whether it generalises to other latent-conditioned designs is a hypothesis this experiment cannot test, because it varies the latent inside one fixed encoder, adapter and policy.

The Deployed Estimate Is the Worst of the Four

The deployed estimate does not merely fail to help; it is the worst of the four, by 20.7% on lateral error and 39.3% on cut height against the mean latent. A genuine but imperfect estimate performs worse than a constant carrying no information, and worse than one that is actively wrong about the plant.

The mechanism follows from Section 7.4, and it is the same property seen from the other side. The estimate varies within an episode by about nine times the spread between plants. The mean and shuffled arms supply a perfectly smooth signal; the deployed arm supplies one that moves continuously. The arms therefore differ in two respects, information and variability, and on this evidence the second dominates: the policy is better served by a constant it can ignore than by a signal that moves under it.

That is not a contradiction of the model-level result but its complement. In Section 7.3 the same time-variation was worth 13.6% to a one-step state predictor, which consumes the latent afresh at every tick and benefits from its responsiveness. The deployed actor is a memoryless MLP and does not integrate anything itself; what accumulates is the closed-loop trajectory, because a latent that moves under the actor changes its action from tick to tick and the plant carries the consequences forward. The same quantity is therefore plausibly informative to a one-step predictor and disruptive to a feedback loop, and the two roles were never separated in the architecture: one latent is routed to both. The arms differ in information and in variability at once, so this reading is the mechanism the evidence favours rather than one this experiment isolates. Section 7.10 separates the two.

Scope

Two limits belong with these numbers. The oracle arm necessarily differs from the others in the adapter as well as the latent, because an adapter must be trained for the latent it consumes and one trained on estimated latents refuses a privileged one; the -3.4% is therefore a comparison across two changes rather than one. And 33 cells resolve only large effects, so the flat ceiling bounds the available benefit rather than proving it exactly zero. What the experiment establishes is that the benefit is not large enough to find with the sample this chapter uses, under conditions chosen to favour it.

7.10 Latent Bandwidth on the Policy Path

The arms of Section 7.9 change what the latent knows and how fast it moves at the same time, so they cannot say which of the two costs the task anything. This section holds the content fixed and varies only the speed. A one-pole low-pass with time constant τ is applied to the latent on the policy path alone; the inverse-model adapter keeps the full-rate estimate in every arm. The checkpoint, the plant draw and the 33 cells are those of Section 7.9, and no network is retrained.

Figure 7.13 reports the sweep. All three metrics take their best value at the same interior τ, and both ends of the axis are worse, so the response has an optimum rather than a trend. Cut-height error falls monotonically across the five filter strengths below that optimum.

Table 7.12 gives the one contrast that resolves. Slowing the latent from $\tau = 0.032\,\mathrm{s}$ to $0.796\,\mathrm{s}$ improves cut-height tracking on the median as well as the mean. Lateral error moves the same way and does not resolve.

Table 7.12: The contrast that resolves. Both arms are filtered, so the comparison isolates bandwidth rather than filtering. Paired over 33 cells.

	mean	median	cells ahead	Wilcoxon p
cut-height error	−26.2%	−14.5%	22/33	0.015
lateral error	−5.7%	−5.5%	19/33	0.150

Three limits belong with that result. Against the *unfiltered* arm every filtered arm improves the mean by 11 to 46 % while the medians stay flat and no Wilcoxon test reaches $p < 0.10$; panel (c) shows why, since a minority of cells carry the mean. The mechanism is also not confirmed: the temporal variance of the latent as the actor reads it is 0.277 unfiltered and between 0.270 and 0.288 across every filtered arm, with no ordering in τ, so the latent's variation is mostly slower than the cutoffs applied and the proposed cause remains unmeasured. And the sweep varies one input to one checkpoint that was trained on the unfiltered signal, so the optimum may be a property of that training rather than of the architecture.

What survives is narrow and is stated as such: on this checkpoint, the speed at which the latent is allowed to move changes cut-height tracking, and the best setting is neither the raw estimate nor a constant.

7.11 Robustness to Plant Variation

The question here is where each stack stops working, which a pass or fail on a fixed factor set cannot answer. The magnitude of the deployment-factor randomisation is therefore swept: each environment keeps the draw it would have received and that draw is stretched away from the nominal plant along the same line, so scale 0 is the nominal machine, 1 reproduces the manifest and 2 doubles every deviation. Only the magnitude changes, never which plants are visited, so the curve is not confounded with a change of population.

No stack degrades with the size of the plant deviation. Mean lateral error over the sweep moves from 0.1286 to 0.1184 m for R, from 0.0277 to 0.0287 m for IF and from 0.0546 to 0.0673 m for the adaptive stack, and ground contact stays within 2.2% of nominal for R and 2.7% for the adaptive stack; IF moves furthest, by 5.5%. The experiment was designed to locate a breakdown point and finds none within twice the deployment range.

That is the same conclusion Section 7.8 reached by a different route. A 40% reduction in valve effectiveness did not move the task, and doubling the full six-factor deviation does not either. Taken together the two results bound the disturbance rather than the controllers: the plant variation this work can generate does not stress these stacks, and a robustness claim made against it would be a claim about a mild population of plants.

The one excursion is instructive precisely because it is not a trend. IF records 0.178 m at scale 1 against roughly 0.03 m everywhere else, including at the *more* extreme scales 1.5 and 2. Resolving it by family shows a single trajectory responsible: the double S-curve reaches 1.352 m, against 0.058 and 0.059 m at the neighbouring scales. One combination of plant and path destabilises that stack, and the combination does not survive being exaggerated. It is a genuine failure on a real manifest draw and it is not a breakdown

point, and the distinction matters: a curve summarised only by its worst value would report a breakdown at scale 1 that the neighbouring points contradict.

The sweep uses one mesh and eleven families per point, so it resolves a trend rather than a small difference between stacks, and its absolute values are not comparable with the three-mesh tables earlier in this chapter.

7.12 Statistics and Reporting

The protocol is declared once here and applied without exception. Primary metrics and seeds are fixed before each experiment is run. Every comparison is paired, and the sign count over the paired unit is reported alongside the mean, because a mean can be carried by a single outlying pair while a unanimous sign cannot. Uncertainty is quantified by a paired bootstrap rather than a parametric test, and intervals are reported as ranges rather than converted into a binary verdict. Model selection uses the validation split only, and reserved test runs are scored once.

The unit of replication differs between the model-level and closed-loop experiments, and conflating them would misstate the evidence. Model-level experiments replicate over training seeds, which change network initialisation and batch order. The closed-loop evaluation is deterministic: terrain is fixed by the mesh identifier, the policy acts deterministically at evaluation time and initial conditions do not vary. Five evaluation seeds were run and returned bit-identical numbers to nine decimal places, so the seed is *not* a unit of replication and quoting a sign count over seeds would report one observation five times. The closed-loop unit is therefore the (mesh, family) cell, and metrics that exist only per mesh are labelled $n = 3$ so that the weaker evidence remains visible.

Three reporting rules follow from failures encountered during this work. Numbers from different collections are never compared, even when they measure the same estimand, because the state distribution and the normaliser differ; a baseline is retrained instead. Every gate reports the measured floor of the quantity it constrains together with a positive control, because a threshold that no run could violate produces a passing verdict carrying no information. And every closed-loop result names its checkpoint and its scoring horizon, for the reason given in Section 7.6.

7.13 Integrated Discussion

This chapter establishes three things: the frozen inverse model is sufficient, the value of an inferred latent lies in its time-variation, and the closed-loop benefit of plant-identity conditioning on this task is bounded by what perfect plant knowledge achieves. The last is obtained before an estimator is built and is the more transferable result.

Supported Findings

The estimator works, and the excitation decides whether it can. On open-loop excitation an inferred latent lowers free-running prediction error by 24.7% in all five seeds; on data gathered under a task controller the same comparison puts it 1.8% behind, because a controller computes its command from the state and leaves command and response

nearly redundant (Sections 7.2 and 7.3). This licenses H1: the gap between the training distribution and the deployment plant is recoverable by a latent-conditioned state model without touching the inverse model, which is what the frozen-inverse architecture requires.

The Bound, and What Sets It

Three measurements delimit the closed-loop benefit, and they point the same way. Knowing the plant exactly is worth 3.4% in lateral error (Section 7.9), so estimator accuracy is not the constraint. The deployed controller already absorbs a 40% reduction in valve effectiveness and a doubling of every deployment factor (Sections 7.8 and 7.11), so plant variation of this size is not the constraint either. And the task response is set by the command rate limit rather than by the hydraulics (Section 7.5.3), which places it downstream of anything the latent influences.

A Single Mechanism

The four latents rank in an order that identifies the mechanism. The deployed estimate is behind both a constant carrying no information and one that is wrong about the plant, and it varies within an episode by about nine times the spread between plants (Section 7.4) while the constant arms are perfectly smooth. The arms differ in information and in variability, and the second dominates.

This complements the model-level result. A one-step predictor consumes the latent afresh each tick and is rewarded for its responsiveness, which is the 13.6% of Section 7.4. The deployed actor is a memoryless MLP and integrates nothing itself; what accumulates is the closed-loop trajectory, because a latent that moves under the actor changes its action from tick to tick and the plant carries the consequences forward. The same quantity is therefore plausibly informative to a predictor and disruptive to a feedback loop, and this architecture routes one latent to both roles. The design that follows is to hold or filter the latent on the policy path while leaving it responsive on the model path. It is specified here and not tested.

The Three Stacks Compared

Beneath the adaptation question, the three stacks differ in ways worth recording. IF tracks best and settles soonest, and fails the head-alignment gate badly, at 0.2247 with -0.855 on one mesh (Section 7.7). R commands the least actuation, tracks worst, and reverses direction four times as often as either other stack, so its effort is largely spent fighting itself (Section 7.7.1). The adaptive stack is the only one to hold head alignment, has the smoothest command of the three, and pays for its ground contact in impacts and tool acceleration. No stack dominates, and the lateral gate is failed by all three, which makes that threshold a statement about the hardest trajectory family rather than a discriminator between architectures.

Relation to the Research Gap

Chapter 2 positioned this work against history-based adaptation methods that infer a low-dimensional plant code and condition a policy on it. This chapter locates two conditions

those arguments leave implicit. A plant latent earns its place when it feeds a consumer that benefits from its responsiveness, and when the plant variation is large enough for identity to be worth having. Both are measurable before an estimator is trained: the identifiability envelope of Section 7.2 establishes the first and the oracle ceiling of Section 7.9 the second. Together they form a pre-deployment protocol, and that is the contribution of this chapter.

7.14 Threats to Validity

The Plant as a Learned Model

The deployment plant is the command reduced-order model driving PhysX, and R's actuator *is* that model. It is therefore a reference condition rather than a third competing approximator, and its advantages should not be read as an approximation result. The reduced-order model was calibrated against PhysX using parked shapes rather than the moving-verge task, so the calibration is weakest exactly where the cutter meets the ground, and the inverse and forward models received no comparable calibration.

An Unmeasured Input

The differential pressure the adaptive module takes as an input is produced by the actuator model and integrated at every tick, not sensed (Section 3.3.1). Within this simulation that is legitimate, because the model is the plant. On hardware it would require pressure transducers the FME 500 does not carry, and this is the channel where the module's measured advantage is concentrated. Any transfer of the model-level result therefore depends on instrumentation that is currently absent. No sensor-noise or quantisation model is applied anywhere in this work, so all observations are exact.

This leaves one question about the observation contract genuinely open, and it is carried here as a limitation rather than presented as settled. Every result in this work uses the pressure-inclusive observation row, in which measured differential pressure is one of the four signal groups the module consumes. A stricter alternative contract was specified alongside it, excluding differential pressure precisely because a deployed machine cannot supply it, and the choice between the two was never resolved. The consequence is bounded but real: the results reported here describe the pressure-inclusive module, and they do not establish what the deployable no-pressure variant would achieve. Given that the module's model-level advantage is concentrated in exactly the channel the stricter contract removes, the honest expectation is that the deployable variant performs worse, and quantifying by how much is unfinished work rather than a detail of implementation.

Resolution of the Evidence

The closed-loop comparisons use 33 (mesh, family) cells and bound differences between architectures at a few per cent; they do not establish equivalence. The evaluation is deterministic, so the replication unit is the cell and not the seed, and a sign count over evaluation seeds would report one observation several times.

Two experiments bound their disturbance rather than the architecture. The mid-episode plant change left the task unmoved, and the robustness sweep found no breakdown within

twice the deployment range. Neither licenses a robustness claim: they establish that the plant variation this work can generate is mild, and a stack that survives it has not been shown to survive more. The sweep also stretches the recorded factor draws rather than sampling new ones, so it says nothing about combinations the manifest never visited.

Conditions Not Identical Across Stacks

The three stacks are matched at evaluation but not in training. R and IF come from runs whose manifests agree on seed, stage, terrain seed, trajectory shape, speed and warm-up; the adaptive stack is a different run at an earlier iteration, chosen for the reason in Section 7.6, and its manifest records neither speed nor trajectory shape. The oracle arm of Section 7.9 additionally uses a different adapter checkpoint, because an adapter must be trained for the latent it consumes, so that contrast spans two changes rather than one.

Quantities Not Recoverable from Operational Data

Closed-loop delay compensation is not measurable from a normal drive: reference and achieved path are coherent only below 0.3 Hz, where phase reflects controller dynamics rather than transport, so Section 7.5.3 injects a step instead. The valve's dynamic model is likewise not identifiable in either dataset, although its transport latency is.

Coverage

The factor design is finite and sparse: 118 runs with 16 reserved configurations, six factors, and an operating envelope that does not cover the full machine. Excitation amplitude is confounded with episode time in the open-loop collection, so no within-episode trend may be read from it. All results are simulation-to-simulation; the Simscape-to-IsaacLab abstraction gap is characterised in Chapter 5 but the simulation-to-reality gap is not addressed anywhere in this work.

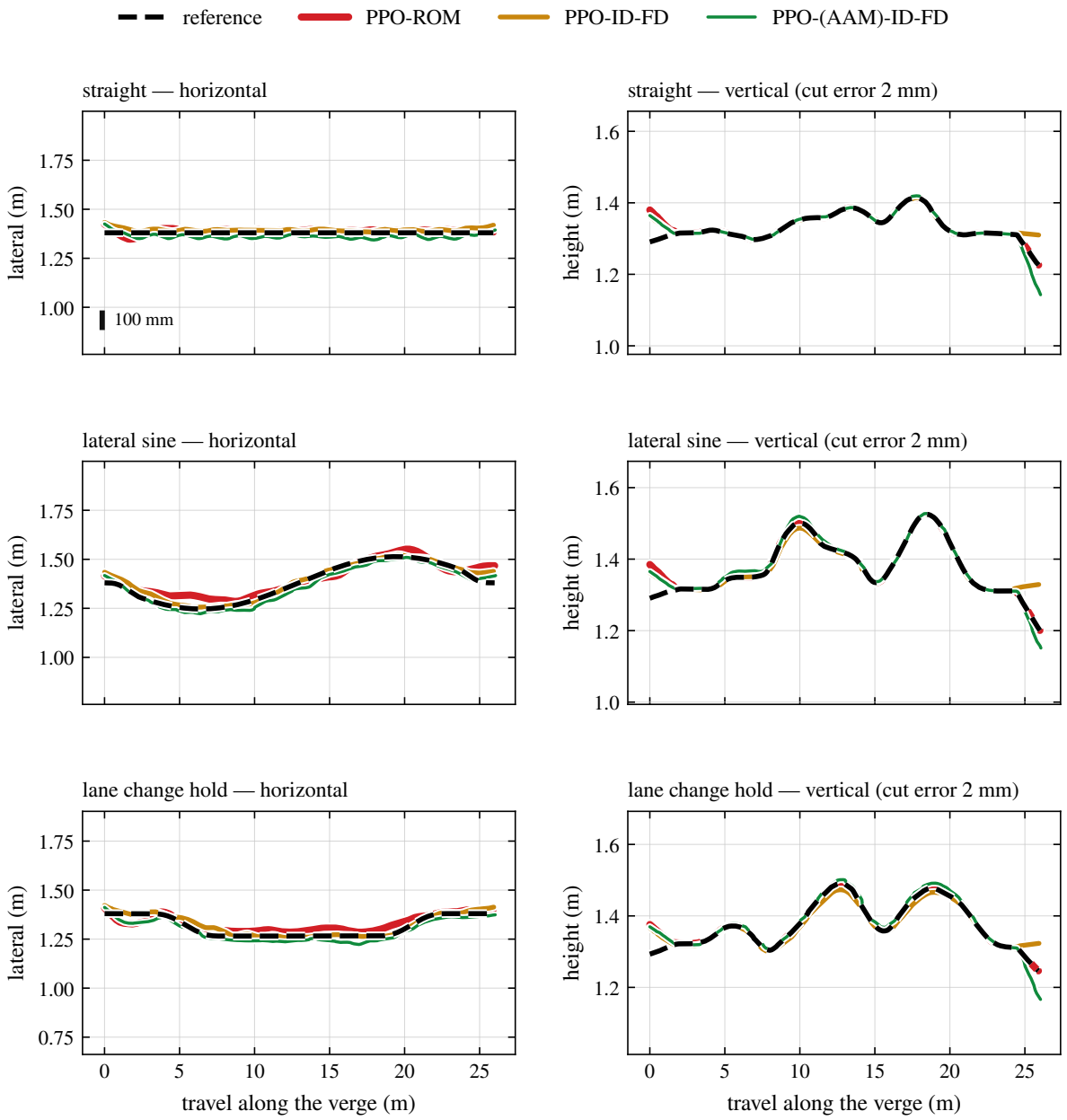

Figure 7.7: Cutter path against the reference on held-out mesh 1337, for the three well-tracked trajectory families that are also structurally different from one another: a flat line, a smooth periodic sweep and a step-and-hold, at 26, 27 and 29 mm mean absolute lateral error averaged over the three stacks. The Gaussian slalom scores marginally better than the lane change but is omitted because its reference is visually a sine and its panel would merely repeat the one above it; the lane change additionally exposes a transient that neither periodic family contains. The left column is the horizontal plane and the right column the vertical plane over the same drive. The vertical reference is not recorded directly and is recovered as $z_{\mathrm{ref}} = z_{\mathrm{cutter}} - e_{\mathrm{cut}}$, which makes both columns like-for-like reference-versus-actual comparisons. In the vertical panels the reference is largely hidden beneath the three traces, which is what a 2 mm mean cut-height error looks like when drawn at scale: the cutter follows the terrain undulation closely on every family, and the stacks separate only in the final metre. The environment-to-family assignment is deterministic and was verified by re-aggregating per-environment errors into per-family means and checking them against the run's own gate report.

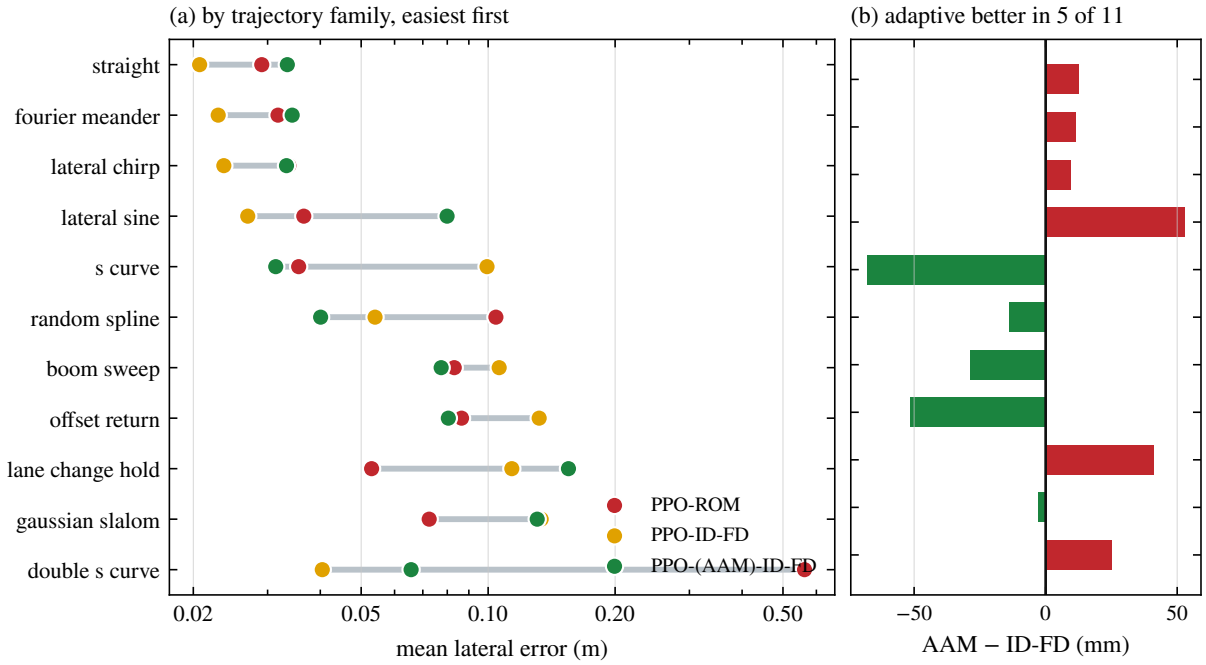

Figure 7.8: Lateral error by trajectory family, which is what a pooled distribution hides. (a) Each row is one family, ordered easiest first, with the bar spanning the three stacks; the suite covers more than an order of magnitude, and the two hardest families are what carry the pooled means in Table 7.8. (b) The paired contrast that isolates adaptation, family by family. No stack dominates: the adaptive stack is better on five of eleven families and worse on six, and the differences are tens of millimetres in both directions.

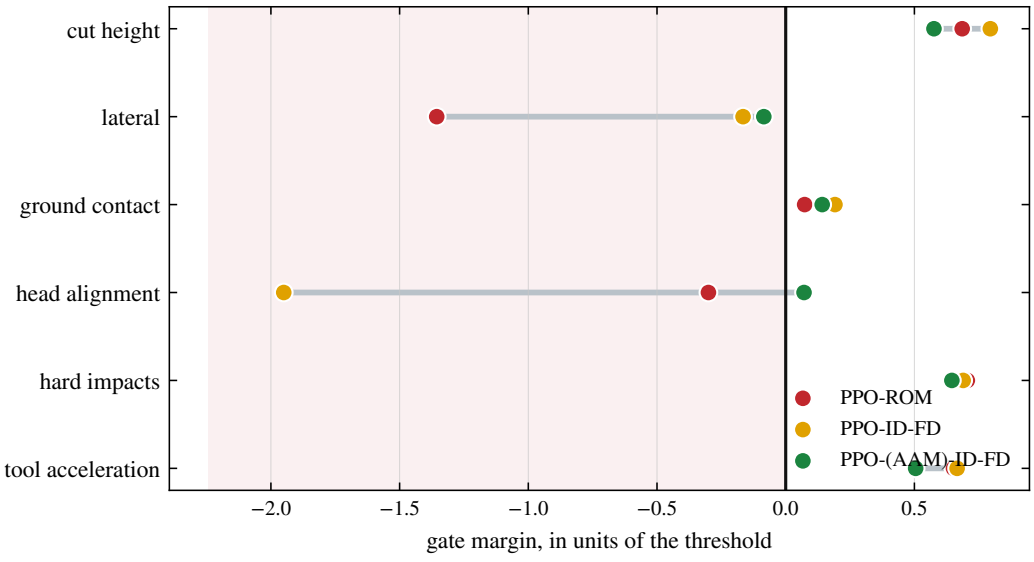

Figure 7.9: Gate margins on worst-mesh statistics, in units of each gate's own threshold, so the six are directly comparable; the shaded half-plane is failure. Two results stand out. The lateral gate is failed by *all three* stacks and most severely by R, so it reflects the difficulty of the worst trajectory family rather than a deficiency peculiar to the adaptive stack. And head alignment is failed badly by IF and passed only by AIF, which is the fine-tuning collapse of Section 7.7 seen against the acceptance contract.

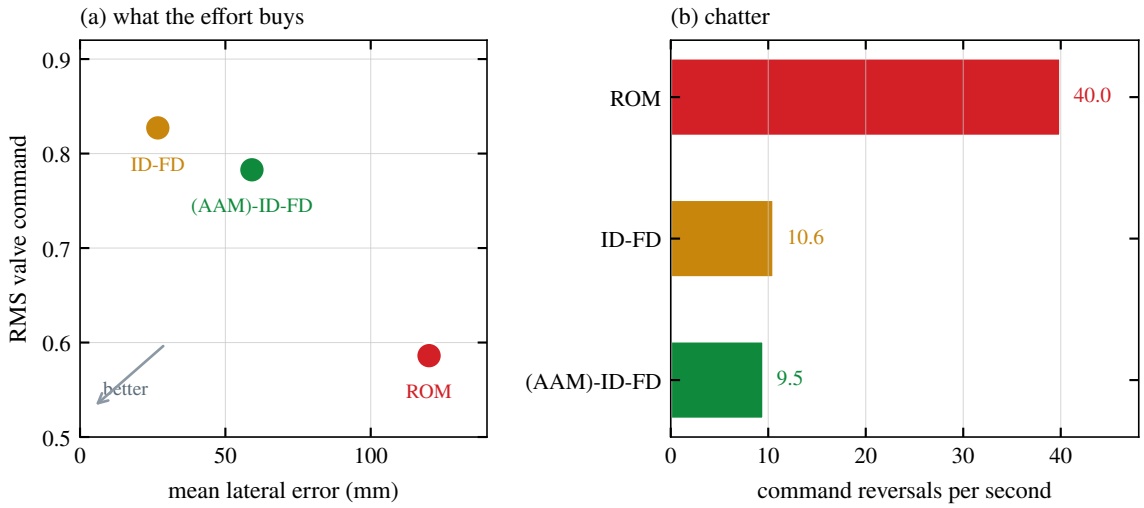

Figure 7.10: (a) Effort against the error it buys; the lower left is better. R spends the least and achieves the least. (b) Command reversals per second, which the root-mean-square does not reveal: R reverses four times as often as either other stack.

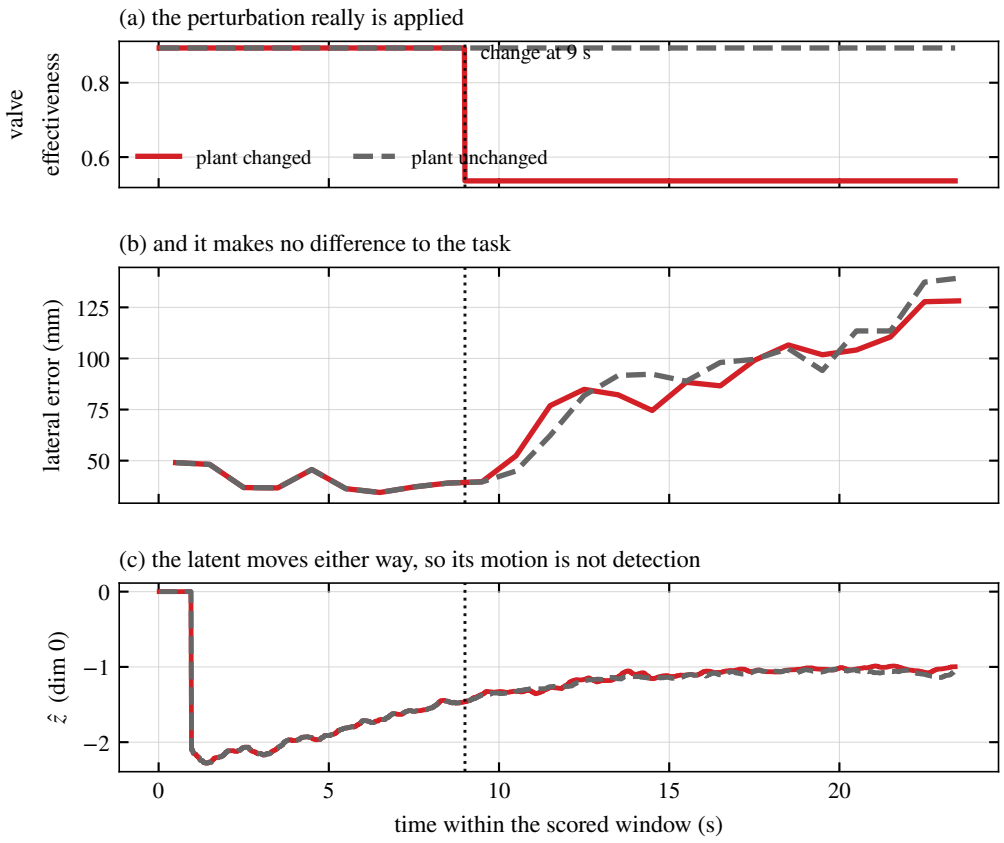

Figure 7.11: Both runs use the same policy, mesh and schedule; in one the valve effectiveness is scaled at 9 s and in the other the schedule fires and the plant is left alone. (a) The perturbation is genuinely applied. (b) The two runs nevertheless track together, and the rise in error after 9 s happens equally without any plant change. (c) The latent moves by much the same amount either way.

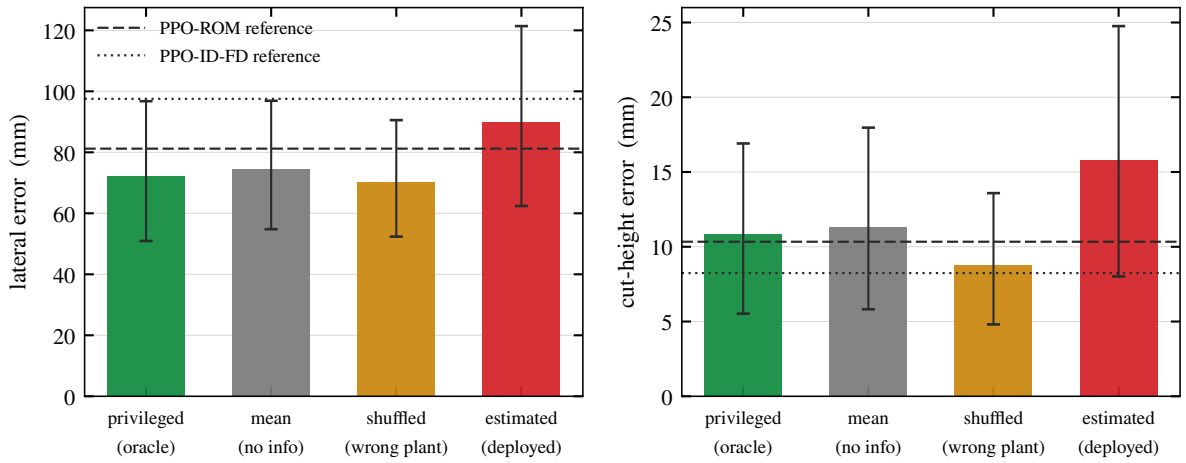

Figure 7.12: What plant knowledge is worth in closed loop. Bars are means over 33 paired cells with 95 % bootstrap intervals; the two reference stacks are drawn as lines. The privileged, mean and shuffled arms are indistinguishable, which is the flat ceiling. The deployed estimate is the worst of the four on both metrics.

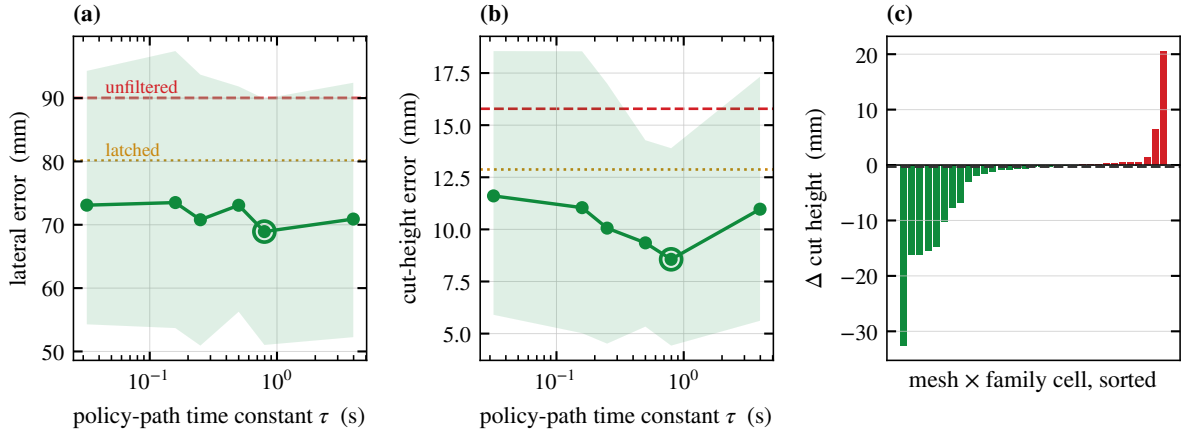

Figure 7.13: Task error against the policy-path time constant. (a) Lateral and (b) cut-height error, mean over 33 cells with a 95 % paired bootstrap band; the dashed line is the unfiltered estimate and the dotted line the latched one, which are the two ends of the axis. The ringed point is the minimum, at $\tau = 0.796\,\mathrm{s}$ on both metrics and on ground contact. (c) Paired per-cell differences for the contrast between $\tau = 0.796\,\mathrm{s}$ and $\tau = 0.032\,\mathrm{s}$ on cut height, sorted, with the median dashed. The improvement is carried by a minority of cells.

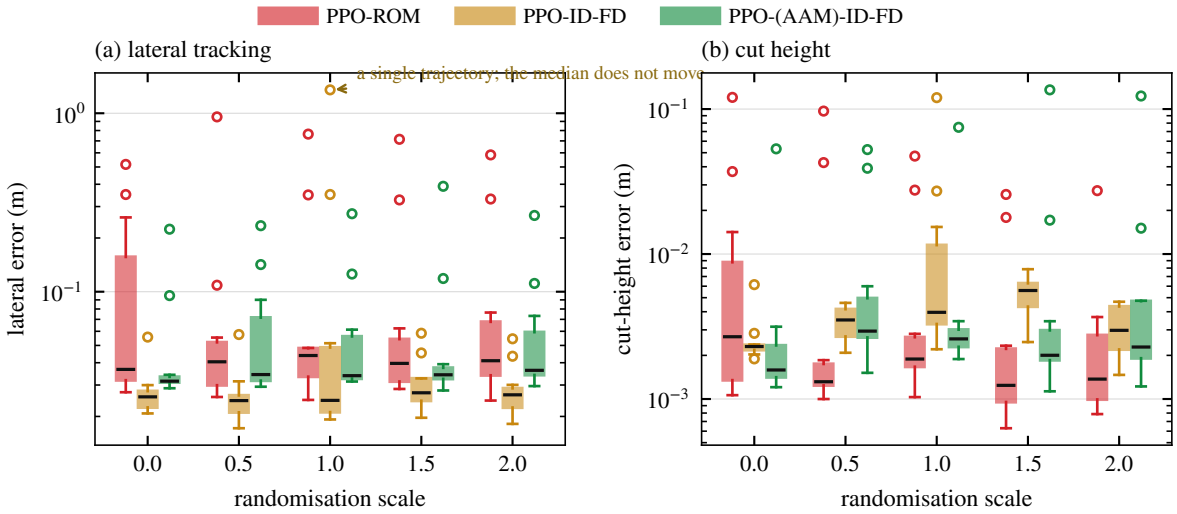

Figure 7.14: Per-trajectory performance at each randomisation scale, over the eleven trajectory families of one held-out mesh. Boxes give the interquartile range and the line the median; points are families outside the whiskers. The medians are flat across the sweep for all three stacks, which is the result. IF's excursion at scale 1 is one trajectory and not a shift of the population, which a plot of means would have reported as a breakdown.

8. Conclusion and Outlook

This thesis asked how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation adds anything to the robustness those learned models already provide. The six research questions of Section 1.4 are answered in the order they were asked, which is also the order in which the answers depend on one another.

8.1 What Was Established

The findings are stated below in the order of the research questions. The first four concern how hydraulic actuation is learned, transferred and controlled. The last two concern online adaptation.

The inverse and forward representation identified in Simscape transfers to the deployment simulation without retraining (RQ1, RQ2). The inverse model was trained on randomised Simscape campaigns and evaluated on 64 IsaacLab deployment runs drawn entirely outside that training distribution, where it reproduces the valve command to within 0.1739 against a recorded signal of RMS magnitude 0.4350, which is roughly 84 % of the valve-command variance. The accuracy is nearly flat across domains, a factor of 1.09 between the best and the worst, and the ordering does not follow plant severity. No deployment-specific retraining was required, and the inverse model stays frozen everywhere downstream. The reduced-order model is the exception and is treated as one throughout: it received a small bounded calibration against the deployment plant, so it is transferred with a correction rather than without one.

Supervised accuracy is not evidence of control authority (RQ1). The first forward model scored near-perfectly and had learned to ignore the valve command, reconstructing motion from the state history instead. Nothing in the validation loss showed it. Ablating the command exposed it, and redesigning the excitation repaired it. Rollout error fell from 75.2 mm to 12.4 mm at 0.5 s and from 157.9 mm to 27.3 mm at 1.0 s, and removing the valve input from the repaired model then costs a factor of 1.81. Excitation design and input ablation are therefore reported here as required tests for a learned actuator

intended for control. This finding concerns how such a model is validated rather than what it represents, so it applies beyond the machine studied here.

Two actuator representations both support closed-loop control, and they do not produce the same controller (RQ3, RQ5). The command-conditioned reduced-order model reaches $R^2 = 0.998$ on the internal valve state and 0.97 on extension, with a diagonally dominant extension–command Jacobian of condition number 2.14, and it runs inside GPU-parallel training. The inverse–forward chain composes two separately learned models in series, which could have compounded their errors and did not. The two representations differ in the control strategy they induce from the same policy interface. R spends the least command and reverses about 40 times per second, against 10.68 for IF, which converts greater actuator effort into lower lateral error. The choice of representation therefore affects the resulting controller and is not an implementation detail.

A learned policy can replace the classical controller at runtime (RQ4). The damped-least-squares controller supplies privileged demonstrations during training and is then removed; the distilled policy emits desired cylinder motion directly into the learned actuator and is the entire deployed controller. Over 33 evaluation cells IF holds a mean lateral error of 70.3 mm, a median of 28.1 mm and 95.29 % ground contact. Reward ascent alone does not deliver this result. Unconstrained fine-tuning raised total return while degrading the primary tracking objective by up to a factor of 12.9, with the critic remaining healthy throughout. The failure is therefore one of objective mismatch rather than of optimisation, and it is the reason the deployed policies are anchored to the demonstrated behaviour.

The non-adaptive closed loop already absorbs the plant variation this work can generate (RQ5). Doubling every deployment-factor deviation moves no stack's median, and a 40 % reduction in valve effectiveness does not move the task either. The sweep was built to locate a breakdown point and finds none within twice the deployment range. This result bounds the disturbance rather than the controllers, and it determines the last finding. Where the closed loop is insensitive to plant variation, knowledge of plant identity has little to contribute.

The remaining findings concern adaptation itself (RQ6).

Under the controller, signal set and excitation regime examined here, plant identity is recoverable only from deliberate excitation, and the limit has a mechanism. Data gathered while this task controller drives the machine does not separate plants, because the command is computed from the state and the two are nearly redundant. This is a statement about the dataset the controller produces, not a claim that closed-loop identification is impossible in general: dedicated closed-loop identification methods exist and were not applied. Open-loop excitation repairs it here. Identity is then legible at low amplitude and gone by amplitude 0.35, and the reason is operating-point drift rather than actuator saturation: by that amplitude the plants have desynchronised in position by half a probe stroke, and a one-second window cannot separate a leaky cylinder from one sitting five centimetres lower. This gives a mechanism-level account of the excitation amplitude at which identification stops working on this plant, which among the excitation-design work reviewed in Chapter 2 is generally left implicit.

The frozen inverse model is sufficient. A state predictor conditioned on an estimated latent, trained without the inverse model in the loop, lowers free-running prediction error by

24.7% against a matched no-latent baseline, in all five seeds (H1, Section 7.3). The residual gap between the training distribution and the deployment plant is recoverable without retraining the inverse model, which is what the frozen-inverse architecture requires. The same comparison on controller-driven data put the latent 1.8% behind, so the excitation result above is what makes this one possible.

The value of the latent is its variation over time, not the plant identity it encodes. Substituting a per-episode constant for the estimate costs 13.6%; substituting a constant that is wrong about the plant costs 0.43%, a factor of about thirty-two between the two (H2, Section 7.4). The estimate carries little plant identity that the consumer uses, and what it does carry is temporal. Whether the same holds for other architectures that infer a plant code is untested here, but it is a cheap thing to check and it is not what such architectures are usually designed around.

No closed-loop benefit of plant knowledge was resolved. A privileged latent read from the true factors, which is the most an estimator could supply *through this encoder, adapter and policy*, improves lateral error by 3.4% and cut-height error by 4.5%, ahead in 18 and 19 of 33 cells (H4, Section 7.9), which is indistinguishable from chance. H3 failed on its own criterion: every paired interval on the three tracking metrics spans zero, so the sign of the deployed estimator's effect is unresolved. That is not equivalence, and it is stated as a failure to resolve rather than as a bound: only ground contact is constrained usefully, to within about 2.4 percentage points, while the tracking intervals are as wide as the baseline means themselves. Two contrasts do exclude zero, and both go against the adaptive stack: hard impacts and p_{95} tool acceleration are worse by 33% and 32%. Both aggregate per mesh, so they rest on three units and indicate a sign rather than a bound.

8.2 What This Means for Design

The four results combine into one design statement, and it is the second contribution of the thesis.

The evidence points at the latent's temporal variability, not at its accuracy, as the dominant limitation in this architecture. A one-step predictor consumes the latent afresh at every tick and is rewarded for its movement. The deployed actor is a memoryless MLP, so it integrates nothing itself; the closed-loop trajectory is what accumulates the consequences of an action that changes as the latent moves under it. This reading reconciles the model-level gain with the closed-loop result, and it matches the ranking of the four latent conditions: the deployed estimate finishes behind both a constant carrying no plant information and a constant that is wrong about the plant, while varying within an episode by about nine times the spread between plants. Those arms differ in information and in variability at once, so a bandwidth sweep was run to separate them (Section 7.10). Filtering the latent on the policy path alone, at fixed content, produces an optimum at $\tau = 0.796\,\mathrm{s}$ on all three metrics, and slowing the latent from $0.032\,\mathrm{s}$ to that value improves cut-height tracking by 14.5% on the median in 22 of 33 cells ($p = 0.015$). Lateral error moves the same way and does not resolve.

The design suggested by that reading is to split the two paths: hold or filter the latent where a feedback policy consumes it, and leave it responsive where a one-step model consumes it. The sweep supports it on one metric and one checkpoint, which is the weakest

form of support that is still evidence. Two things it does not establish should be carried with it. The temporal variance of the latent as the actor reads it does not order with τ, so the proposed cause is not the measured one, and the optimum may belong to a policy trained on the unfiltered signal rather than to the architecture.

Run the two pre-deployment measurements first. The identifiability envelope establishes whether plant identity is present in the data the machine produces. The oracle ceiling establishes whether the controller's behaviour depends on what that identity describes. Both are cheap, both precede estimator training, and either can end the effort before it is incurred. On this task the second alone would have shown that no estimator feeding this conditioning architecture, however accurate, recovers a resolvable improvement, and it would have shown it before the estimator, the adapter and the conditioned policy were built. This is the thesis's principal contribution: a decision procedure that converts an open-ended engineering effort into a measurement.

8.3 Contributions

Section 1.4 stated five contributions. Their standing at the end of the work is as follows.

Transferable hydraulic actuator learning is delivered. Both representations were identified from Simscape and realised in IsaacLab, and the inverse model transfers without deployment-specific retraining.

The **control-relevant identification and validation** procedure is delivered, and it is the contribution most likely to transfer independently of this machine. It covers both the input-ablation test that exposed a forward model with no command authority and the identifiability audit that established the excitation envelope, including the mechanism behind the upper amplitude limit.

The **reduced-order actuator model** is delivered and carries everything downstream, being accurate enough to serve as the deployment plant and roughly thirty times faster than the high-fidelity model it replaces in the training loop. The qualification stated throughout stands. It was calibrated against IsaacLab and is therefore the reference condition rather than an independent competitor to the inverse and forward pair.

Policy-only control through learned hydraulics is delivered for both representations, with the classical controller absent from the deployed loop.

The measurements of **robustness and the limits of adaptation** are delivered, and their main product is the discipline that produced them rather than any single number. Two findings exist only because of it: that a selection window covering 8.9 of 26 m certifies a third of an episode, and that a quantity which is gated but not priced is traded away by the optimizer. The causal plant estimator and the bounded adaptation path behave as specified, with no privileged access, no future information, bounded output, and reversion to the non-adaptive actuator path on invalid input. They remain enablers of this contribution rather than claims in their own right.

One element is graded down. The **eight-gate acceptance contract** was intended as a system-level specification and does not yet function as one. Under the protocol of Chapter 7 no stack passes all eight gates, because all three fail the lateral gate on the hardest of eleven trajectory families, and the hard-impact gate is passed comfortably by

a do-nothing policy (Section 3.9.1). Six of the eight discriminate, and the comparative claims of this work rest on those six. Repairing the other two is specification work rather than control work.

8.4 Limitations

The evidence is simulation-to-simulation, and the deployment plant is itself a learned model. No claim here is a claim about hardware.

The differential pressure the estimator consumes is produced by the actuator model rather than measured, so transferring the model-level result to hardware requires instrumentation the FME 500 does not carry. An observation contract remains unresolved alongside this: every result uses the pressure-inclusive row, while the stricter deployable alternative excludes measured differential pressure. Since the estimator's advantage is concentrated in that channel, the deployable variant should be expected to perform worse, and by how much is not established.

The closed-loop comparisons resolve differences to a few per cent over 33 paired conditions and do not establish equivalence. The available plant variation bounds the disturbance rather than the architecture: a stack shown to absorb a 40% reduction in valve effectiveness and a doubling of every deployment factor has been shown to tolerate that much, not more. The three stacks are matched on evaluation but not on training budget, so the comparison describes three deployed controllers rather than three equally tuned training procedures.

Finally, the bound is a bound on this loop. A commanded step cannot complete faster than 417 ms at the command rate limit while the plant answers in 117 ms, so the task response is set downstream of anything the latent influences. That is close to the least favourable case for adaptation of this kind. The finding generalises through the mechanism rather than through the number.

8.5 Outlook

Four directions follow, ordered by how directly the evidence supports them.

The split-path latent design is first. It is a contained architectural change, it follows from a measured mechanism, and it is testable with instruments already built: the four-way latent substitution isolates it, and the two consumers are already separate in the implementation. If the mechanism is correct, filtering the latent on the policy path should move the deployed estimate ahead of the uninformative constant while leaving the 13.6% model-level result intact.

The second is to move the task rather than the plant. Every perturbation applied here left the controller inside its envelope, so the disturbance results bound the interior and not the edge. A perturbation the commanded line cannot absorb would locate the boundary at which adaptation begins to pay, and that boundary determines whether the architecture is worth carrying on a machine.

The third is pressure instrumentation, without which the one channel carrying hydraulic information stays a model output. The stricter no-pressure contract can be quantified in simulation first, by retraining the estimator on that row.

The fourth is the acceptance contract. The lateral threshold needs re-deriving against the widened trajectory library, and the hard-impact gate needs replacing with a manufacturer damage specification. Until both are done, no comparison on this task can be certified, including a favourable one.

A. System Parameters and Coordinate Mappings

The kinematic structure, the cylinder-to-joint mapping and the calibrated cutter Jacobian are derived in Section 3.2 and are not repeated here. This appendix records the quantities a reader needs in order to reproduce a run and cannot read off those derivations: the rate of every layer, the limits the command path enforces, and the geometry of the scored course. Every value below is read from the shipped configuration or from a stored evaluation record, and the source is named in each case.

A.1 Rates

Five different rates operate in this system and confusing them is the most common way to misread a result. The control step is 1/60 s, the physics substep is half that because the environment runs with a decimation of two, and every learned component operates on the 20 Hz grid at which the reduced-order model and the identification data were sampled.

Table A.1: The rate of every layer. Control and physics rates are read from the stored evaluation records (`control_dt_s`); the learned-model sample time is stored inside the frozen forward-model checkpoint (`sample_time_s`); the AAM update period is the runtime configuration `update_period_ticks`.

Layer	Rate	Period
PhysX physics substep	120 Hz	8.33 ms
Policy and control step	60 Hz	16.67 ms
Inverse / forward model	20 Hz	50 ms
AAM inference update	20 Hz	50 ms (every 3rd control step)
Simscape logging grid	20 Hz	50 ms

Two derived latencies follow from Table A.1 and are used in Section 7.5. The AAM consumes a history of 20 rows on the 50 ms grid, so its input window is exactly 1.0 s and no estimate at all is available for the first 1.0 s after a reset; until then the adapter is bypassed

and the frozen inverse action is returned unchanged. Its output smoother averages five consecutive predictions, which adds a further 250 ms of group delay.

A.2 Command-Path Limits

The command path enforces four limits, all of which bound the policy rather than advise it. They are listed in Table A.2.

Table A.2: Limits enforced on the command path, per cylinder where applicable. Valve command limits and the adapter residual bound are stored inside the frozen adapter checkpoint (`command_limits`, `residual_bound`); lead caps and the c3 reserve are read from the stored evaluation records.

Limit	c1	c2	c3	c4	Meaning
Valve command limit	0.95	0.95	0.80	0.80	accumulated command is clipped here
Adapter residual bound	0.15	0.15	0.15	0.15	maximum adapter correction
Target lead cap (m)	0.03	0.03	0.03	0.02	how far the target may lead the state
c3 extension reserve	—	0.008 m on c3			solver-enforced stroke reserve

The lead cap is the mechanism by which the reference governor prevents the commanded target from running away from the measured state, and the c3 reserve is enforced by the solver while headroom is still scored against the original stroke, so the gate is not satisfied by moving the goalposts. The adapter residual bound is what makes the adaptive path fail safe: an adapter output that is invalid, uninitialised or late is replaced by zero, which returns the stack to the frozen-inverse configuration rather than to an untested one.

A.3 Course and Episode Geometry

Every closed-loop result in Chapter 7 uses the geometry in Table A.3. The scored window is the full course: the count of control steps differs between evaluations only because the carrier speed does, and it is the travelled distance rather than the step count that defines the window.

Table A.3: Course geometry and the two evaluation speeds that appear in this thesis. Values are read from the stored evaluation records (`tracking_active_length_m`, `max_truck_travel_m`, `steps`).

Quantity	Chapter 7 protocol	Earlier 2 km/h records
Tracking line length	26.0 m	26.0 m
Carrier speed	4.0 km/h	2.0 km/h
Control steps scored	1403	2808
Truck travel achieved	25.98 m	26.00 m
Episode length	47.3 s	46.8 s
Trajectory families	11	8

The right-hand column is included because stored records under it exist in the project and are not comparable to the results of Chapter 7; Section 7.6 explains why one such record, which passes all eight gates, is not used.

B. Dataset Manifests and Factor Ranges

B.1 Trajectories

Each run receives its own reference trajectory, sampled rather than scripted. Between twelve and twenty waypoints are drawn by Latin hypercube sampling within the joint limits, and the arm moves between them without dwelling, so that the data contains transitions rather than long static holds. Segment durations are drawn from four speed classes, slow (1.5–2.5 s), medium (0.8–1.5 s), fast (0.3–0.7 s), and variable (0.5–2.5 s), and about a third of the runs receive additional exploratory perturbations on top of the waypoint path.

An earlier version of this generator used four to eight waypoints with dwell periods between them. It produced data in which the arm was stationary for a large fraction of the time, which flatters any model that learns to predict persistence. The current settings were chosen to remove that easy mass of near-static samples, and the increase in waypoint count is the single change that most improved the usefulness of the batch.

B.2 Execution and Logging

Each run simulates 22 s of operation. The solver is the stiff implicit `ode23t` integrator in accelerator mode with relative and absolute tolerances of 10^{-2} and 10^{-3}. The loose tolerances are a deliberate throughput compromise, and their effect is bounded by the consistency checks of Section 4.10. Output is written on a fixed 0.05 s grid, giving 441 logged samples per run, and the batch is checkpointed to disk every hundred runs so that a failure late in a multi-day batch does not cost the whole collection.

For every run the archive stores the time vector, the five valve commands, the four cylinder extensions, the four chamber pressure differences, the four rod forces, the five joint positions and velocities, the reference position and velocity signals, and, crucially, the complete parameter structure that produced the run. That last item is what makes the batch usable for the latent-adaptation work: the privileged plant descriptor $\boldsymbol{\xi}$ of Section 2.6 is not something that has to be reconstructed later, it is recorded alongside the trajectory.

The cost quantifies the argument made in Section 2.1.3. A single 64 s open-loop run takes roughly 2000 s of wall time on the collection machine, a real-time factor of about 0.03. The Simscape plant runs some thirty times slower than the machine it models. No policy-optimisation loop that needs millions of environment steps can be built on that, which is why the learned actuator surrogates exist.

B.3 Parameter Ranges and Dataset Audit

Table B.1: Randomized hydraulic parameters. The seal-wear coordinate is drawn independently for each of the four cylinders and drives three physically linked quantities at once.

Parameter	Nominal	Randomized range	Acts on
Entrained air fraction ε_{air}	0.005	0.001 – 0.080	fluid compliance
Dead-volume fraction ε_{vol}	0.020	0.005 – 0.080	fluid compliance
Discharge coefficient c_d	0.70	0.56 – 0.84	valve flow gain
Seal wear $w_{\text{seal},i}$	0	0 – 0.25 (per cylinder)	clearance, leakage, spool lag
Relief setting p_{set}	27.6 MPa	24.8 MPa – 30.4 MPa	pressure ceiling

Table B.2: Randomized mechanical parameters. The rigid-body perturbations are applied independently to each of the fourteen bodies.

Parameter	Nominal	Randomized range
Body mass	CAD value	±20 %
Centre-of-mass offset	CAD value	±5 %
Inertia tensor and products	CAD value	consistent with mass draw
Joint stiffness, J1 and J2	60 kN/m	40 – 120 kN/m
Joint stiffness, J3 and J4	60 kN/m	40 – 120 kN/m
Joint stiffness, J5	25 kN/m	15 – 60 kN/m
Joint damping	250	100 – 600

Table B.3: The six randomized physical factors of the IsaacLab adaptation collection and their final ranges. One factor vector is drawn per run and held fixed for the whole run.

Coordinate	Range	Domain	Acts on
e_{flow}	$[-0.25, 0.25]$	hydraulic	valve flow gain
$e_{\text{compliance}}$	$[-0.25, 0.10]$	hydraulic	effective chamber compliance
e_{seal}	$[0, 1]$	hydraulic	seal wear, leakage, spool lag
e_{pressure}	$[-0.15, 0.15]$	hydraulic	supply/relief pressure level
e_{mass}	$[0.8, 1.5]$	mechanical	cutter mass scale
e_{COM_z}	$[-0.10, 0.05]$	mechanical	cutter centre-of-mass height

B.4 The Frozen Adaptation Collection

The IsaacLab adaptation collection is frozen, and every model-level result in this thesis is fitted and scored on it. Its manifest records 118 runs, one factor vector per run, held fixed

Table B.4: Audit of the strict open-loop excitation export. All checks are computed by the extraction stage and stored with the data; the export is accepted only if every one of them passes.

Check	Result	Interpretation
Rows, runs, columns	25 600 / 4 / 49	four complete runs
Median sample time	0.01 s	as commanded
Maximum sample-time error	9.1×10^{-15} s	numerically exact grid
Maximum Δextension error	0	targets match state differences
Maximum Δvelocity error	0	targets match state differences
Maximum $\Delta(\Delta p)$ error	0 Pa	targets match state differences
Non-finite values	0	no solver escapes
Valid fraction	0.937	startup, gaps, shutdown excluded
Command rank	4	all four channels independent
Lagged excitation rank	80	history matrix full rank
Valve saturation	8.5 % per channel	limits reached but not dominant
Stroke coverage	12 % – 33 %	above floor, below target

for the duration of that run. The domain composition and the run-level split are given in Table B.5.

Table B.5: Composition of the frozen adaptation collection, counted directly from the manifest. The run-level split is 102 training runs against 16 held-out test runs; the training portion is further divided 82/20 into fitting and validation runs, so that no window from a validation or test run is ever seen during fitting.

Domain	Runs	Factors varied
`combined`	64	all six
`hydraulic`	40	flow, compliance, seal, pressure
`mechanical`	12	mass scale, centre-of-mass height
`nominal`	2	none (factors at nominal)
Total	118	
`train` split	102	further split 82 fit / 20 validation
`test` split	16	held out

The split is at *run* level rather than window level, and this is the single most important property of the collection. Within one run the factor vector is constant, so windows drawn from the same run are highly redundant; a window-level split would place near-duplicates of the same plant on both sides of the boundary and every held-out score in this thesis would be optimistic. The manifest stores the factor key of every run alongside its split label, so this property is checkable rather than asserted.

C. Model Architectures, Normalizers and Checkpoint Hashes

This appendix pins the frozen artifacts. Every model-level and closed-loop result in this thesis was produced with the exact files listed in Table C.1, and the SHA-256 digests are given so that a later reader can establish whether a file on disk is the one that produced a number here. This matters more than usual in this project, because resumed training runs overwrite checkpoints in place.

Table C.1: Frozen artifacts and their SHA-256 digests (first 32 hex digits shown). "Frozen" means the file's weights were not updated by any subsequent stage; the three policy checkpoints are the ones evaluated in Chapter 7.

Artifact	Role	SHA-256 (truncated)
`aam.pt`	AAM, frozen	`cd6a7f0b118b96c7ce66b889b9b3e422`
`zhat_adapter.pt`	ID adapter, frozen	`30629a4f83db5c2bed3c777bcf82dc57`
`best_id_cylinder_control.pt`	inverse model, frozen	`8e26c52e82d89efe193e001d0a8b07a6`
`best_fd_cylinder_strict.pt`	forward model, frozen	`1f41e44309a8f320430a3382ca88ce5f`
`headroom_priced_v1/model_299.pt`	AIF policy	`aaf6b4bab7cfeac53fb95a3d15470cde`
`rom_s6_aligned/model_598.pt`	R policy	`ce7e3ab9649b1193f123f9f91107bbf8`
`idfd_s6_aligned/model_598.pt`	IF policy	`06533b8e8614b0ad982dd8c238a17772`

C.1 The Inverse and Forward Models

Both learned actuator models share one architecture: a linear input projection to 512 units, four pre-norm residual blocks of two 512-wide linear layers each, a final normalisation and a linear output projection. They differ only in their input and output layouts, given in Table C.2. Each carries just over 2.12 million parameters. Both are trained on the Simscape campaign, both operate on the 50 ms grid, and both are frozen for every result in Chapters 6 and 7.

Only the predicted extension increment of the forward model is integrated in the control loop. Its velocity output failed validation on two axes (Section 5.3) and is carried as a diagnostic rather than used.

Table C.2: Input and output layouts of the two frozen actuator models, read from the `input_layout` and `output_layout` fields stored in the checkpoints themselves.

	Inverse model	Forward model (strict)
Input width	28	20
Input layout	extension(4), velocity(4), $\Delta p(4)$, force(4), target Δextension(4), target Δvelocity(4), $\Delta t(4)$	valve(4), extension(4), velocity(4), $\Delta p(4)$, force(4)
Output width	4	12
Output layout	valve command(4)	Δextension(4), Δvelocity(4), $\Delta(\Delta p)(4)$
Target mode	normalised Δ valve command	normalised Δ cylinder state
Parameters	2 124 292	2 124 300

C.2 The Adaptation Module

The AAM is a causal one-dimensional convolutional network over the last 20 rows of the 50 ms deployment observable. The input row is 18 columns wide,

$$h_t = \Big[\, q(5),\ \dot{q}(5),\ \Delta p(4),\ u_{t-1}(4) \,\Big],$$

and the trunk is four convolutional stages of width 64 and kernel three, the first carrying a 1×1 skip projection from 18 to 64 channels, each stage normalised. A two-layer head maps the final 64-channel activation to the 2-dimensional latent $\hat{z}$. The whole module is 42 562 parameters, which is what makes 20 Hz inference affordable alongside 256 parallel environments.

The latent dimension is 2 because the module regresses onto the frozen state predictor's latent, which is 2-dimensional by construction. This is the structural reason the deployed latent decodes only one of the six physical factors, and Section 7.4 treats it as a finding rather than a configuration choice.

The ID adapter is deliberately small: a two-layer network of width 64, mapping a 30-dimensional feature row to a 4-dimensional residual, 76 stored tensors' worth of normalisation statistics included. Its feature row is the inverse model's 28 inputs concatenated with the 2 latent components, which is the pressure-inclusive contract discussed in Section 7.14. Its output is bounded at 0.15 per cylinder (Table A.2) and scaled by a declared residual factor, 0.25 for every result reported here.

The adapter checkpoint additionally stores the digests of the inverse and forward models it was fitted against, so a latent adapter cannot silently be paired with a different frozen actuator than the one it was trained for.

D. PPO and IsaacLab Configuration

D.1 The Observation Vector

The policy observation is 66 columns for R and IF, and 68 for AIF: the same 66 with the two latent components appended. Table D.1 gives the layout by index. The final twelve columns are the model-error block, and they exist because the policy is a memoryless multilayer perceptron: it already observes the actuator's prediction and the current measured extension, but forming the error between them requires differencing across two steps, which a feedforward network cannot do. Without these columns the standing model error of a learned actuator is invisible to the policy no matter how long it trains.

Channel 46 caused a campaign-scale failure. It carries truck speed, and with a frozen observation normaliser inherited from a fixed-speed run its recorded standard deviation was zero. The normaliser divides by that standard deviation plus a small constant, which multiplied the channel by roughly one hundred everywhere except at the speed the statistics were collected at. The policy scored perfectly at that one speed and collapsed at every other. Freezing a normaliser is correct when resuming; freezing one whose statistics were gathered under a narrower distribution than the one about to be trained on is not.

D.2 The Action

The action is four-dimensional in every stack: a desired cylinder motion, not a valve command. What converts it to a valve command is the actuator under test, which is the only free variable in the three-way comparison. The limits enforced between the action and the plant are given in Table A.2.

D.3 The Reward

The ten reward terms and their shaping functions are given in Table 6.2 and are not repeated here. Two weights behind that table are the ones a reproduction is most likely to get wrong: contact loss is weighted at 20.0 and hard impacts at 20.0, against a support reward of 4.0. The hard-impact weight was introduced after the measurement described

Table D.1: Policy observation layout. Indices are zero-based and half-open. Columns 0–53 are the task block, 54–65 the model-error block, and 66–67 exist only for AIF.

Index	Width	Contents
0–9	9	verge height profile ahead of the cutter
9	1	cut-height error
10	1	lateral error
11	1	cutter vertical velocity
12–16	4	measured cylinder extensions
16–20	4	measured cylinder velocities
20–24	4	cylinder headroom, low side (normalised)
24–28	4	cylinder headroom, high side (normalised)
28–32	4	previous applied action
32–36	4	current valve commands
36–40	4	actuator-predicted Δextension
40–44	4	commanded target lead
44	1	head-down alignment
45	1	head-down alignment rate
46	1	truck speed
47	1	scan valid fraction
48–50	2	world-XY components of the cutter's local $+Z$ axis
50	1	normalised cutter–terrain interaction force
51–54	3	path preview at 0.5, 1.0, 2.0 m ahead
54–58	4	cylinder tracking residual (measured − commanded)
58–62	4	actuator prediction residual
62–66	4	normalised cylinder effort
66–68	2	$\hat{z}$, AIF only

in Section 3.9.1: the quantity was gated but not priced, and the optimizer traded it away while its return rose.

D.4 PPO Hyperparameters

Table D.2 lists the optimisation settings of the three evaluated runs, read from their stored run manifests rather than from the scripts that launched them. The manifests share a schema, which is what makes the comparison in Section 7.1 checkable; the fields on which the runs differ are exactly those discussed there.

Table D.2: PPO configuration of the three evaluated runs, from `run_manifest.json`. A dash marks a field the manifest does not record. Differences between the adaptive run and the matched pair are discussed in Section 7.1.

	R	IF	AIF
manifest schema		`terrain_shared_v2`	
seed	17	17	41
terrain base seed	17	17	41
environments	256	256	256
curriculum stage	6	6	6
actor learning rate	3×10^{-4}	3×10^{-4}	3×10^{-6}
entropy coefficient	0.001	0.001	0.001
frozen-actor warm-up	150	150	0
observation normaliser	freeze	freeze	freeze
observation width	66	66	68
discount γ	—	—	0.97
GAE λ	—	—	0.95
steps per environment	—	—	32
episode length	—	—	47.3 s
truck speed	—	—	2–7 km/h, per episode
residual scale	—	—	0.25
trajectory families		all 11	
actor parameters	—	—	59 080
critic parameters	—	—	58 881

The dashes are informative and are left visible rather than filled in from the launch scripts. The two reference runs' manifests do not record their truck speed schedule, discount or GAE settings, so those cannot be verified from the frozen record; the adaptive run's manifest does record them. Section 7.1 states the consequence.

E. Supplementary Results

E.1 Per-Family Results on the Reference Mesh

Table E.1 gives the per-family breakdown behind the pooled means of Table 7.8, on mesh 1337, for the adaptive stack. It is included because the pooled statistics of Chapter 7 are carried by a small number of cells, and the reader should be able to see which.

Table E.1: AIF per-family results on mesh 1337, full 26 m path at 4 km/h, checkpoint `headroom_priced_v1/299`. Families are ordered by lateral error. Nine of eleven sit near 0.03 m; the distribution is not approximately uniform and its mean is not a typical value.

Family	lateral (m)	cut height (m)	ground contact
`s_curve`	0.0288	0.0014	0.9768
`lateral_sine`	0.0293	0.0012	0.9754
`straight`	0.0309	0.0015	0.9694
`lane_change_hold`	0.0312	0.0026	0.9629
`gaussian_slalom`	0.0313	0.0015	0.9701
`random_spline`	0.0315	0.0020	0.9622
`boom_sweep`	0.0318	0.0016	0.9729
`fourier_meander`	0.0324	0.0013	0.9786
`lateral_chirp`	0.0342	0.0032	0.9615
`double_s_curve`	0.0951	0.0021	0.9772
`offset_return`	**0.2240**	**0.0532**	**0.8485**

Two features of Table E.1 govern how Chapter 7 should be read. The first is the gap between the ninth and eleventh rows: a factor of seven in lateral error separates the typical family from the worst one, so the worst-family aggregation used by the acceptance gates is a different measurement from the mean, not a conservative version of it. The second is that `offset_return` is the family that fails, and it fails on all three metrics at once. Lateral error, cut height and ground contact degrade together, which is the signature of the cutter leaving the ground rather than of a tracking error alone.

E.2 The Latent Substitution Arms

The four latent conditions of Section 7.9 share one checkpoint and one set of weights, and differ only in what occupies the latent slot. This is the comparison that carries the most weight in this thesis, because it is immune to the training-condition asymmetries of Section 7.1: no training differs between the arms, since no training occurs.

Table E.2: The four latent arms, as evaluated. All four use checkpoint `headroom_priced_v1/299` with identical weights; only the value supplied in the latent slot changes.

Arm	Latent supplied
`estimated`	the deployed AAM estimate, updated at 20 Hz
`true`	privileged encoding of the run's actual factor vector
`mean`	a constant carrying no plant information
`shuffled`	a constant belonging to a *different* plant

The ordering of these four arms is the mechanism result of this thesis. The `shuffled` arm is the control that distinguishes the two candidate explanations: if the deployed estimate were losing because it carries the wrong plant identity, then `shuffled`, which is by definition wrong about the plant, would lose by at least as much. It does not. What separates the estimate from both constants is not its information content but its movement.

E.3 Reproduction

Every closed-loop number in Chapter 7 is produced by the experiment scripts stored with the project under `experiments/E1` through `experiments/E5`, each of which records the checkpoint, the mesh set, the family set, the speed and the scored horizon in its own manifest. The model-level results come from the frozen collection of Appendix B.4. The artifacts named in Table C.1 are sufficient, together with those scripts, to regenerate every table in this thesis.

Bibliography

[Åström 95] K. J. Åström, B. Wittenmark, *Adaptive Control*, 2nd ed., Reading, MA: Addison-Wesley, 1995.

[Ba 16] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization", arXiv preprint arXiv:1607.06450, 2016.

[Bengio 15] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. 2015, pp. 1171–1179.

[Benner 15] P. Benner, S. Gugercin, K. Willcox, "A Survey of Projection-Based Model Reduction Methods for Parametric Dynamical Systems", *SIAM Review*, vol. 57, no. 4, pp. 483–531, 2015.

[Cho 14] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[Chua 18] K. Chua, R. Calandra, R. McAllister, S. Levine, "Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31. 2018.

[Egli 20] P. Egli, M. Hutter, "Towards RL-Based Hydraulic Excavator Automation", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 2692–2697.

[Egli 22] P. Egli, M. Hutter, "A General Approach for the Automation of Hydraulic Excavator Arms Using Reinforcement Learning", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5679–5686, 2022.

[Elfwing 18] S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", *Neural Networks*, vol. 107, pp. 3–11, 2018.

[He 16] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[Hochreiter 97] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[Hu 26] P. Hu, T. Wen, D. Zhang, H. Chen, J. Gong, "Bayesian reinforcement learning for adaptive control of energy recuperation in hydraulic excavator arms", *Scientific Reports*, vol. 16, p. 6195, 2026.

[Huber 64] P. J. Huber, "Robust Estimation of a Location Parameter", *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[Hwangbo 19] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, "Learning Agile and Dynamic Motor Skills for Legged Robots", *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[Isermann 11] R. Isermann, M. Münchhof, *Identification of Dynamic Systems: An Introduction with Applications*, Berlin, Heidelberg: Springer, 2011.

[Jelali 03] M. Jelali, A. Kroll, *Hydraulic Servo-systems: Modelling, Identification and Control*, London: Springer, 2003.

[Johannink 19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, S. Levine, "Residual Reinforcement Learning for Robot Control", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 6023–6029.

[Kakade 02] S. Kakade, J. Langford, "Approximately Optimal Approximate Reinforcement Learning", in *Proceedings of the 19th International Conference on Machine Learning (ICML)*. 2002, pp. 267–274.

[Karniadakis 21] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, "Physics-Informed Machine Learning", *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[Kingma 15] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization", in *International Conference on Learning Representations (ICLR)*. 2015.

[Kumar 21] A. Kumar, Z. Fu, D. Pathak, J. Malik, "RMA: Rapid Motor Adaptation for Legged Robots", in *Proceedings of Robotics: Science and Systems (RSS)*. 2021.

[Lee 20] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, "Learning Quadrupedal Locomotion over Challenging Terrain", *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[Liu 25] X. Liu, C. Lu, H. Wu, B. Hu, X. Li, Z. Li, X. Guo, "FRMA: Four-Phase Rapid Motor Adaptation Framework", *Machines*, vol. 13, no. 10, p. 885, 2025.

[Ljung 99] L. Ljung, *System Identification: Theory for the User*, 2nd ed., Upper Saddle River, NJ: Prentice Hall, 1999.

[Lutter 19] M. Lutter, C. Ritter, J. Peters, "Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning", in *International Conference on Learning Representations (ICLR)*. 2019.

[Maitan 26] L. Maitan, L. Toschi, C. Zanette, E. G. Vergamini, L. F. Santos, T. Boaventura, "Contraction Metric Based Safe Reinforcement Learning Force Control for a Hydraulic Actuator with Real-World Training", arXiv preprint arXiv:2602.08977, 2026.

[Makoviychuk 21] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning", arXiv preprint arXiv:2108.10470, 2021.

[Mattila 17] J. Mattila, J. Koivumäki, D. G. Caldwell, C. Semini, "A Survey on Control of Hydraulic Robotic Manipulators With Projection to Future Trends", *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 2, pp. 669–680, 2017.

[McKay 79] M. D. McKay, R. J. Beckman, W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code", *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[Merritt 67] H. E. Merritt, *Hydraulic Control Systems*, New York: John Wiley & Sons, 1967.

[Mittal 23] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments", *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[MULAG Fahrzeugwerk 26] MULAG Fahrzeugwerk, "FME 500 Front Mower — Product Documentation", Heinz Wössner GmbH u. Co. KG, Oppenau, Germany. `https://www.mulag.de/en/roadside-equipment/products/unimog-equipment/front-mower/fme-500/`, 2026. Accessed July 2026.

[Nagabandi 18] A. Nagabandi, G. Kahn, R. S. Fearing, S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 7559–7566.

[Nakamura 86] Y. Nakamura, H. Hanafusa, "Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control", *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[Nguyen-Tuong 11] D. Nguyen-Tuong, J. Peters, "Model Learning for Robot Control: A Survey", *Cognitive Processing*, vol. 12, no. 4, pp. 319–340, 2011.

[Paszke 19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. 2019, pp. 8024–8035.

[Peng 18] X. B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810.

[Rajeswaran 18] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, "Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations", in *Proceedings of Robotics: Science and Systems (RSS)*. 2018.

[Ross 11] S. Ross, G. J. Gordon, J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning", in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, pp. 627–635.

[Rudin 22] N. Rudin, D. Hoeller, P. Reist, M. Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", in *Proceedings of the Conference on Robot Learning (CoRL)*. 2022, pp. 91–100.

[Schoukens 19] J. Schoukens, L. Ljung, "Nonlinear System Identification: A User-Oriented Road Map", *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 28–99, 2019.

[Schulman 15] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, P. Abbeel, "Trust Region Policy Optimization", in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. 2015, pp. 1889–1897.

[Schulman 16] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation", in *International Conference on Learning Representations (ICLR)*. 2016.

[Schulman 17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms", arXiv preprint arXiv:1707.06347, 2017.

[Siciliano 09] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, London: Springer, 2009.

[Silver 18] T. Silver, K. Allen, J. Tenenbaum, L. Kaelbling, "Residual Policy Learning", arXiv preprint arXiv:1812.06298, 2018.

[Slotine 91] J.-J. E. Slotine, W. Li, *Applied Nonlinear Control*, Englewood Cliffs, NJ: Prentice Hall, 1991.

[Sobol' 67] I. M. Sobol', "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals", *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, 1967.

[Sofrony 24] J. Sofrony, M. C. Turner, C. M. Richards, "Anti-windup compensation for model reference adaptive control schemes", *International Journal of Robust and Nonlinear Control*, vol. 34, no. 15, pp. 10176–10193, 2024.

[Spinelli 24] F. A. Spinelli, P. Egli, J. Nubert, F. Nan, T. Bleumer, P. Goegler, S. Brockes, F. Hofmann, M. Hutter, "Reinforcement Learning Control for Autonomous Hydraulic Material Handling Machines with Underactuated Tools", in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024. arXiv:2410.05093.

[Srivastava 14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[Sutton 18] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: MIT Press, 2018.

[The MathWorks, Inc. 24a] The MathWorks, Inc , "Simscape Fluids: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-fluids.html`.

[The MathWorks, Inc. 24b] The MathWorks, Inc , "Simscape Multibody: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-multibody.html`.

[Tobin 17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30.

[von Szadkowski 20] K. von Szadkowski, S. Reichel, "Phobos: A Tool for Creating Complex Robot Models", *Journal of Open Source Software*, vol. 5, no. 45, p. 1326, 2020.

[Wampler 86] C. W. Wampler, "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods", *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[Yamamori 26] S. Yamamori, K. Ishihara, K. Minamikawa, R. Ohmori, T. Yasaki, N. Sugimoto, J. Morimoto, "Actuator Reality Shaping for Zero-Shot Sim-to-Real Robot Learning", arXiv preprint arXiv:2607.02205, 2026.

[Yang 26] N. Yang, C. Ji, J. Chen, H. Zheng, "State-Constrained Control for Hydraulic Manipulator Position Servo Systems with Valve Dead-Band Compensation", *Actuators*, vol. 15, no. 4, p. 196, 2026.

[Yin 25] P. Yin, T. Westenbroek, S. Bagaria, K. Huang, C.-A. Cheng, A. Kolobov, A. Gupta, "Rapidly Adapting Policies to the Real World via Simulation-Guided Fine-Tuning", arXiv preprint arXiv:2502.02705, 2025.

[Yu 26] S.-Y. Yu, J.-W. Wang, M. Horii, M. Tomizuka, V. Garg, "Learning Context-Aware Neural ODE Dynamics for Adaptive Robotic Control", arXiv preprint arXiv:2606.15469, 2026.